



BeagleV-Fire



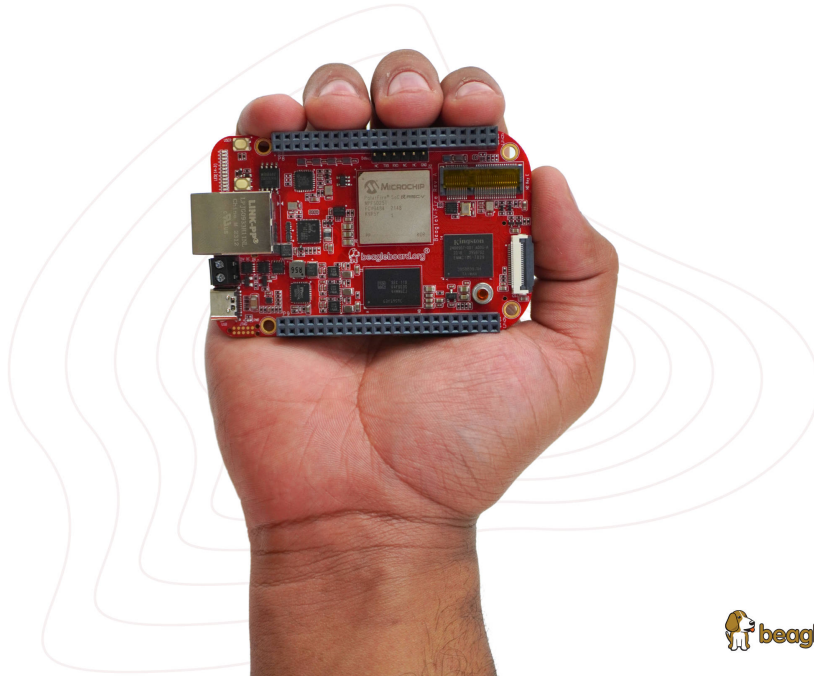
Table of contents

1	Introduction	3
1.1	Pinout Diagrams	3
1.2	Detailed overview	6
1.3	Board components location	6
1.3.1	Front components location	6
1.3.2	Back components location	7
2	Quick Start	9
2.1	What's included in the box?	9
2.2	Unboxing	9
2.3	Tethering to PC	9
2.4	Flashing eMMC	9
2.4.1	Device Firmware Update (DFU)	9
2.4.2	BeagleBoard Imager	11
2.5	Access UART debug console	11
2.6	Demos and Tutorials	12
3	Design & specifications	13
3.1	Block diagram	13
3.2	System on Chip (SoC)	14
3.3	Power management	14
3.4	General Connectivity and Expansion	14
3.4.1	USB-C port	14
3.4.2	P8 & P9 cape header pins	14
3.4.3	ADC	14
3.5	Buttons and LEDs	14
3.5.1	User LEDs and Power LED	14
3.5.2	User and reset button	14
3.6	Connectivity	14
3.6.1	Ethernet	14
3.7	Memory, Media and Data storage	17
3.7.1	DDR memory	17
3.7.2	eMMC	17
3.7.3	microSD	17
3.7.4	EEPROM	17
3.7.5	SPI flash	17
3.8	Multimedia I/O	17
3.8.1	CSI	17
3.9	Debug	17
3.9.1	UART debug port	17
3.9.2	JTAG debug port	17
3.10	Mechanical Specifications	21
4	Expansion	29
4.1	Cape Headers	29
4.1.1	Connector P8	29
4.1.2	Connector P9	33

5 Demos	37
5.1 Upgrade BeagleV-Fire Gateway	37
5.1.1 Required Equipment	37
5.1.2 Connect to BeagleV-Fire Linux Command Line Interface	37
5.1.3 Gateway Upgrade Linux Commands	38
5.2 Flashing gateway and Linux image	39
5.2.1 Programming & Debug tools installation	39
5.2.2 Flashing gateway image	40
5.2.3 Flashing eMMC	42
5.3 Microchip 2025.1 FPGA Tools Installation Guide	45
5.3.1 Install Libero SoC	47
5.3.2 Install SoftConsole 2022.2	48
5.3.3 Request a Libero Silver license	48
5.3.4 Setup tools script	49
5.3.5 License Issues running the gateway script	49
5.4 Microchip 2024.2 and Older FPGA Tools Installation Guide	50
5.4.1 Install Libero	50
5.4.2 Install SoftConsole 2022.2	51
5.4.3 Install the Libero licensing daemon	51
5.4.4 Request a Libero Silver license	51
5.4.5 Execute tool setup script	52
5.5 Gateway Design Introduction	54
5.5.1 Gateway Architecture	54
5.6 How to retrieve BeagleV-Fire's gateway version	56
5.6.1 Device Tree	56
5.6.2 Bootloader messages	57
5.7 Building Linux for BeagleV-Fire using Buildroot	57
5.7.1 Introduction	57
5.7.2 Start Building	58
5.8 Gateway Full Build Flow	59
5.8.1 Introduction	59
5.8.2 Programming artifacts	60
5.8.3 Programming BeagleV-Fire with new gateway	61
5.9 Gateway TCL Scripts Structure	61
5.9.1 Gateway Project	61
5.9.2 Gateway Components	62
5.9.3 Gateway Build Options	62
5.9.4 Gateway Component Directories	63
5.9.5 Opening the gateway as a libero project	63
5.10 Customize BeagleV-Fire Cape Gateway Using Verilog	64
5.10.1 Fork BeagleV-Fire Gateway Repository	65
5.10.2 Create A Custom Gateway Build Option	65
5.10.3 Rename A Copy Of The Cape Gateway Verilog Template	66
5.10.4 Customize The Cape's Verilog Source Code	66
5.10.5 Commit And Push Changes To Your Forked Repository	69
5.10.6 Retrieve The Forked Repositories Artifacts	69
5.10.7 Program BeagleV-Fire With Your Custom Bitstream	69
5.11 How to use PicoRV Softcore on BeagleV-Fire	70
5.11.1 Introduction	70
5.11.2 Prerequisites	70
5.11.3 How to use the Softcore	71
5.12 How to build the BeagleV-Fire Gateway on Windows	73
5.12.1 Introduction	73
5.12.2 Prerequisites	73
5.13 Exploring Gateway Design with Libero	74
5.13.1 Prerequisites	74
5.13.2 Cloning and Building the Gateway	74
5.13.3 Exploring The Design	75

5.13.4 Adding Custom HDL	75
5.13.5 Exporting The Design	80
5.13.6 Final Verification	82
5.14 Simulating Gateway Design with Libero	84
5.14.1 Prerequisites	84
5.14.2 Setting up ModelSim	85
5.14.3 Simulating the Blinky LED Design	85
5.14.4 Exploring ModelSim and Running the simulations	86
5.15 Comms Cape Gateway for BeagleV-Fire	88
5.15.1 Cape schematics, layout, and mechanicals	88
5.15.2 Usage	89
5.15.3 Pinout	89
5.16 Accessing APB and AXI Peripherals Through Linux	91
5.16.1 AXI	91
5.16.2 APB	91
5.16.3 Accessing AXI and APB Peripherals from Linux	92
5.17 Change the MSS configuration	94
5.17.1 Quick recap	94
5.17.2 Backannotation	95
5.17.3 Version check	95
5.18 Help! I broke my board. Now what?	95
5.18.1 The Big Picture	97
5.18.2 Level 1: Triage	97
5.18.3 Level 2: U-Boot	98
5.18.4 Level 3: Linux boot	99
5.19 Libero Setup - Ubuntu 24.04	100
5.19.1 Steps	100
5.19.2 Support and Acknowledgements	103
6 Support	105
6.1 Production board boot media	105
6.2 Certifications and export control	105
6.2.1 Export designations	105
6.2.2 Size and weight	105
6.3 Additional documentation	105
6.3.1 Hardware docs	105
6.3.2 Software docs	105
6.3.3 Support forum	106
6.3.4 Pictures	106
6.4 Change History	106
6.4.1 Board Changes	106

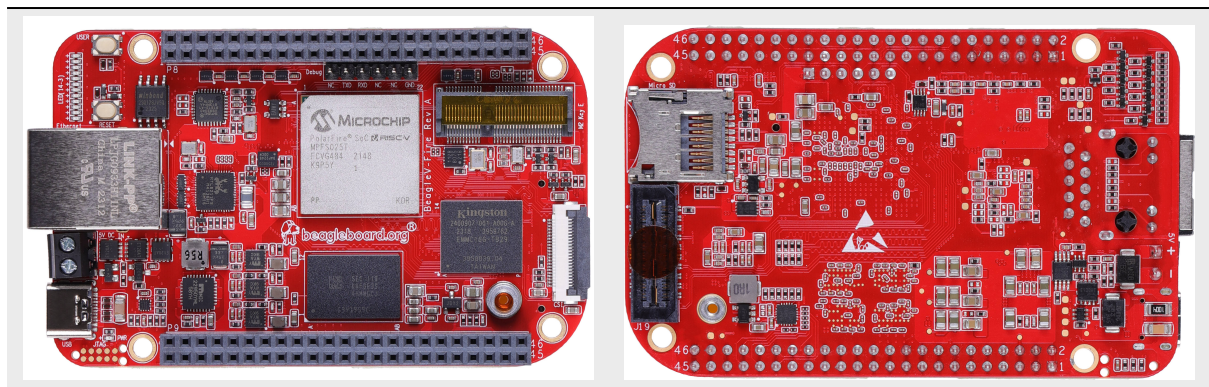
BeagleV®-Fire is a revolutionary SBC powered by the Microchip's PolarFire® MPFS025T RISC-V System on Chip (SoC) with FPGA fabric. BeagleV®-Fire opens up new horizons for developers, tinkerers, and the open-source community to explore the vast potential of RISC-V architecture and FPGA technology. It has the same P8 & P9 cape header pins as BeagleBone Black allowing you to stack your favorite BeagleBone cape on top to expand it's capability. Built around the powerful and energy-efficient RISC-V instruction set architecture (ISA) along with its versatile FPGA fabric, BeagleV®-Fire SBC offers unparalleled opportunities for developers, hobbyists, and researchers to explore and experiment with RISC-V technology.



Chapter 1

Introduction

BeagleV®-Fire is a revolutionary SBC powered by the Microchip's PolarFire® MPFS025T System on Chip (SoC) with 4x RV64GC Application cores, 1x RV64IMAC monitor/boot core, and FPGA fabric. BeagleV®-Fire opens up new horizons for developers, tinkerers, and the open-source community to explore the vast potential of RISC-V architecture and FPGA technology. It has the same P8 & P9 cape header pins as BeagleBone Black allowing you to stack your favourite BeagleBone cape on top to expand it's capability. Built around the powerful and energy-efficient RISC-V instruction set architecture (ISA) along with its versatile FPGA fabric, BeagleV®-Fire SBC offers unparalleled opportunities for developers, hobbyists, and researchers to explore and experiment with RISC-V technology.



1.1 Pinout Diagrams

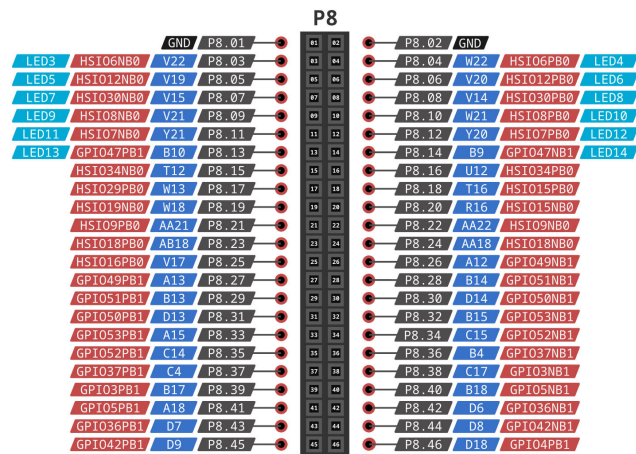
Choose the cape header to see respective pinout diagram.

P8 cape header

P9 cape header

BeagleV-Fire

P8 cape header pinout



GND
 CAPE HEADER PIN NAME
 FPGA PIN NUMBER
 FPGA PIN NAME
 USER LED



Fig. 1.1: BeagleV-Fire P8 cape header pinout

BeagleV-Fire
P9 cape header pinout

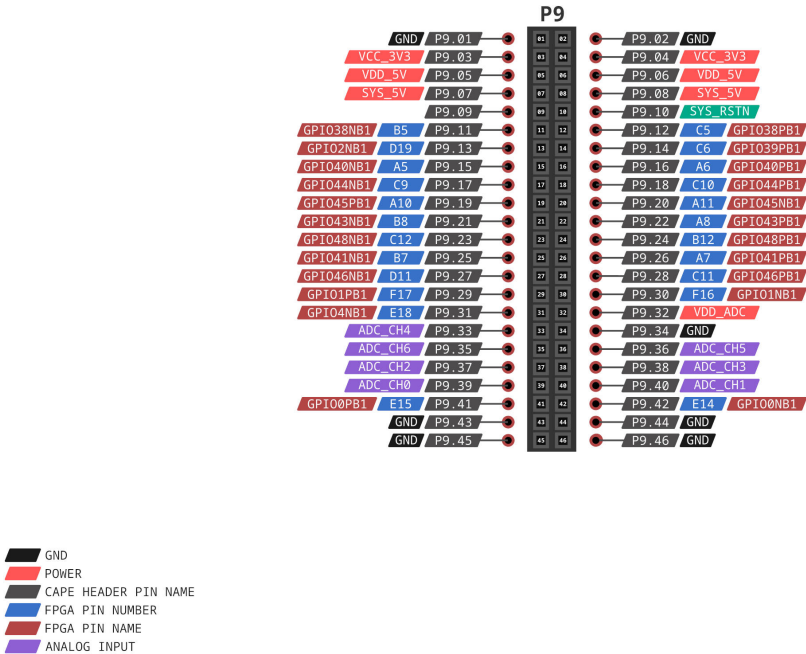


Fig. 1.2: BeagleV-Fire P9 cape header pinout

1.2 Detailed overview

Table 1.1: BeagleV-Fire features

Feature	Description
Processor	MPFS025T-FCVG484E
Memory	2GB (1Gb x 16)- 1866MHz 3733Mbps, LPDDR4
Storage	Kingston 16GB eMMC
Wireless	1x M.2 Key E, support 2.4GHz/5GHz WiFi module
Ethernet	- PHY: Realtek RTL8211F-VD-CG Gigabit Ethernet phy - Connector: integrated magnetics RJ-45
USB C	- Connectivity: Flash/programming support - Power: Input: 5V @ 3A
Other connectors	1x SYZYGY High speed connector - microSD card slot - CSI connector compatible with BeagleBone AI-64, BeagleV-Ahead, Raspberry Pi Zero / CM4 (22-pin)

1.3 Board components location

This section describes the key components on the board, their location and function.

1.3.1 Front components location

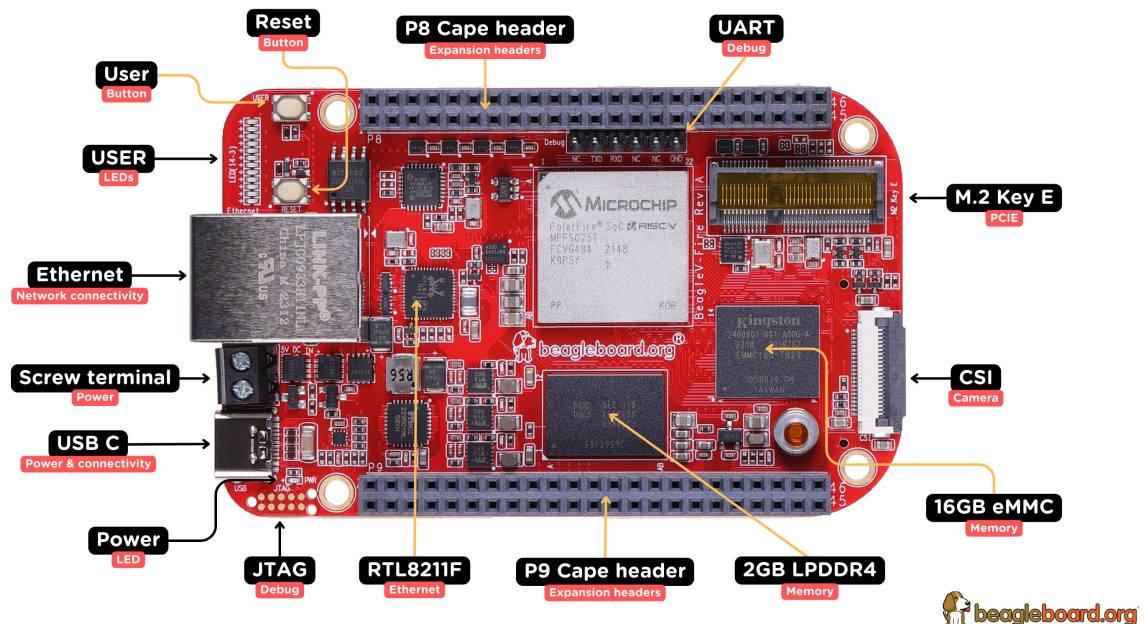


Fig. 1.3: BeagleV-Fire board front components location

Table 1.2: BeagleV-Fire board front components location

Feature	Description
Power LED	Power (Board ON) indicator
JTAG (MPFS025T)	MPFS025T SoC JTAG debug port
RTL8211F	Gigabit IEEE 802.11 Ethernet PHY
P8 & P9 cape header	Expansion headers for BeagleBone capes.
2GB RAM	2GB (1Gb x 16)- 1866MHz 3733Mbps, LPDDR4
16GB eMMC	Kingston 16GB eMMC Flash storage
CSI	22pin MIPI Camera connectors
M.2 Key E	PCIe M.2 Key E connector
UART debug header	6 pin UART debug header
Reset button	Press to reset BeagleV-Fire board (MPFS025T SoC)
User button	User defined (custom) action button
User LEDs	12x user programmable LEDs to show various board status during boot.
GigaBit Ethernet	1Gb/s Wired internet connectivity
Barrel jack	Power input
USB C	Power, connectivity, and board flashing.

1.3.2 Back components location

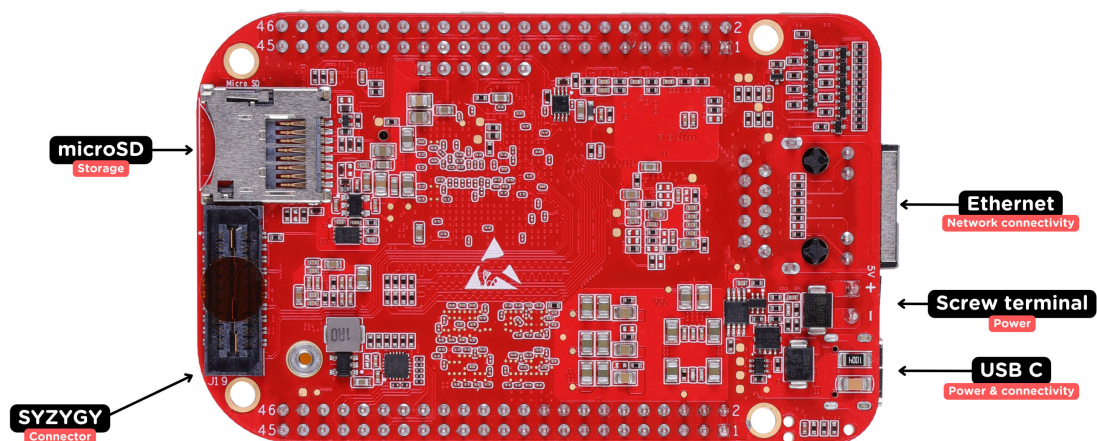


Fig. 1.4: BeagleV-Fire board back components location

Table 1.3: BeagleV-Fire board back components location

Feature	Description
microSD	microSD card slot
SYZYGY	SYZYGY High speed connector

Chapter 2

Quick Start

2.1 What's included in the box?

When you purchase a brand new BeagleV-Fire, In the box you'll get:

1. [BeagleV-Fire board](#)
2. Quick-start card

Tip: For board files, 3D model, and more, you can checkout [BeagleV-Fire repository on OpenBeagle](#).

2.2 Unboxing

2.3 Tethering to PC

To connect BeagleV-Fire board to PC via USB Type C receptacle you need a USB type C cable. Connection guide for the same is shown below:

Tip: To get a USB type C cable you can checkout links below:

1. [USB C cable 0.3m \(mouser\)](#)
 2. [USB C cable 1.83m \(digikey\)](#)
-

2.4 Flashing eMMC

First we need to go get [Beagleboard Imager](#).

Grab the version suitable for your operating system.

2.4.1 Device Firmware Update (DFU)

Next we need your Beagle to be ready to receive an updated image:

To enter "DFU" mode, you press and hold the USER button while connecting your Fire to the USB port on your machine.

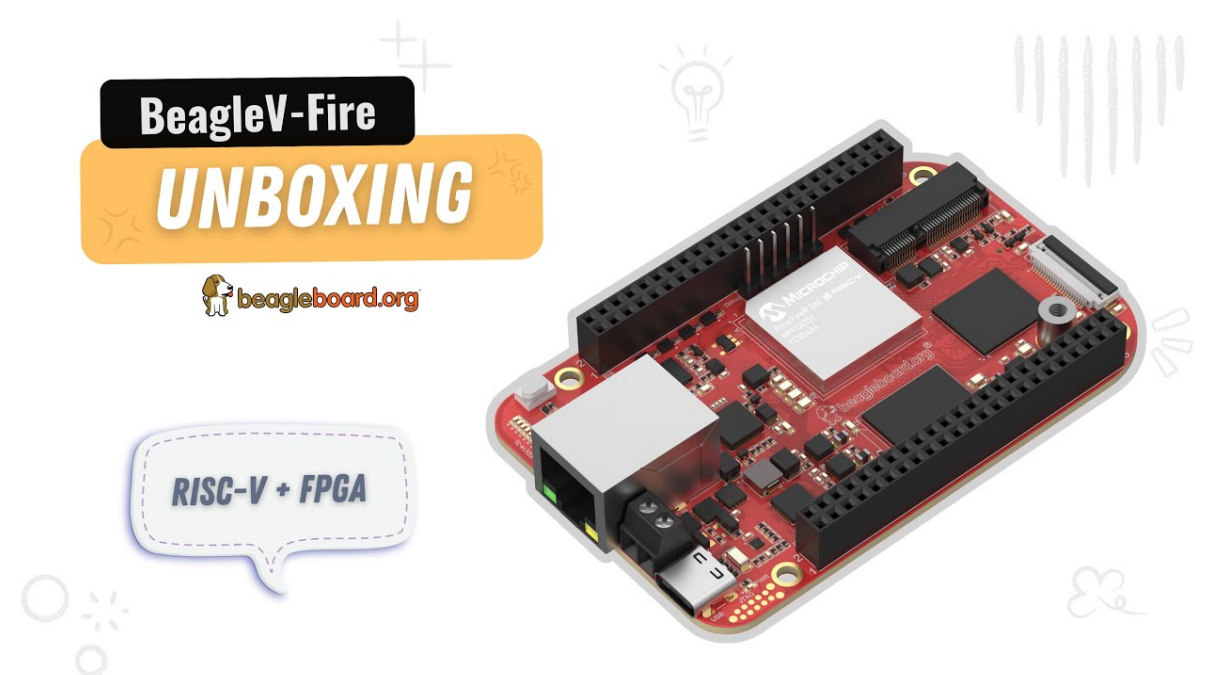


Fig. 2.1: <https://youtu.be/5cylv1R-1mc>



Fig. 2.2: BeagleV-Fire tethered connection

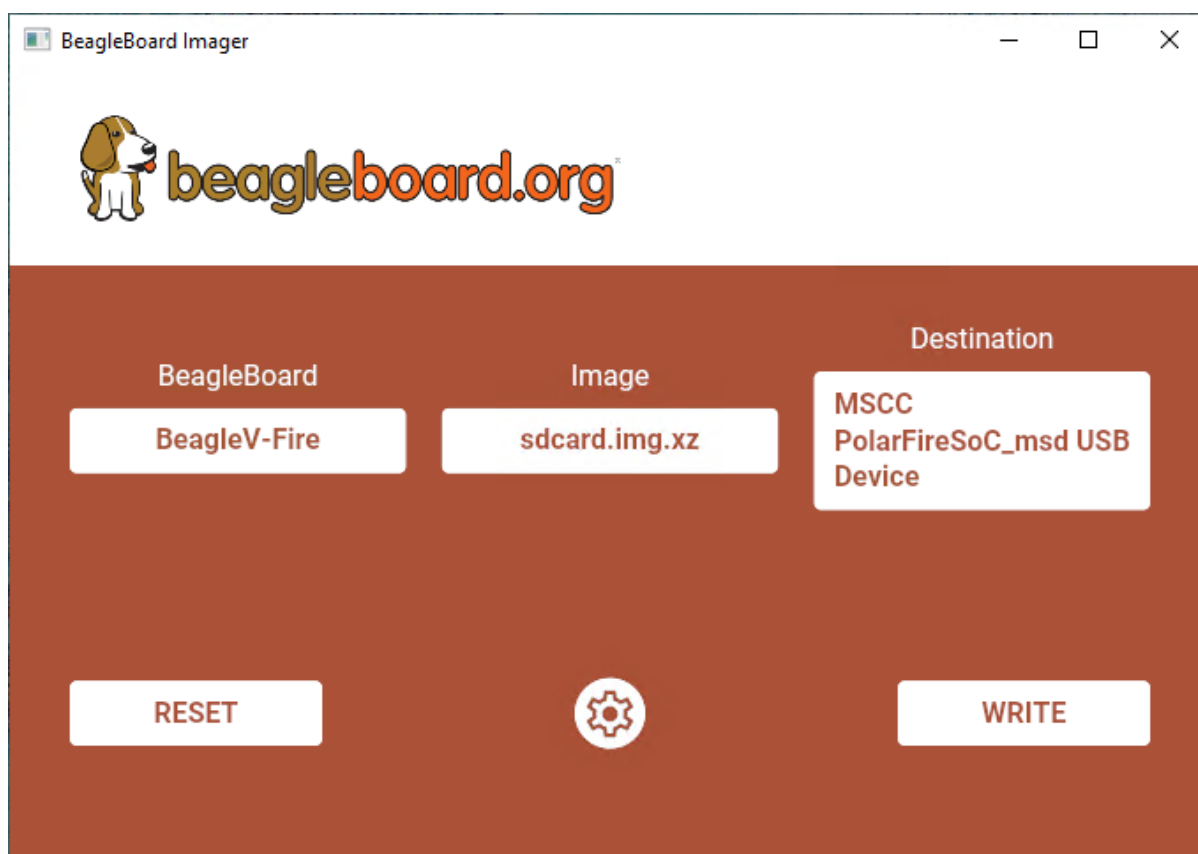
You should let the button go once the first LED comes on.

Once you have two solid lit LEDs and an extra USB drive, we're all set to continue.

2.4.2 BeagleBoard Imager

With BeagleBoard Imager running, work your way from left to right, like so:

1. Select board. This one's obvious.
2. Select your image variant.
3. Select your drive. This one is critical to get right; see below.
4. Click WRITE and watch it go!



With a little luck, all goes well and BeagleBoard Imager reports no errors.

In that case, you can close out BeagleBoard Imager and disconnect the USB cable.

On the next power-up, your Beagle will boot up the new image and you're ready to rock!

2.5 Access UART debug console

Note: Some tested devices that are known to be working well include:

1. [Adafruit CP2102N Friend - USB to Serial Converter](#)
2. [Raspberry Pi Debug Probe Kit for Pico and RP2040](#)

To access a BeagleV-Fire serial debug console you can connect a USB to UART to your board as shown below:

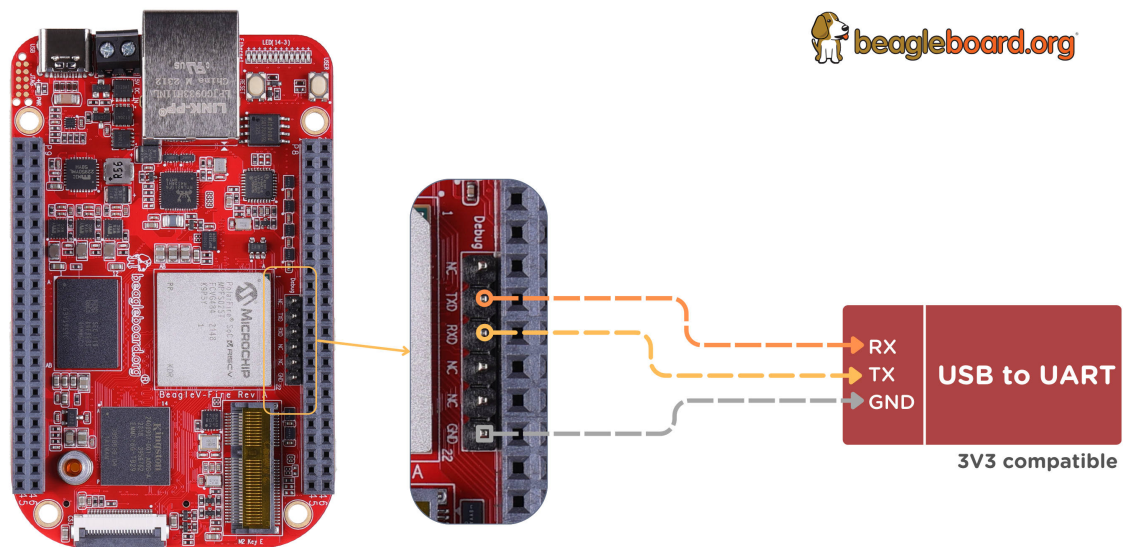


Fig. 2.3: BeagleV-Fire UART debug port connection

To see the board boot log and access your BeagleV-Fire's console you can use application like `tio` to access the console. If you are using Linux your USB to UART converter may appear as `/dev/ttyUSB0`. It will be different for Mac and Windows operating systems. To find serial port for your system you can checkout [this guide](#).

```
[lorforlinux@fedora ~] $ tio /dev/ttyUSB0
tio v2.5
Press ctrl-t q to quit
Connected
```

2.6 Demos and Tutorials

- [How to retrieve BeagleV-Fire's gateway version](#)
- [Upgrade BeagleV-Fire Gateway](#)
- [Flashing gateway and Linux image](#)
- [Gateway Design Introduction](#)
- [Microchip 2024.2 and Older FPGA Tools Installation Guide](#)
- [Microchip 2025.1 FPGA Tools Installation Guide](#)
- [How to build the BeagleV-Fire Gateway on Windows](#)

Chapter 3

Design & specifications

If you want to know how BeagleV-Fire board is designed and what are it's high-level specifications then this chapter is for you. We are going to discuss each hardware design element in detail and provide high-level device specifications in a short and crisp form as well.

Tip: For hardware design files and schematic diagram you can checkout BeagleV-Fire GitLab repository: <https://git.beagleboard.org/beaglev-fire/beaglev-fire>

3.1 Block diagram

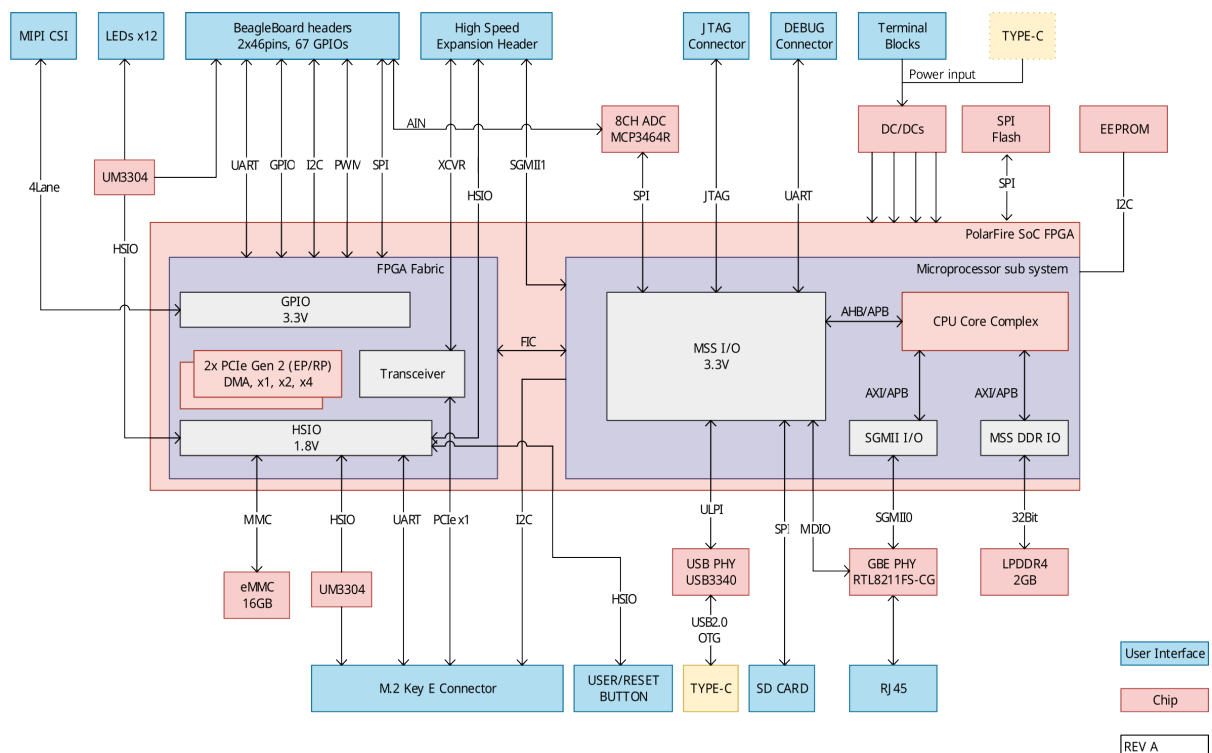


Fig. 3.1: System block diagram

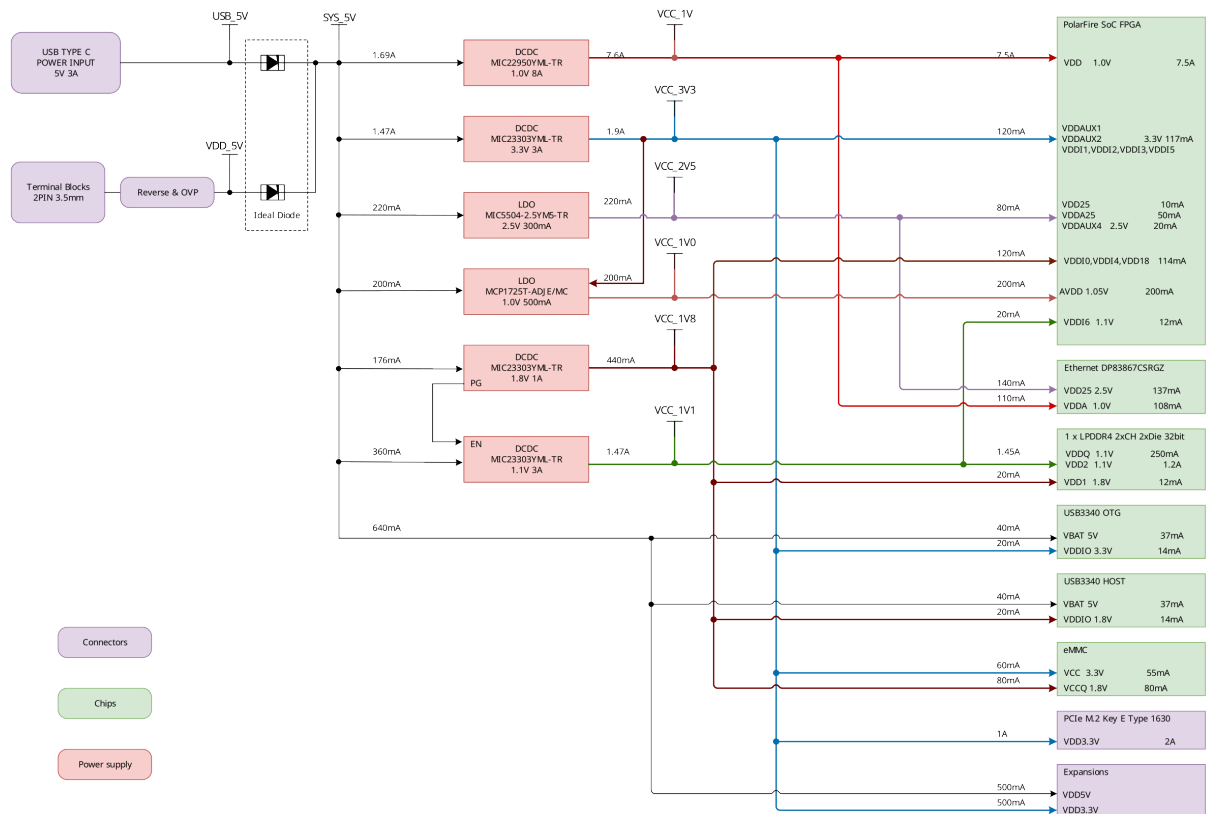


Fig. 3.2: Power tree diagram

3.2 System on Chip (SoC)

3.3 Power management

3.4 General Connectivity and Expansion

3.4.1 USB-C port

3.4.2 P8 & P9 cape header pins

3.4.3 ADC

3.5 Buttons and LEDs

3.5.1 User LEDs and Power LED

3.5.2 User and reset button

3.6 Connectivity

3.6.1 Ethernet

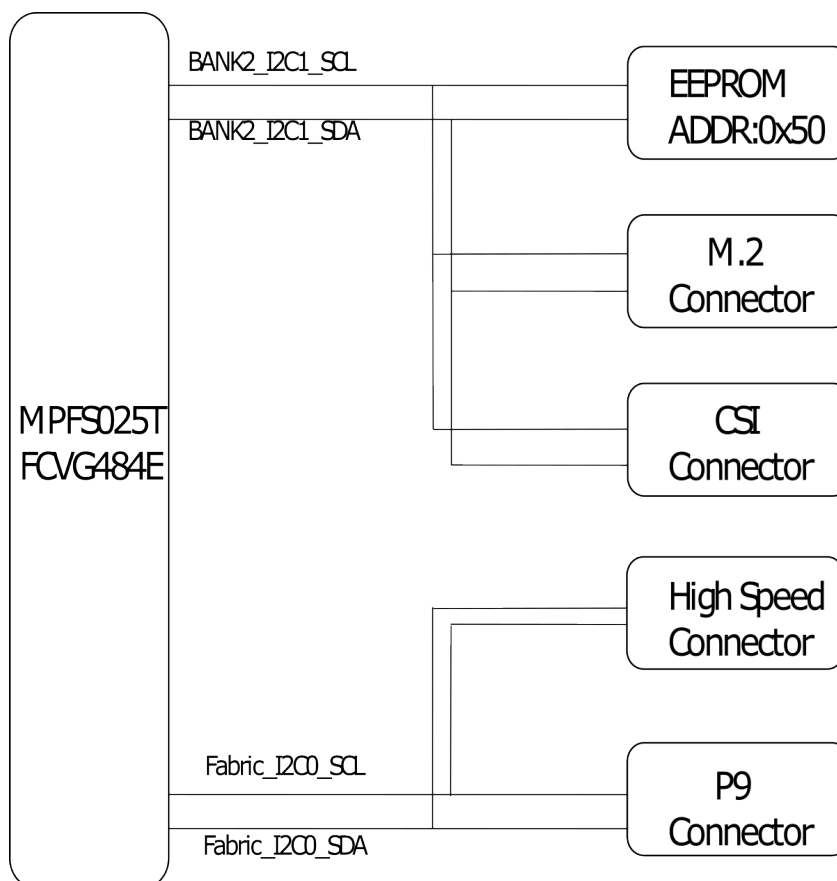


Fig. 3.3: I2C tree diagram

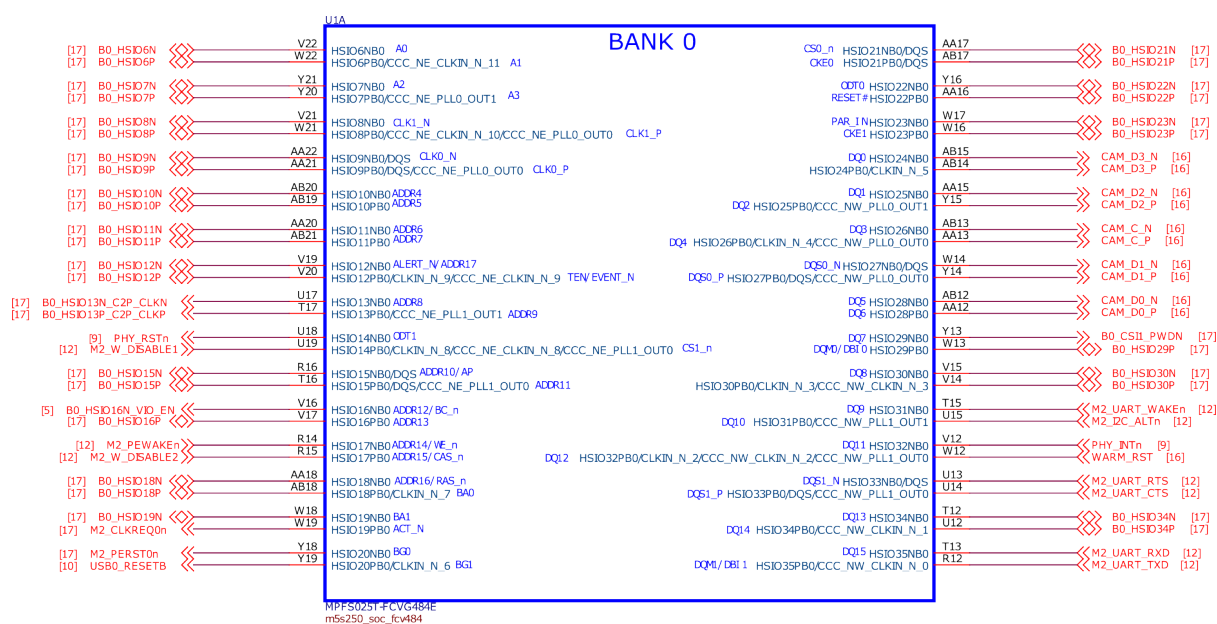


Fig. 3.4: SoC bank0

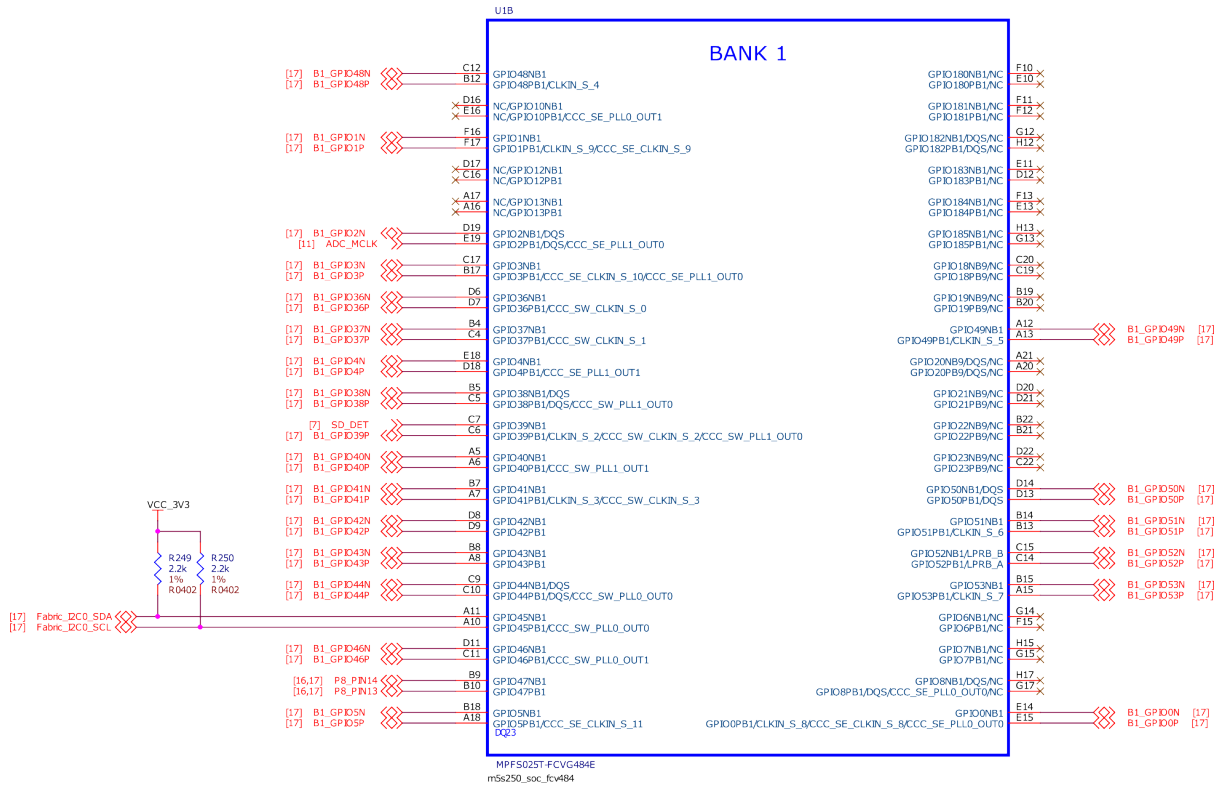


Fig. 3.5: SoC bank1

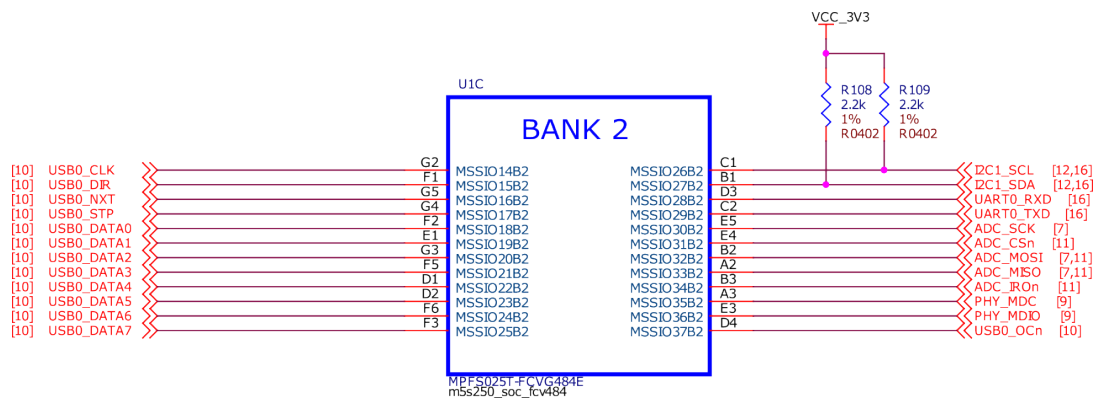


Fig. 3.6: SoC bank2

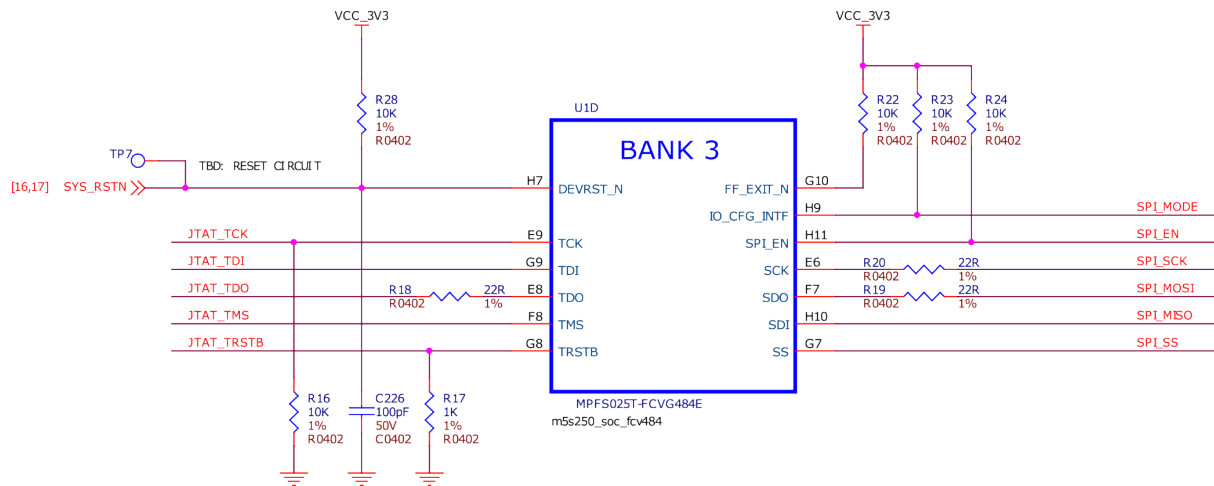


Fig. 3.7: SoC bank3

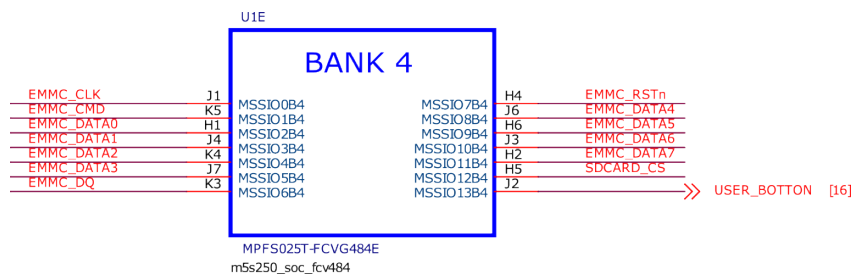


Fig. 3.8: SoC bank4

3.7 Memory, Media and Data storage

3.7.1 DDR memory

3.7.2 eMMC

3.7.3 microSD

3.7.4 EEPROM

3.7.5 SPI flash

3.8 Multimedia I/O

3.8.1 CSI

3.9 Debug

3.9.1 UART debug port

3.9.2 JTAG debug port

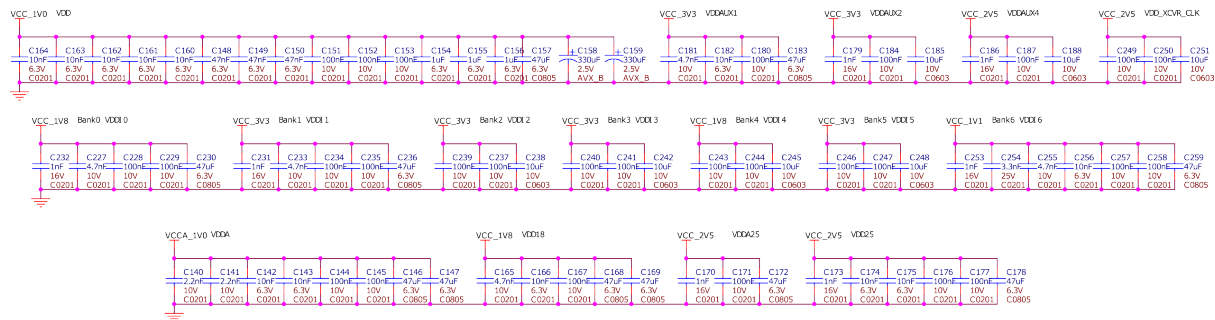


Fig. 3.9: SoC power

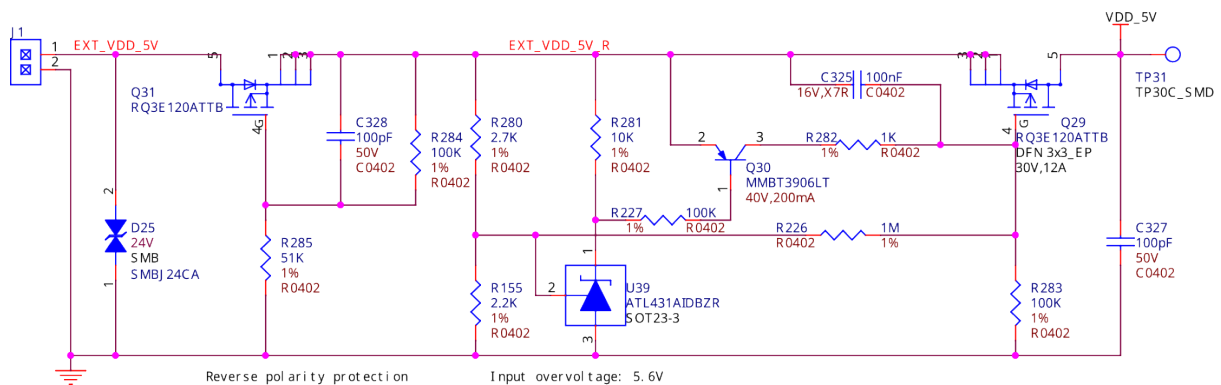


Fig. 3.10: DC 5V input

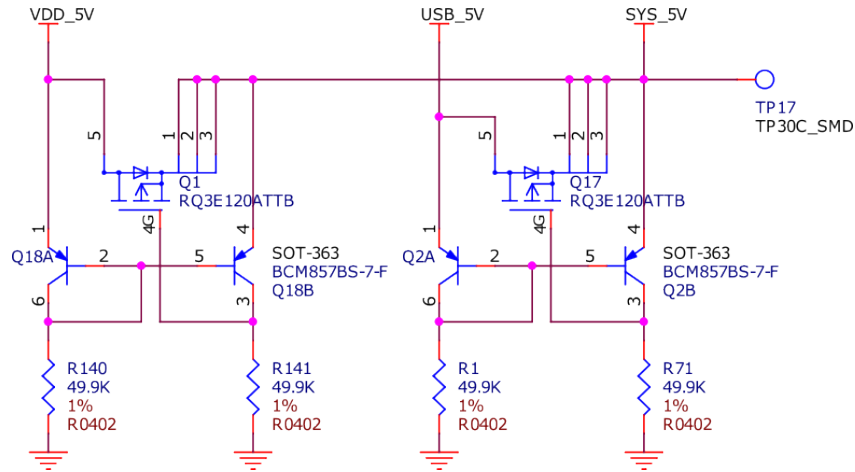


Fig. 3.11: Ideal diode

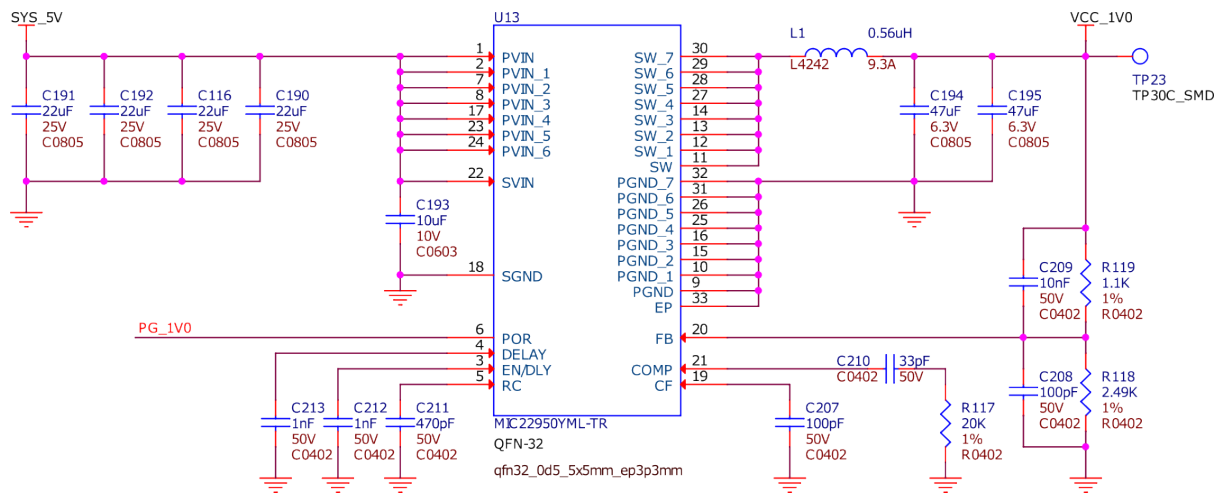


Fig. 3.12: VCC 1V0

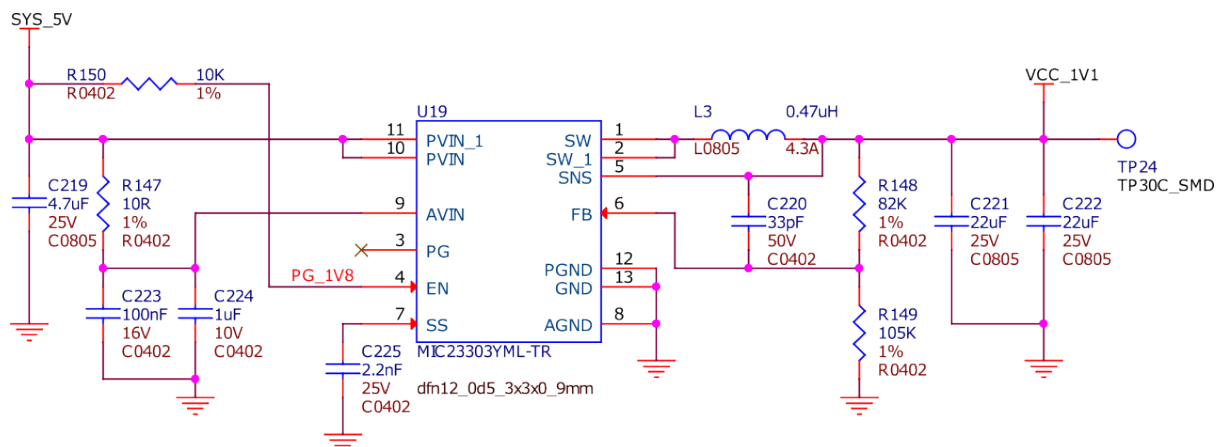
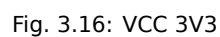
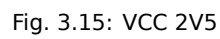
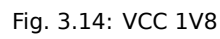


Fig. 3.13: VCC 1V1



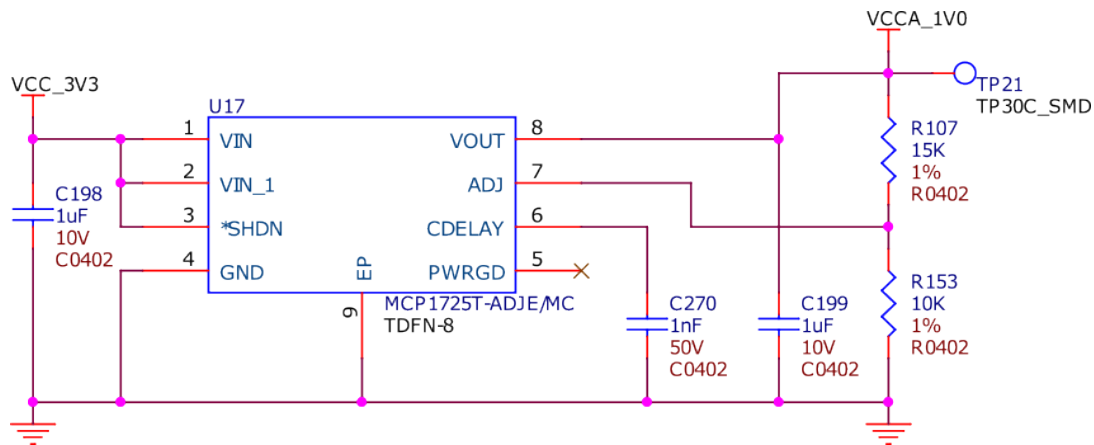


Fig. 3.17: VCCA 1V0

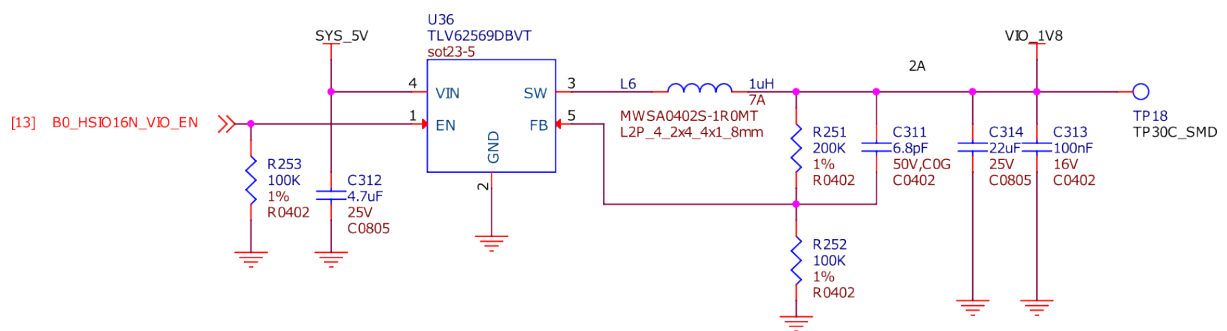
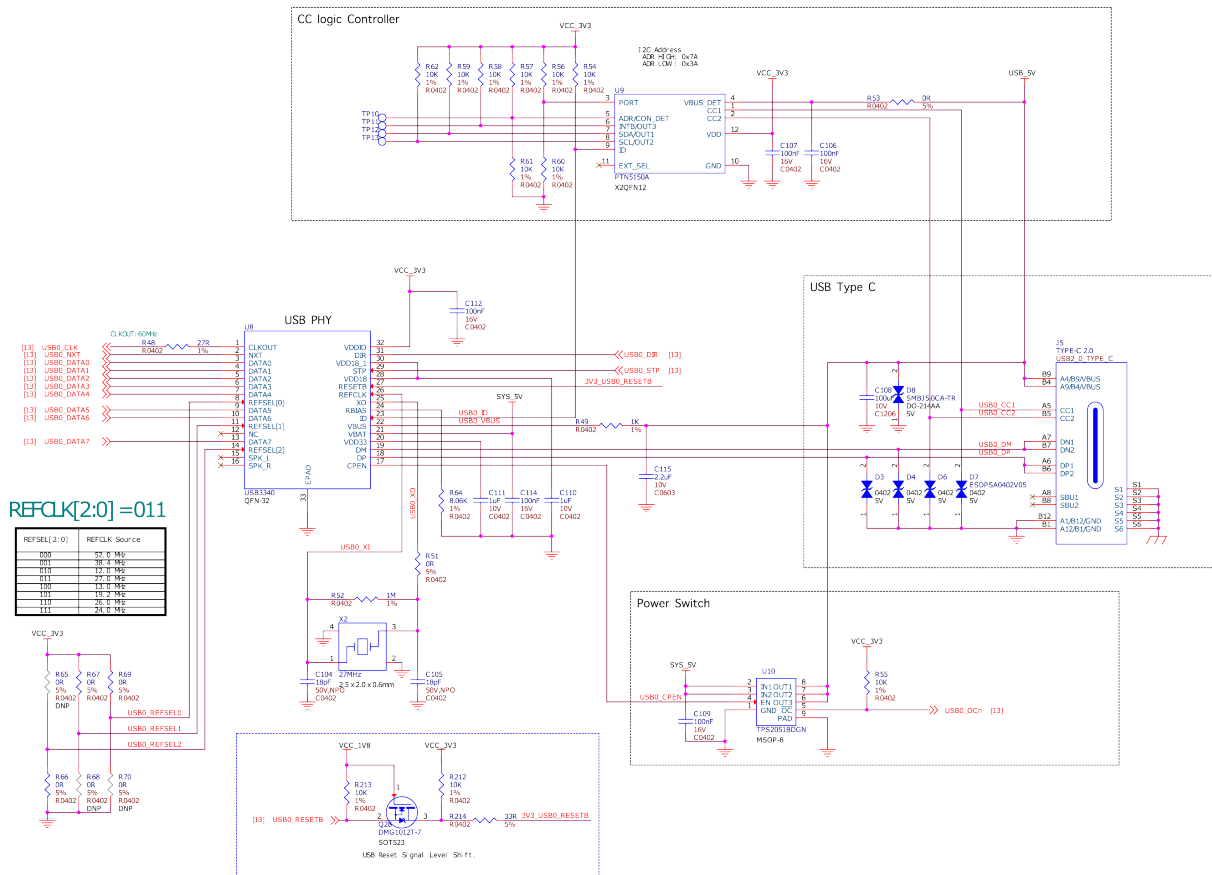


Fig. 3.18: VIO enable

3.10 Mechanical Specifications

Table 3.1: Dimensions & weight

Parameter	Values
Size	86.38 * 54.61 * 18.8 mm
Max heigh	18.8 mm
PCB Size	86.38 * 54.6 mm
PCB Layers	12 Layers
PCB Thickness	1.6 mm
RoHS compliant	Yes
Gross Weight	106 g
Net weight	45.8 g



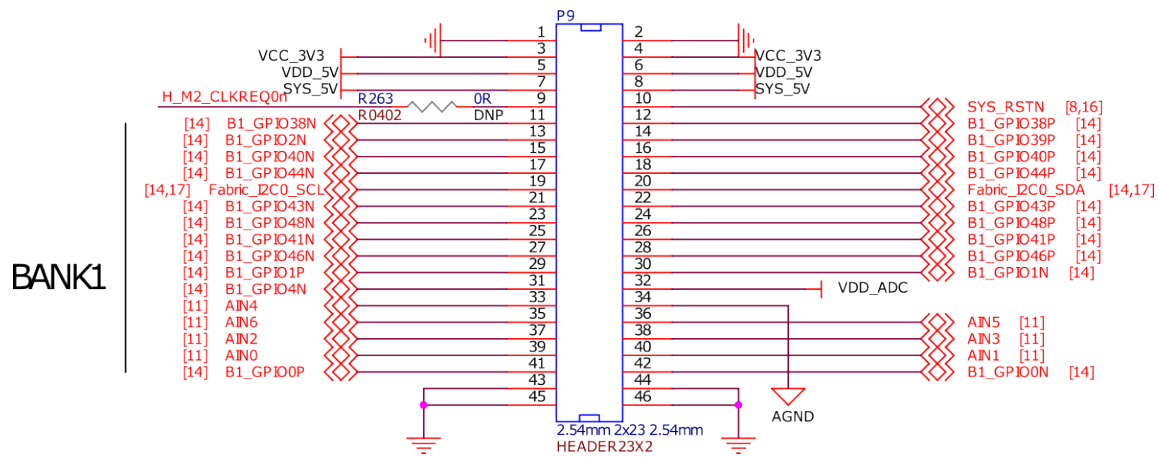


Fig. 3.21: P9 cape header

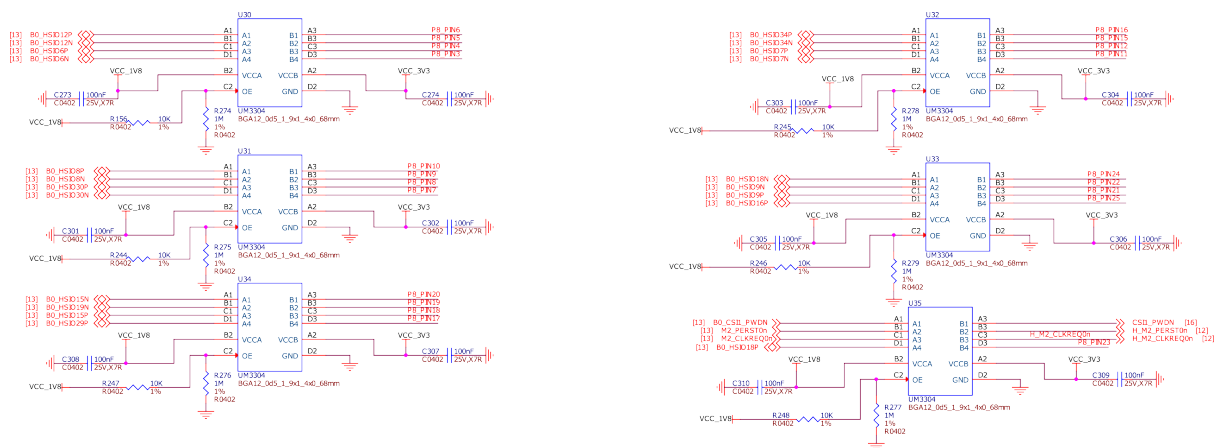


Fig. 3.22: Cape header voltage level translator

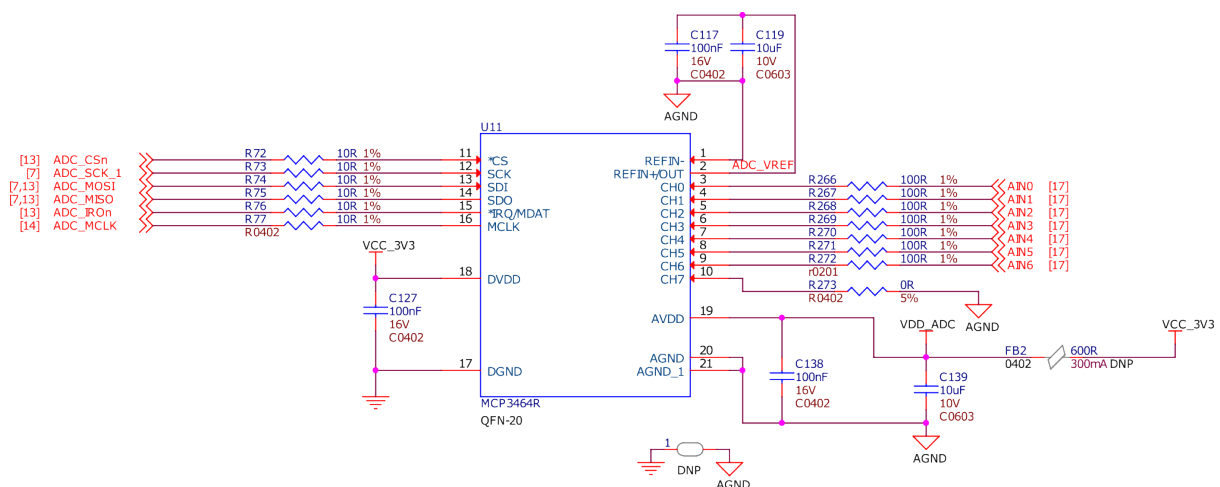


Fig. 3.23: 16bit Delta-Sigma ADC

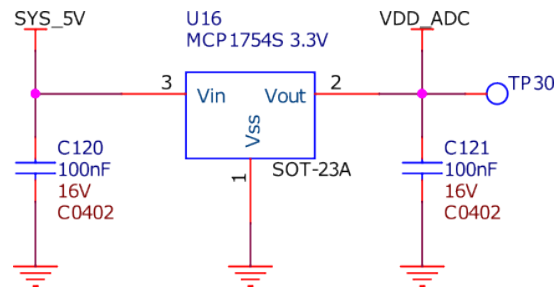


Fig. 3.24: ADC LDO power supply

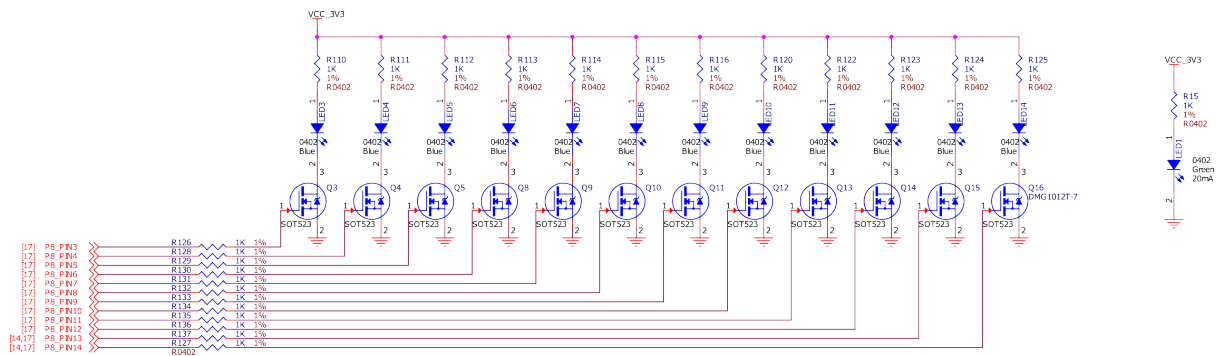


Fig. 3.25: User LEDs and power LED

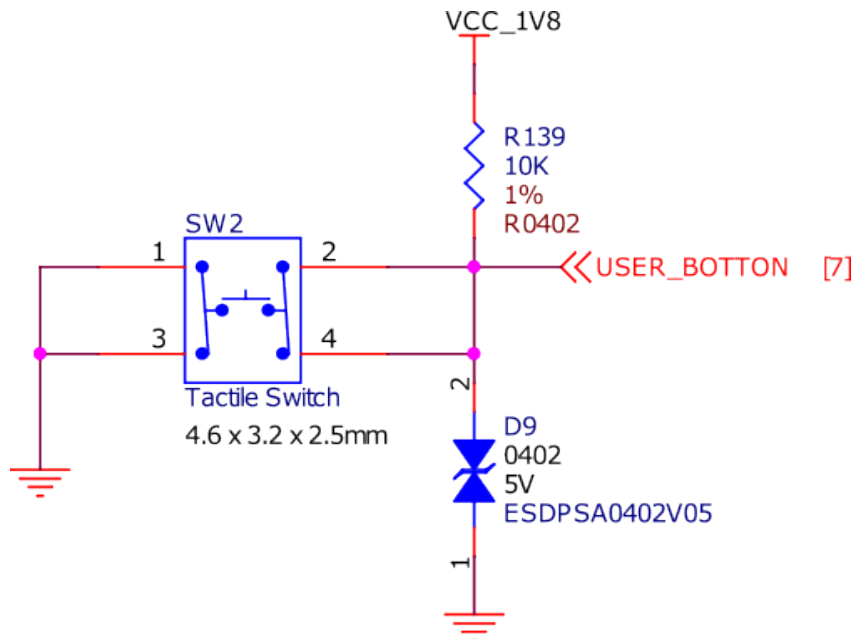


Fig. 3.26: User button

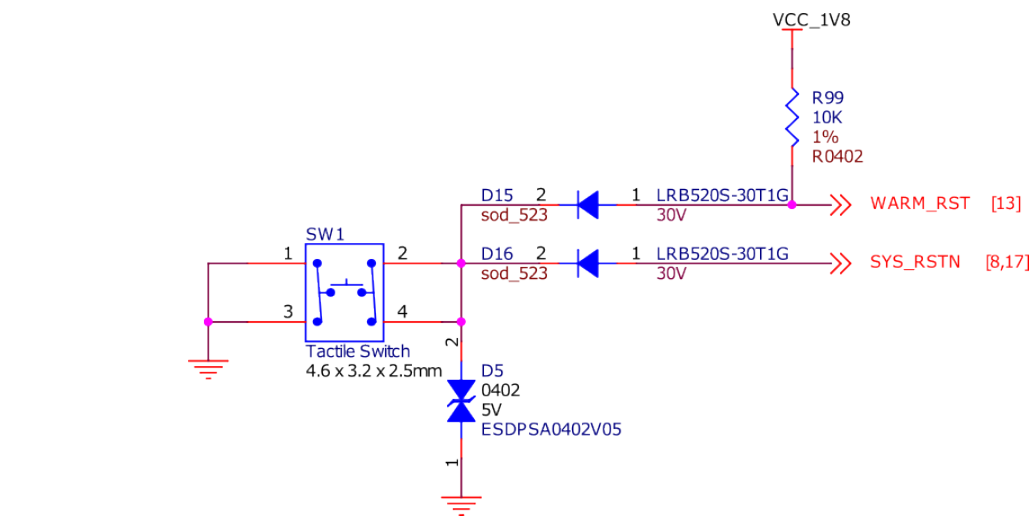
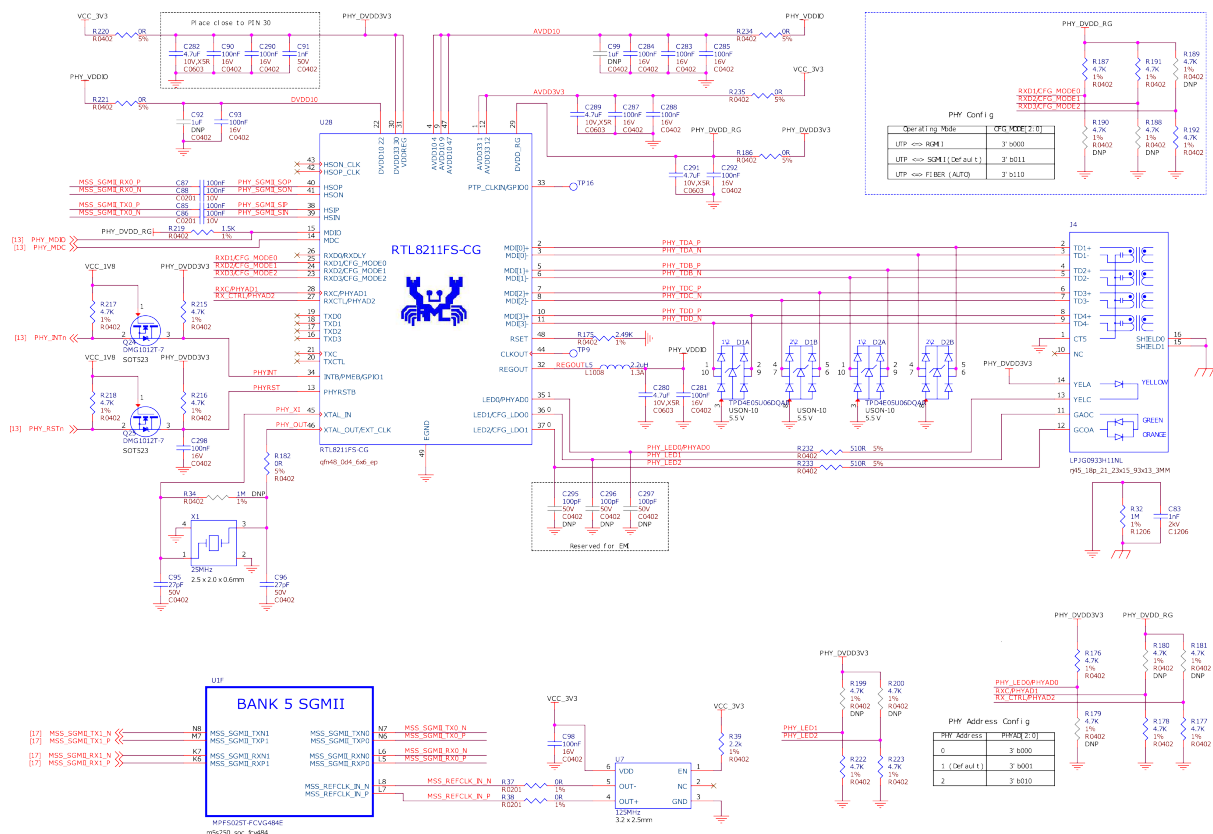


Fig. 3.27: Reset button



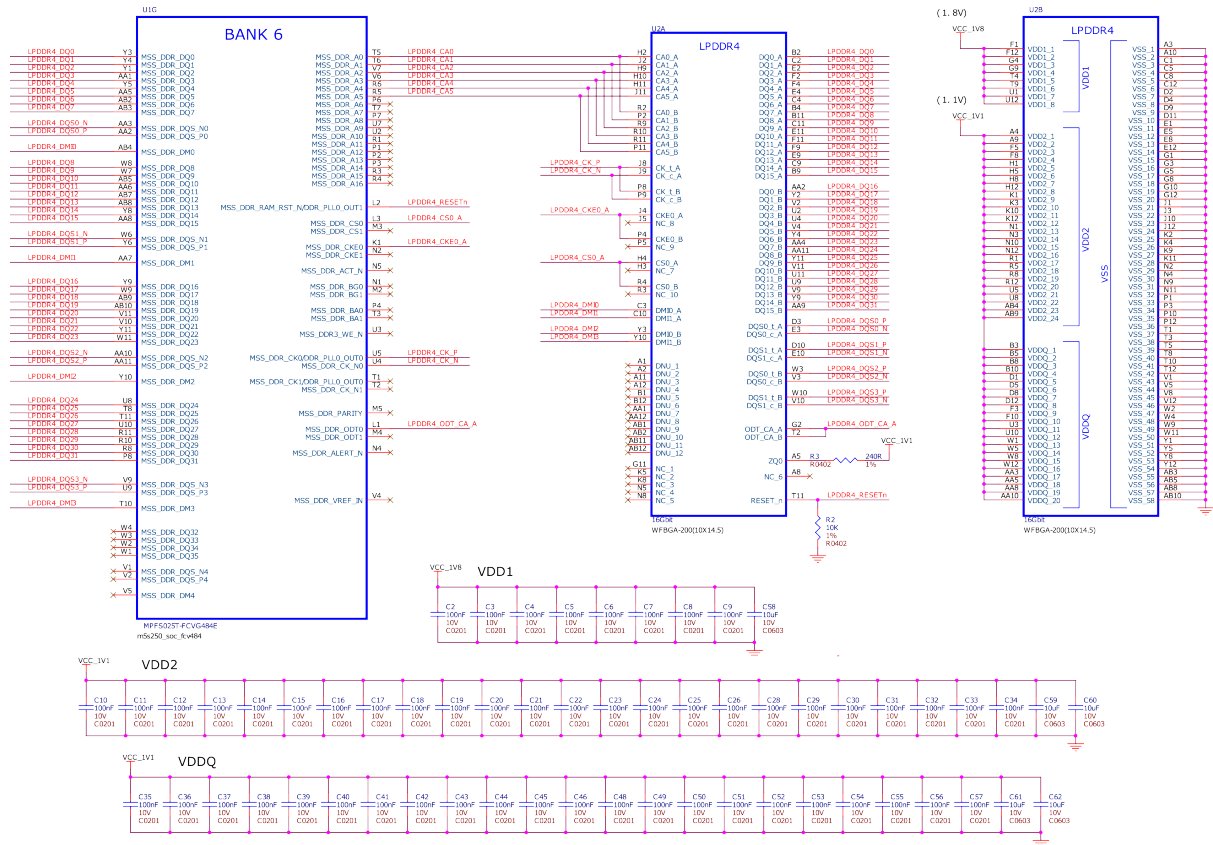


Fig. 3.29: LPDDR memory

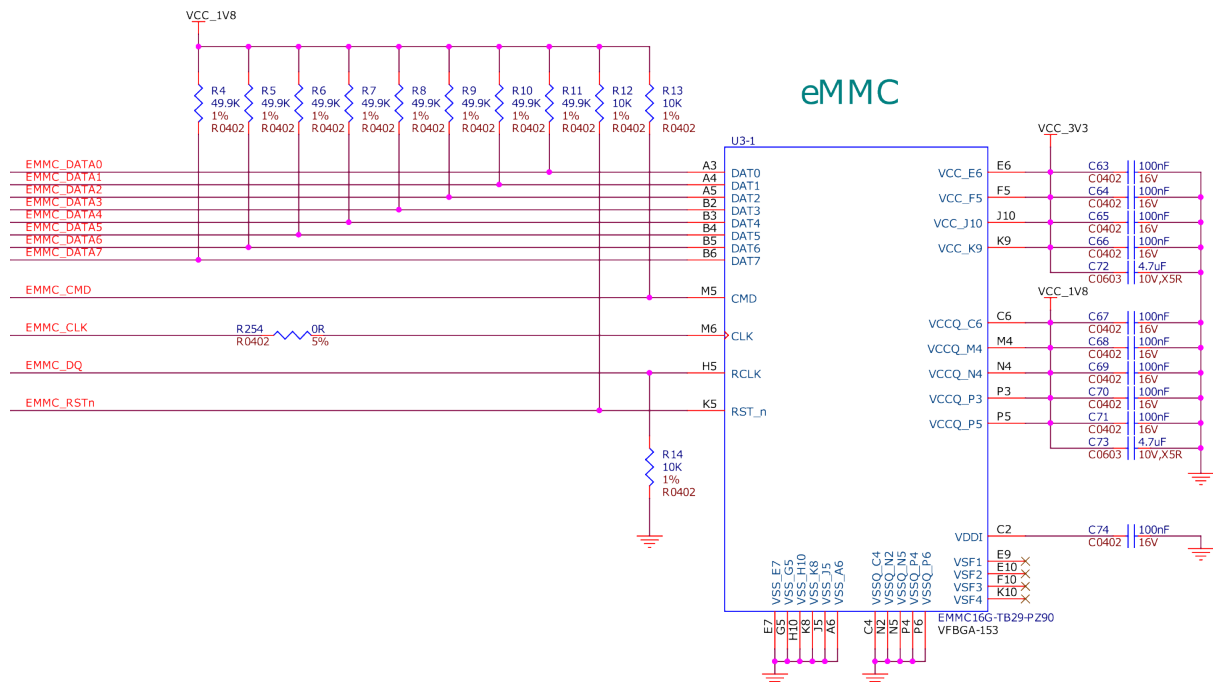
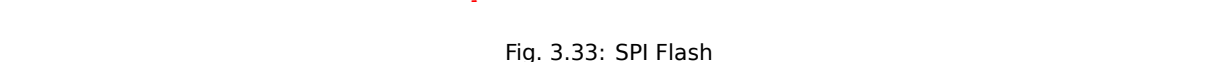
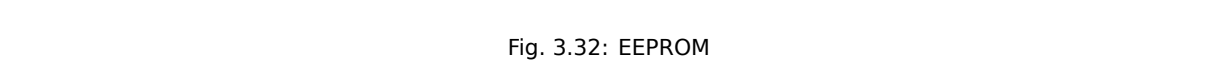


Fig. 3.30: EMMC flash storage



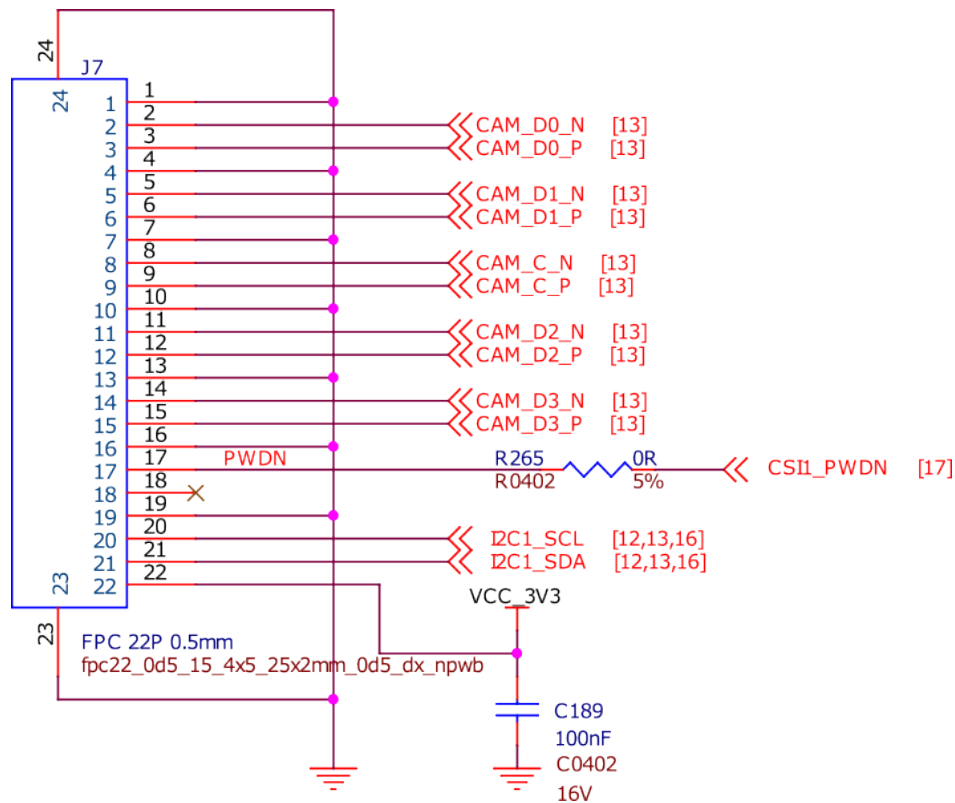


Fig. 3.34: CSI

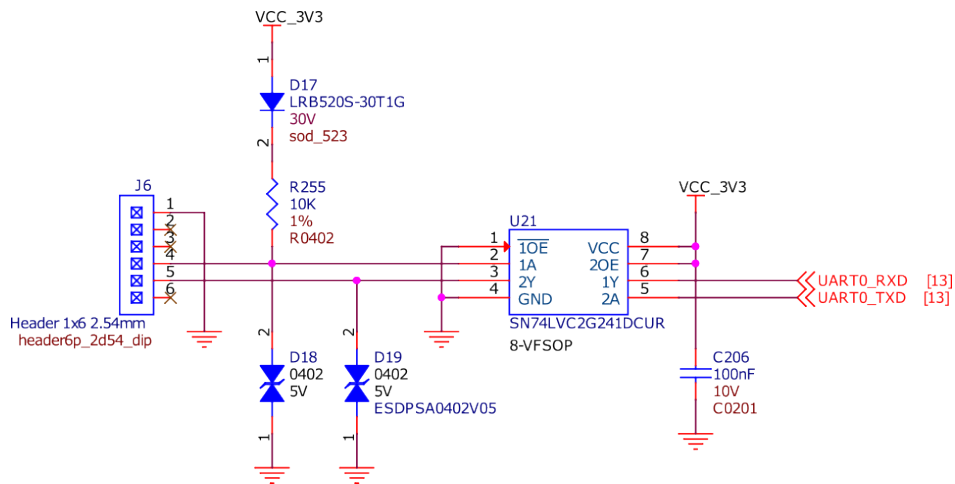


Fig. 3.35: UART debug header

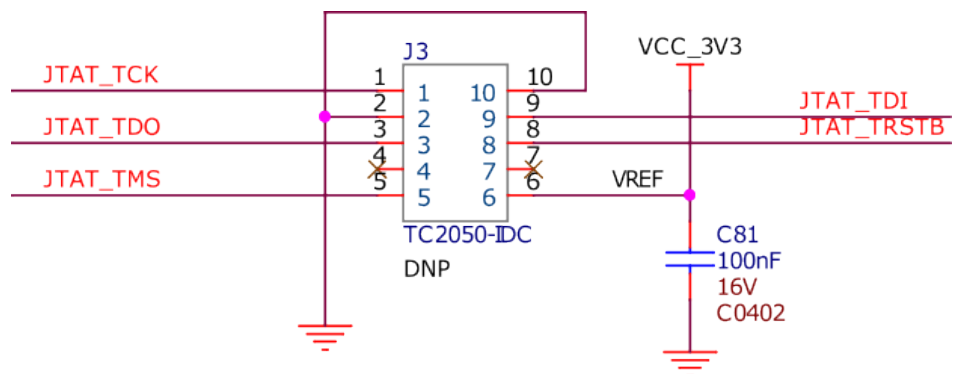


Fig. 3.36: JTAG debug header

Chapter 4

Expansion

4.1 Cape Headers

The expansion interface on the board is comprised of two headers P8 (46 pin) & P9 (46 pin). All signals on the expansion headers are **3.3V** unless otherwise indicated.

Note: Do not connect 5V logic level signals to these pins or the board will be damaged.

Note: DO NOT APPLY VOLTAGE TO ANY I/O PIN WHEN POWER IS NOT SUPPLIED TO THE BOARD. IT WILL DAMAGE THE PROCESSOR AND VOID THE WARRANTY.

NO PINS ARE TO BE DRIVEN UNTIL AFTER THE SYS_RESET LINE GOES HIGH.

4.1.1 Connector P8

The following tables show the pinout of the **P8** expansion header. The gateway image is responsible for setting the function of each pin. Refer to the processor documentation for more information on these pins and detailed descriptions of all of the pins listed. In some cases there may not be enough signals to complete a group of signals that may be required to implement a total interface.

The column heading is the pin number on the expansion header.

The **Name** row is the pin name on the processor.

The **BALL** row is the pin number on the processor.

The rows below **BALL** are the gateway setting for each pin.

NOTES:

DO NOT APPLY VOLTAGE TO ANY I/O PIN WHEN POWER IS NOT SUPPLIED TO THE BOARD. IT WILL DAMAGE THE PROCESSOR AND VOID THE WARRANTY.

NO PINS ARE TO BE DRIVEN UNTIL AFTER THE SYS_RESET LINE GOES HIGH.

P8.01-P8.02

P8.01	P8.02
GND	GND

P8.03-P8.05

Pin	P8.03	P8.04	P8.05
Name	HSIO6NB0	HSIO6PB0/CCC_NE_CLKIN_N_11	HSIO12NB0
BALL	V22	W22	V19
DEFAULT	MSS GPIO_2[0] User LED 0	MSS GPIO_2[1] User LED 1	MSS GPIO_2[2] User LED 2
GPIOs	MSS GPIO_2[0] User LED 0	MSS GPIO_2[1] User LED 1	MSS GPIO_2[2] User LED 2
ROBOTICS	MSS GPIO_2[0] User LED 0	MSS GPIO_2[1] User LED 1	MSS GPIO_2[2] User LED 2

P8.06-P8.09

Pin	P8.06	P8.07	P8.08	P8.09
Name	HSIO12PB0/CLKIN_N_9/CCC_NE_CLKIN_N_5	HSIO30NB0	HSIO30PB0/CLKIN_N_3/CCC_NW_CLKIN_N_7	HSIO8NB0
BALL	V20	V15	V14	V21
DEFAULT	MSS GPIO_2[3] User LED 3	MSS GPIO_2[4] User LED 4	MSS GPIO_2[5] User LED 5	MSS GPIO_2[6] User LED 6
GPIOs	MSS GPIO_2[3] User LED 3	MSS GPIO_2[4] User LED 4	MSS GPIO_2[5] User LED 5	MSS GPIO_2[6] User LED 6
ROBOTICS	MSS GPIO_2[3] User LED 3	MSS GPIO_2[4] User LED 4	MSS GPIO_2[5] User LED 5	MSS GPIO_2[6] User LED 6

P8.10-P8.13

Pin	P8.10	P8.11	P8.12	P8.13
Name	HSIO8PB0/CCC_NE_CLKIN_N_10/CCC_NE_PLL0_OUT	HSIO7NB0	HSIO7PB0/CCC_NE_PLL0_OUT	GPIO47PB1
BALL	W21	Y21	Y20	B10
DEFAULT	MSS GPIO_2[7] User LED 7	MSS GPIO_2[8] User LED 8	MSS GPIO_2[9] User LED 9	core_pwm[1] @ 0x41500000 PWM_2:1
GPIOs	MSS GPIO_2[7] User LED 7	MSS GPIO_2[8] User LED 8	MSS GPIO_2[9] User LED 9	MSS GPIO_2[10] User LED 10
ROBOTICS	MSS GPIO_2[7] User LED 7	MSS GPIO_2[8] User LED 8	MSS GPIO_2[9] User LED 9	core_pwm[1] @ 0x41500000 PWM_2:1

P8.14-P8.16

Pin	P8.14	P8.15	P8.16
Name	GPIO47NB1	HSIO34NB0	HSIO34PB0/CCC_NW_CLKIN_N_1
BALL	B9	T12	U12
DEFAULT	MSS GPIO_2[11] User LED 11	MSS GPIO_2[12] GPIO	MSS GPIO_2[13] GPIO
GPIOs	MSS GPIO_2[11] User LED 11	MSS GPIO_2[12] GPIO	MSS GPIO_2[13] GPIO
ROBOTICS	MSS GPIO_2[11] User LED 11	MSS GPIO_2[12] GPIO	MSS GPIO_2[13] GPIO

P8.17-P8.19

Pin	P8.17	P8.18	P8.19
Name	HSIO29PB0	HSIO15PB0/DQS/CCC_NE_PLL1_OUT0	HSIO19NB0
BALL	W13	T16	W18
DEFAULT	MSS GPIO_2[14] GPIO	MSS GPIO_2[15] GPIO	core_pwm[0] @ 0x41500000 PWM_2:0
GPIOs	MSS GPIO_2[14] GPIO	MSS GPIO_2[15] GPIO	MSS GPIO_2[16] GPIO
ROBOTICS	MSS GPIO_2[14] GPIO	MSS GPIO_2[15] GPIO	core_pwm[0] @ 0x41500000 PWM_2:0

P8.20-P8.22

Pin	P8.20	P8.21	P8.22
Name	HSIO15NB0/DQS	HSIO9PB0/DQS/CCC_NE_PLL0_OUT0	HSIO9NB0/DQS
BALL	R16	AA21	AA22
DEFAULT	MSS GPIO_2[17] GPIO	MSS GPIO_2[18] GPIO	MSS GPIO_2[19] GPIO
GPIOs	MSS GPIO_2[17] GPIO	MSS GPIO_2[18] GPIO	MSS GPIO_2[19] GPIO
ROBOTICS	MSS GPIO_2[17] GPIO	MSS GPIO_2[18] GPIO	MSS GPIO_2[19] GPIO

P8.23-P8.26

Pin	P8.23	P8.24	P8.25	P8.26
Name	HSIO18PB0/CLKIN_N_7	HSIO18NB0	HSIO16PB0	GPIO49NB1
BALL	AB18	AA18	V17	A12
DEFAULT	MSS GPIO_2[20] GPIO	MSS GPIO_2[21] GPIO	MSS GPIO_2[22] GPIO	MSS GPIO_2[23] GPIO
GPIOs	MSS GPIO_2[20] GPIO	MSS GPIO_2[21] GPIO	MSS GPIO_2[22] GPIO	MSS GPIO_2[23] GPIO
ROBOTICS	MSS GPIO_2[20] GPIO	MSS GPIO_2[21] GPIO	MSS GPIO_2[22] GPIO	MSS GPIO_2[23] GPIO

P8.27-P8.29

Pin	P8.27	P8.28	P8.29
Name	GPIO49PB1/CLKIN_S_5	GPIO51NB1	GPIO51PB1/CLKIN_S_6
BALL	A13	B14	B13
DEFAULT	MSS GPIO_2[24] GPIO	MSS GPIO_2[25] GPIO	MSS GPIO_2[26] GPIO
GPIOs	MSS GPIO_2[24] GPIO	MSS GPIO_2[25] GPIO	MSS GPIO_2[26] GPIO
ROBOTICS	MSS GPIO_2[24] GPIO	MSS GPIO_2[25] GPIO	MSS GPIO_2[26] GPIO

P8.30-P8.32

Pin	P8.30	P8.31	P8.32
Name	GPIO50NB1/DQS	GPIO50PB1/DQS	GPIO53NB1
BALL	D14	D13	B15
DEFAULT	MSS GPIO_2[27] GPIO	core_gpio[0] @ 0x41100000 GPIO	core_gpio[1] @ 0x41100000 GPIO
GPIOs	MSS GPIO_2[27] GPIO	core_gpio[0] @ 0x41100000 GPIO	core_gpio[1] @ 0x41100000 GPIO
ROBOTICS	MSS GPIO_2[27] GPIO	core_gpio[0] @ 0x41100000 GPIO	core_gpio[1] @ 0x41100000 GPIO

P8.33-P8.35

Pin	P8.33	P8.34	P8.35
Name	GPIO53PB1/CLKIN_S_7	GPIO52NB1/LPRB_B	GPIO52PB1/LPRB_A
BALL	A15	C15	C14
DEFAULT	core_gpio[2] @ 0x41100000 GPIO	core_gpio[3] @ 0x41100000 GPIO	core_gpio[4] @ 0x41100000 GPIO
GPIOs	core_gpio[2] @ 0x41100000 GPIO	core_gpio[3] @ 0x41100000 GPIO	core_gpio[4] @ 0x41100000 GPIO
ROBOTICS	core_gpio[2] @ 0x41100000 GPIO	core_gpio[3] @ 0x41100000 GPIO	core_gpio[4] @ 0x41100000 GPIO

P8.36-P8.38

Pin	P8.36	P8.37	P8.38
Name	GPIO37NB1	GPIO37PB1/CCC_SW_CLKIN_S_1	GPIO3NB1
BALL	B4	C4	C17
DEFAULT	core_gpio[5] @ 0x41100000 GPIO	core_gpio[6] @ 0x41100000 GPIO	core_gpio[7] @ 0x41100000 GPIO
GPIOs	core_gpio[5] @ 0x41100000 GPIO	core_gpio[6] @ 0x41100000 GPIO	core_gpio[7] @ 0x41100000 GPIO
ROBOTICS	core_gpio[5] @ 0x41100000 GPIO	core_gpio[6] @ 0x41100000 GPIO	core_gpio[7] @ 0x41100000 GPIO

P8.39-P8.41

Pin	P8.39	P8.40	P8.41
Name	GPIO3PB1/CCC_SE_CLKIN_S_10/CCC_SE_PLL1_OUT0	GPIO5NB1	GPIO5PB1/CCC_SE_CLKIN_S_11
BALL	B17	B18	A18
DEFAULT	core_gpio[8] @ 0x41100000 GPIO	core_gpio[9] @ 0x41100000 GPIO	core_gpio[10] @ 0x41100000 GPIO
GPIOs	core_gpio[8] @ 0x41100000 GPIO	core_gpio[9] @ 0x41100000 GPIO	core_gpio[10] @ 0x41100000 GPIO
ROBOTICS	core_gpio[8] @ 0x41100000 GPIO	core_gpio[9] @ 0x41100000 GPIO	core_gpio[10] @ 0x41100000 GPIO

P8.42-P8.44

Pin	P8.42	P8.43	P8.44
Name	GPIO36NB1	GPIO36PB1/CCC_SW_CLKIN_S_0	GPIO42NB1
BALL	D6	D7	D8
DEFAULT	core_gpio[11] @ 0x41100000 GPIO	core_gpio[12] @ 0x41100000 GPIO	core_gpio[13] @ 0x41100000 GPIO
GPIOs	core_gpio[11] @ 0x41100000 GPIO	core_gpio[12] @ 0x41100000 GPIO	core_gpio[13] @ 0x41100000 GPIO
ROBOTICS	core_gpio[11] @ 0x41100000 GPIO	core_gpio[12] @ 0x41100000 GPIO	core_gpio[13] @ 0x41100000 GPIO

P8.45-P8.46

Pin	P8.45	P8.46
Name	GPIO42PB1	GPIO4PB1/CCC_SE_PLL1_OUT1
BALL	D9	D18
DEFAULT	core_gpio[14] @ 0x41100000 GPIO	core_gpio[15] @ 0x41100000 GPIO
GPIOs	core_gpio[14] @ 0x41100000 GPIO	core_gpio[15] @ 0x41100000 GPIO
ROBOTICS	core_gpio[14] @ 0x41100000 GPIO	core_gpio[15] @ 0x41100000 GPIO

4.1.2 Connector P9

The following tables show the pinout of the **P9** expansion header. The gateway image is responsible for setting the function of each pin. Refer to the processor documentation for more information on these pins and detailed descriptions of all of the pins listed. In some cases there may not be enough signals to complete a group of signals that may be required to implement a total interface.

The column heading is the pin number on the expansion header.

The **Name** row is the pin name on the processor.

The **BALL** row is the pin number on the processor.

The rows below **BALL** are the gateway setting for each pin.

NOTES:

DO NOT APPLY VOLTAGE TO ANY I/O PIN WHEN POWER IS NOT SUPPLIED TO THE BOARD. IT WILL DAMAGE THE PROCESSOR AND VOID THE WARRANTY.

NO PINS ARE TO BE DRIVEN UNTIL AFTER THE SYS_RESET LINE GOES HIGH.

P9.01-P9.05

P9.01	P9.02	P9.03	P9.04	P9.05
GND	GND	VCC_3V3	VCC_3V3	VDD_5V

P9.06-P9.10

P9.06	P9.07	P9.08	P9.10
VDD_5V	SYS_5V	SYS_5V	SYS_RSTN

Pin	P9.09
Name	HSIO19PB0
BALL	W19

P9.11-P9.13

Pin	P9.11	P9.12	P9.13
Name	GPIO38NB1/DQS	GPIO38PB1/DQS/CCC_SW_PLL1_OUT0	GPIO2NB1/DQS
BALL	B5	C5	D19
DEFAULT	MMUART4 UART4 RX	core_gpio[1] @ 0x41200000 GPIO	MMUART4 UART4 TX
GPIOs	core_gpio[0] @ 0x41200000 GPIO	core_gpio[1] @ 0x41200000 GPIO	core_gpio[2] @ 0x41200000 GPIO
ROBOTICS	~ ~	core_gpio[0] @ 0x41200000 GPIO	core_gpio[7] @ 0x41200000 GPIO

P9.14-P9.16

Pin	P9.14	P9.15	P9.16
Name	GPIO39PB1/CLKIN_S_2/CCC_SW_CLKIN_S_2/CCC_SW_PLL1_O	GPIO40NB1	GPIO40PB1/CCC_SW_PLL1_OUT1
BALL	C6	A5	A6
DEFAULT	core_pwm[0] @ 0x41400000 PWM_1:0	core_gpio[4] @ 0x41200000 GPIO	@ core_pwm[1] @ 0x41400000 PWM_1:1
GPIOs	core_gpio[3] @ 0x41200000 GPIO	core_gpio[4] @ 0x41200000 GPIO	@ core_gpio[5] @ 0x41200000 GPIO
ROBOTICS	core_pwm[0] @ 0x41400000 PWM_1:0	core_gpio[1] @ 0x41200000 GPIO	@ core_pwm[1] @ 0x41400000 PWM_1:1

P9.17-P9.19

Pin	P9.17	P9.18	P9.19
Name	GPIO44NB1/DQS	GPIO44PB1/DQS/CCC_SW_PLL0_OUT0	GPIO45PB1/CCC_SW_PLL0_OUT0
BALL	C9	C10	A10
DEFAULT	~ ~	~ ~	MSS I2C0 I2C0 SCL
GPIOs	core_gpio[6] @ 0x41200000 GPIO	core_gpio[7] @ 0x41200000 GPIO	MSS I2C0 I2C0 SCL
ROBOTICS	~ ~	~ ~	MSS I2C0 I2C0 SCL

P9.20-P9.22

Pin	P9.20	P9.21	P9.22
Name	GPIO45NB1	GPIO43NB1	GPIO43PB1
BALL	A11	B8	A8
DEFAULT	MSS I2C0 I2C0 SDA	~ ~	~ ~
GPIOs	MSS I2C0 I2C0 SDA	core_gpio[8] @ 0x41200000 GPIO	core_gpio[8] @ 0x41200000 GPIO
ROBOTICS	MSS I2C0 I2C0 SDA	~ ~	~ ~

P9.23-P9.25

Pin	P9.23	P9.24	P9.25
Name	GPIO48NB1	GPIO48PB1/CLKIN_S_4	GPIO41NB1
BALL	C12	B12	B7
DEFAULT	core_gpio[10] @ 0x41200000 GPIO	~ ~	core_gpio[12] @ 0x41200000 GPIO
GPIOs	core_gpio[10] @ 0x41200000 GPIO	core_gpio[11] @ 0x41200000 GPIO	core_gpio[12] @ 0x41200000 GPIO
ROBOTICS	core_gpio[2] @ 0x41200000 GPIO	~ ~	core_gpio[3] @ 0x41200000 GPIO

P9.26-P9.28

Pin	P9.26	P9.27	P9.28
Name	GPIO41PB1/CLKIN_S_3/CCC_SW_CLKIN_S_3	GPIO46NB1	GPIO46PB1/CCC_SW_PLL0_OUT1
BALL	A7	D11	C11
DEFAULT	~ ~	core_gpio[14] @ 0x41200000 GPIO	~ ~
GPIOs	core_gpio[13] @ 0x41200000 GPIO	core_gpio[14] @ 0x41200000 GPIO	core_gpio[15] @ 0x41200000 GPIO
ROBOTICS	~ ~	~ ~	~ ~

P9.29-P9.31

Pin	P9.29	P9.30	P9.31
Name	GPIO1PB1/CLKIN_S_9/CCC_SE_CLKIN_S_9	GPIO1NB1	GPIO4NB1
BALL	F17	F16	E18
DEFAULT	~ ~	core_gpio[17] @ 0x41200000 GPIO	~ ~
GPIOs	core_gpio[16] @ 0x41200000 GPIO	core_gpio[17] @ 0x41200000 GPIO	core_gpio[18] @ 0x41200000 GPIO
ROBOTICS	~ ~	core_gpio[5] @ 0x41200000 GPIO	~ ~

P9.32-P9.40

P9.32	P9.34
VDD_ADC	GND

P9.33	P9.35	P9.36	P9.37	P9.38	P9.39	P9.40
AIN4	AIN6	AIN5	AIN2	AIN3	AIN0	AIN1

P9.41-P9.42

Pin	P9.41	P9.42
Name	GPIO0PB1/CLKIN_S_8/CCC_SE_CLKIN_S_8/CCC_SE_PLL0_OUT0	GPIO0NB1
BALL	E15	E14
DEFAULT	core_gpio[19] @ 0x41200000 GPIO	core_pwm[0] @ 0x41000000 PWM_0:0
GPIOs	core_gpio[19] @ 0x41200000 GPIO	core_gpio[20] @ 0x41200000 GPIO
ROBOTICS	core_gpio[19] @ 0x41200000 GPIO	~ ~

P9.43-P9.46

P9.43	P9.44	P9.45	P9.46
GND	GND	GND	GND

Chapter 5

Demos

5.1 Upgrade BeagleV-Fire Gateway

This document describes how to upgrade your BeagleV-Fire's gateway. This approach can be used out of the box using Linux commands executed on BeagleV-Fire

5.1.1 Required Equipment

- BeagleV-Fire board
- USB-C cable
- Ethernet cable

The USB-C cable provides power, a serial interface to BeagleV-Fire and allows connecting to BeagleV-Fire through a browser using IP address `192.168.7.2`.

The Ethernet cable connected to your local network (LAN) allows connecting to BeagleV-Fire using the SSH protocol. It also allows BeagleV-Fire to retrieve updated packages through your local network's Internet connection.

5.1.2 Connect to BeagleV-Fire Linux Command Line Interface

BeagleV-Fire boots Linux out of the box. Like all Beagleboard boards there are several methods to get BeagleV-Fire's Linux command prompt.

- Cockpit
- SSH
- Serial port

Cockpit

Enter the following URL in your web browser: `https://beaglev.localdomain:9090/`

On first use, click through the security warning. Login using `beagle/temppwd` as user/password. Click on Terminal in the left pane. You now have a Linux command prompt running on your BeagleV-Fire. Next step: enter the commands described in the Gateway Upgrade Linux Commands section of this document.

Note: You can connect to the Cockpit using the IP address dynamically assigned to your BeagleV-Fire in your local Ethernet network. One method of finding the value of that dynamically assigned IP address is to open

a serial terminal through the USB port and use the `ip address` Linux command. Please refer to the USB Serial Port section.

SSH

Like all Beagleboard boards, you can SSH to the board through the USB interface by using IP address 192.168.7.2.

Note: On Windows, this approach may require some drivers to be updated or installed. Use one of the other approaches if you are not immediately successful with this one. You can circle back later to adjust your Windows installation if required.

Serial Port

A serial port is available through the USB-C port. This serial port becomes available once Linux has booted on BeagleV-Fire. Please wait a couple of minutes after powering up the board before looking for additional serial ports reported by your host computer's operating system. You can then use your favorite serial port terminal tool such as Putty or Screen to access the BeagleV-Fire Linux command prompt.

For example on your Linux host computer:

```
screen /dev/ttyACM0 115200
```

Where `ttyACM0` is an additional serial port that appeared after BeagleV-Fire was connected to your Linux host computer. This serial port can be identified using the `dmesg | grep tty` Linux command which will show the most recent serial port added to the host computer.

On Windows, BeagleV-Fire's serial port number will show in the Windows Device Manager. Use that serial port number in Putty with a speed 115200 baud, no flow-control.

5.1.3 Gateway Upgrade Linux Commands

Note: BeagleV-Fire needs to be connected to the internet through your local network for the commands in this section to work. The connection can be through the Ethernet port or the Wi-Fi module.

Install bbb.io-gateway

You need to install the `bbb.io-gateway` package. This will allow retrieving the most up-to-date gateway.

```
sudo apt install bbb.io-gateway
```

Retrieve Available Updated Linux packages List

The list will include the latest BeagleV-Fire gateway packages.

```
sudo apt update
```

Upgrade Linux Packages

This will upgrade the BeagleV-Fire gateway Linux programming files located under `/usr/share/beagleboard/gateway`. Several directories are found in that location, each containing programming files for one individual gateway configuration.

```
sudo apt upgrade
```

Launch Reprogramming of BeagleV-Fire's FPGA

Change directory to `/usr/share/beagleboard/gateway`. This directory contains a script performing the gateway's reprogramming. It also contains one directory for each of the possible gateway configuration that can be programmed into your BeagleV-Fire. The name of one of these directories is passed as argument to the script to specify which gateway configuration you wish to program your BeagleV-Fire with.

```
cd /usr/share/beagleboard/gateway
. ./change-gateway.sh default
```

Important: Do not power-off BeagleV-Fire until it has rebooted by itself. The gateway reprogramming may take a couple of minutes.

The change-gateway script programs the selected gateway and its associated device tree overlays into the PolarFire SoC System Controllers SPI flash and triggers a software reboot. During the reboot, the Hart Software Services (HSS) will request the PolarFire SoC System Controller to reprogram the FPGA and eNVM. The PolarFire SoC System Controller will reprogram the FPGA if it finds it contains a different design version than the one in the SPI Flash. The board reboots on completion of the FPGA reprogramming.

5.2 Flashing gateway and Linux image

In this tutorial we are going to learn to flash the gateway image to FPGA and `sdcard.image` to eMMC storage.

Important: Additional hardware required:

1. FlashPro5/6 programmer
 2. Tag connect TC2050-IDC-NL 10-Pin No-Legs Cable with Ribbon connector
 3. TC2050-CLIP-3PACK Retaining CLIP board for TC2050-NL cables
-

5.2.1 Programming & Debug tools installation

To flash a gateway image to your BeagleV-Fire board you will require a FlashPro5/6 and FlashPro Express (FPEXpress) tool which comes pre-installed as part of [Libero SoC Design Suite](#). A standalone FlashPro Express tool is also available with MicroChip's [Programming and Debug Tools](#) package, which we are going to use for this tutorial. Below are the steps to install the software:

1. Download the zip for your operating system from [Programming and Debug Tools](#) page.
2. Unzip the file and in the unzipped folder you will find `launch_installer.sh` and `Program_Debug_v2023.1.bin`.
3. Execute the `launch_installer.sh` script to start the installation procedure.

```
[lorforlinux@fedora Program_Debug_v2023.1_lin] $ ./launch_installer.sh  
No additional packages to install for installer usage  
Requirement search complete.  
See /tmp/check_req_installer608695.log for information.  
Launch of installer  
Preparing to install  
Extracting the JRE from the installer archive...  
Unpacking the JRE...
```

Note: It's recommended to install under *home/user/microchip* for linux users.

Enabling non-root user to access FlashPro

1. Download 60-openocd.rules
2. Copy udev rule `sudo cp 60-openocd.rules /etc/udev/rules.d`
3. Trigger udevadm using `sudo udevadm trigger` or reboot the PC for the changes to take effect

5.2.2 Flashing gateway image

Note: content below is valid for beta testers only.

Launch FPEXpress

1. Download FlashProExpress.zip file and unzip, it contains the *.job file which we need to create a new project in FPEXpress.
2. Open up a terminal and go to `/home/user/microchip/Program_Debug_v202X.Y/Program_Debug_Tool/bin` which includes FPEXpress tool.
3. Execute `./FPEXpress` in terminal to start FlashPro Express software.

Create new project

Important: Make sure you have your FlashPro5/6 connected before you create a new project.

Press CTRL+N to create a file and you will see a pop-up window like shown below,

Follow the steps below as annotated in the image above:

1. Click on browse (1) button to select the job file.
2. Click on browse (2) button to select the project location.
3. Click ok button to finish.

If your FlashPro5/6 is connected properly you'll see the window shown below:

Following the annotation in the image above:

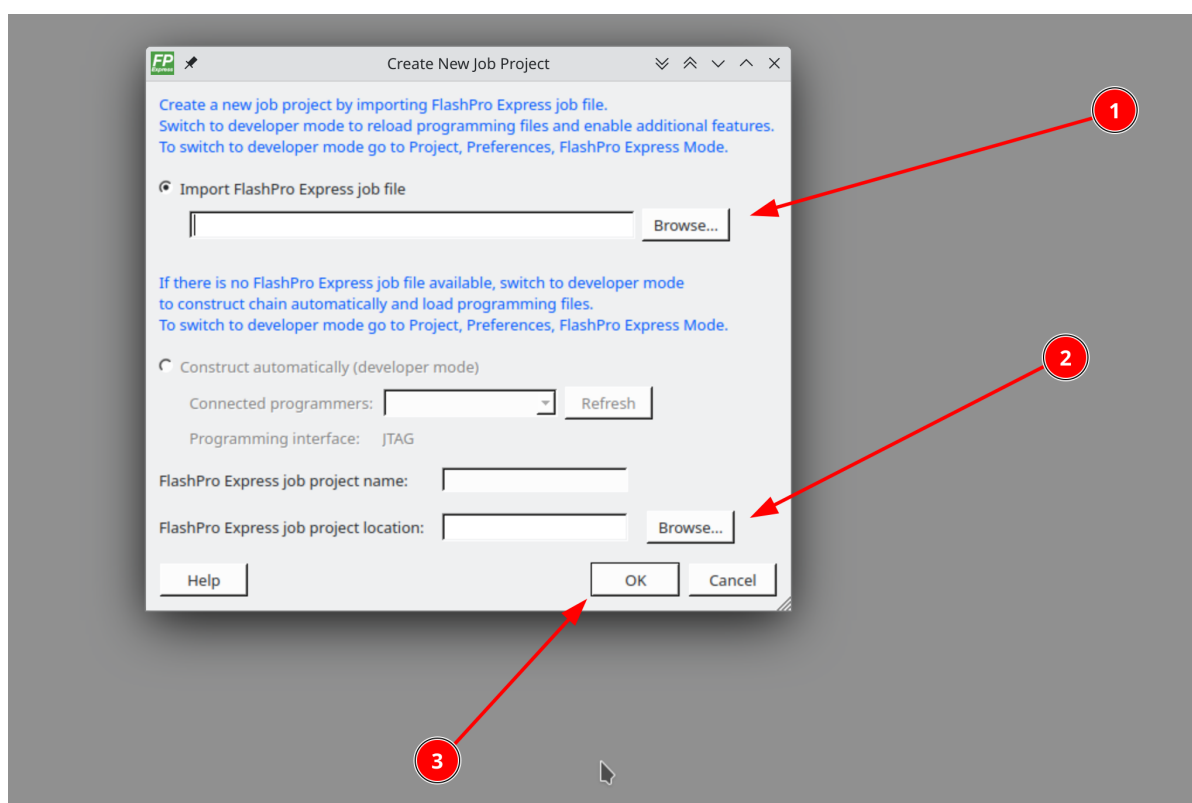


Fig. 5.1: FPEXpress new project creation window

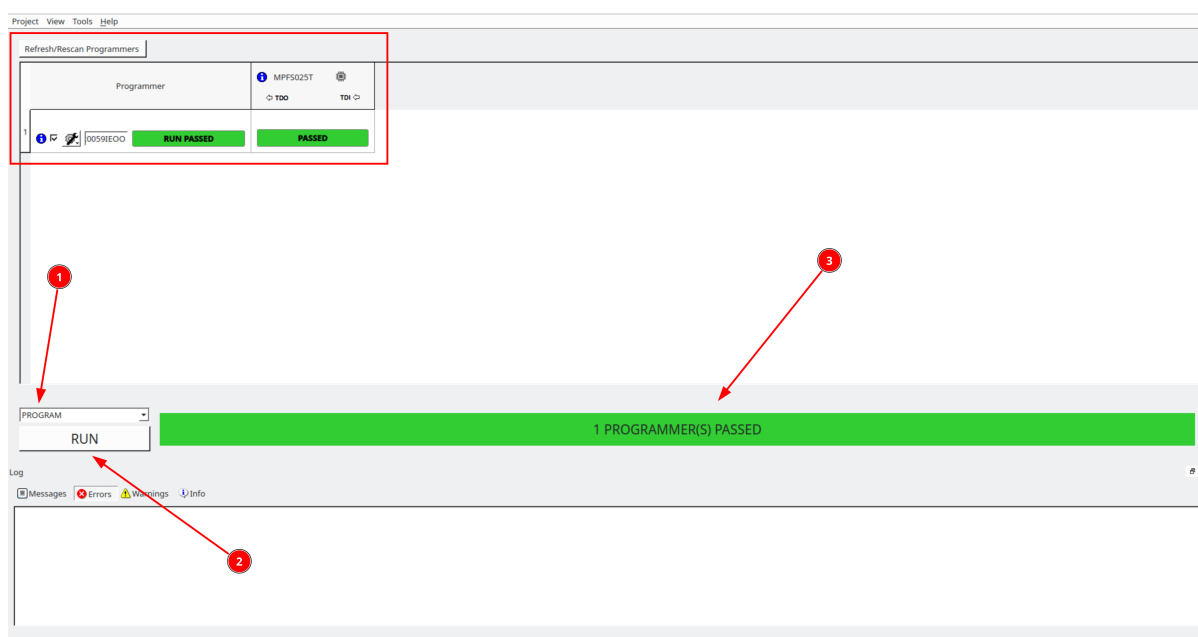


Fig. 5.2: FPEXpress new project flash window

1. From drop-down select Program action
2. Click on RUN button
3. Shows the progress

If you see a lot of green color and the progress bar says PASSED then well done you have successfully flashed the gateway image on your BeagleV-Fire board.

5.2.3 Flashing eMMC

Connect to BeagleV-Fire UART debug port using a 3.3v USB to UART bridge.

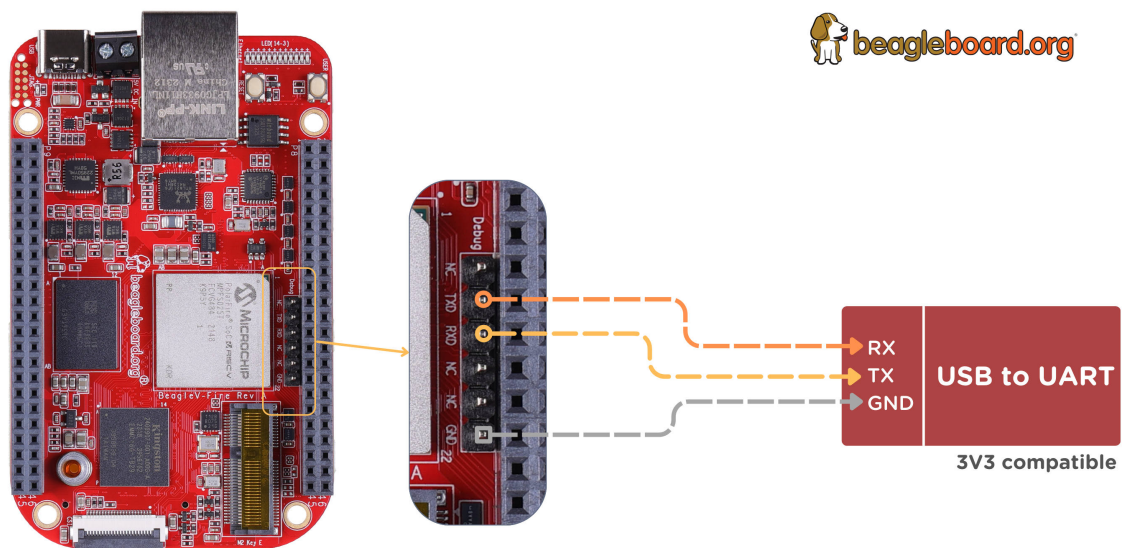


Fig. 5.3: UART debug connection

Now you can run `tio <port> -b 115200` in a terminal window to access the UART debug port connection. Once you are connected properly you can press the Reset button which will show you a progress bar like in the

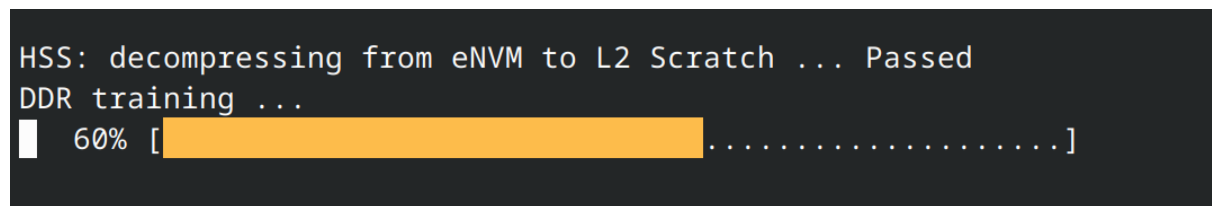


Fig. 5.4: BeagleV-Fire booting HSS with progress bar

Once you see that progress bar on your screen you can start pressing any button (0-9/a-z) which will stop the board from fully booting and you'll be able to access Hart Software Services (HSS) prompt. BeagleV-Fire's eMMC content is written by the Hart Software Services (HSS) using the `usbdmisc` command. The HSS `usbdmisc` command exposes the eMMC as a USB mass storage device USB type C connector.

Once you see `>>` you can execute the commands below:

1. `>> mmc`

5.2. Flashing gateway and Linux image 43

2. >> `usbdmso`

```
[9.595339] >> mmc
[8647.667675] Selecting SDCARD/MMC (fallback) as boot source ...
[8647.674532] Attempting to select eMMC ... Passed
[8647.929915] >> usbdmso
[8655.594694] initialize MMC
[8655.598426] Attempting to select eMMC ... Passed
[8655.853809] MMC - 512 byte pages, 512 byte blocks, 30621696 pages
Waiting for USB Host to connect... (CTRL-C to quit)
. 0 bytes written, 0 bytes readUSB Host connected. Waiting for disconnect... (CTRL-C to quit)
. 0 bytes written, 2300416 bytes read
```

Fig. 5.6: HSS commands to show eMMC as mass storage

After executing the commands above your BeagleV-Fire's eMMC will be exposed as a mass storage device like shown in the image below:

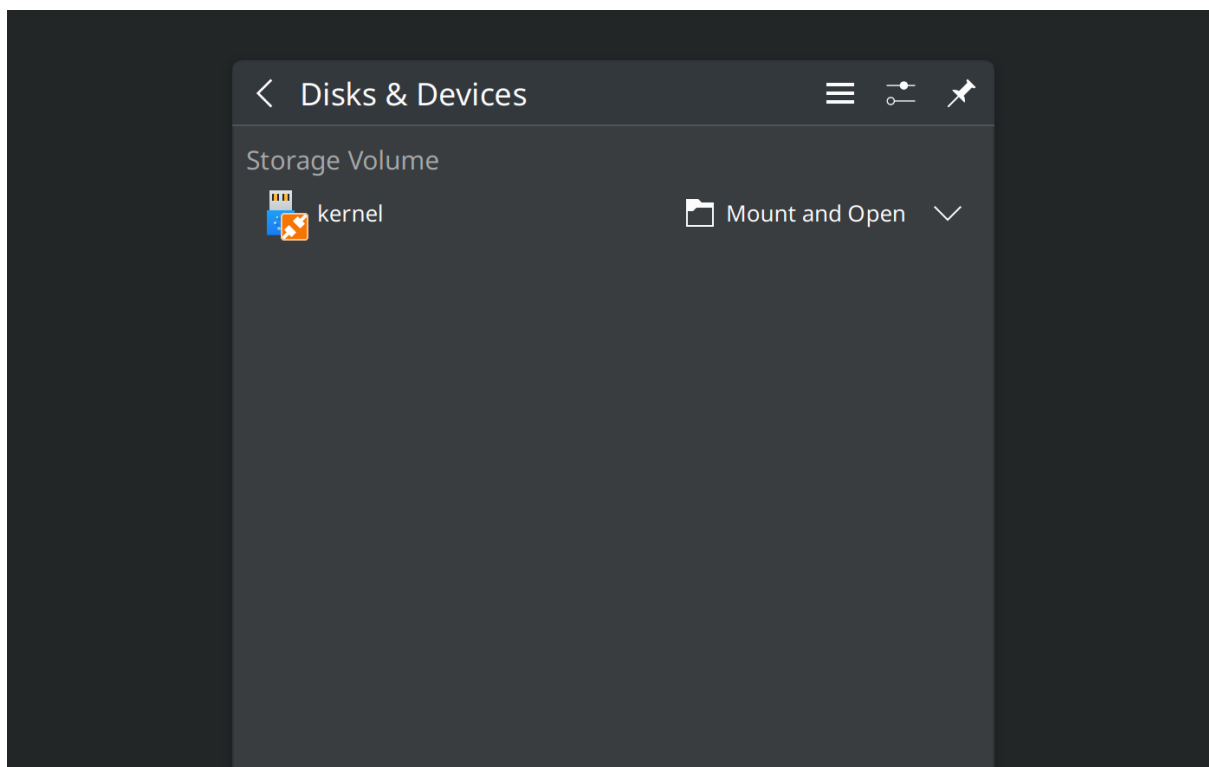


Fig. 5.7: BeagleV-Fire eMMC as mass storage

Once your board is exposed as a mass storage device, you can proceed to flash the `sdcard.img` on your BeagleV-Fire's eMMC.

This document outlines two methods to flash from your local machine to BeagleV-Fire's eMMC.

1. First method is to use [Balena Etcher](#) software. This software can be used to flash image in either Windows or Linux operating system.

Select image

1. Select the `sdcard.img` file from your local drive storage.
2. Click on select target.

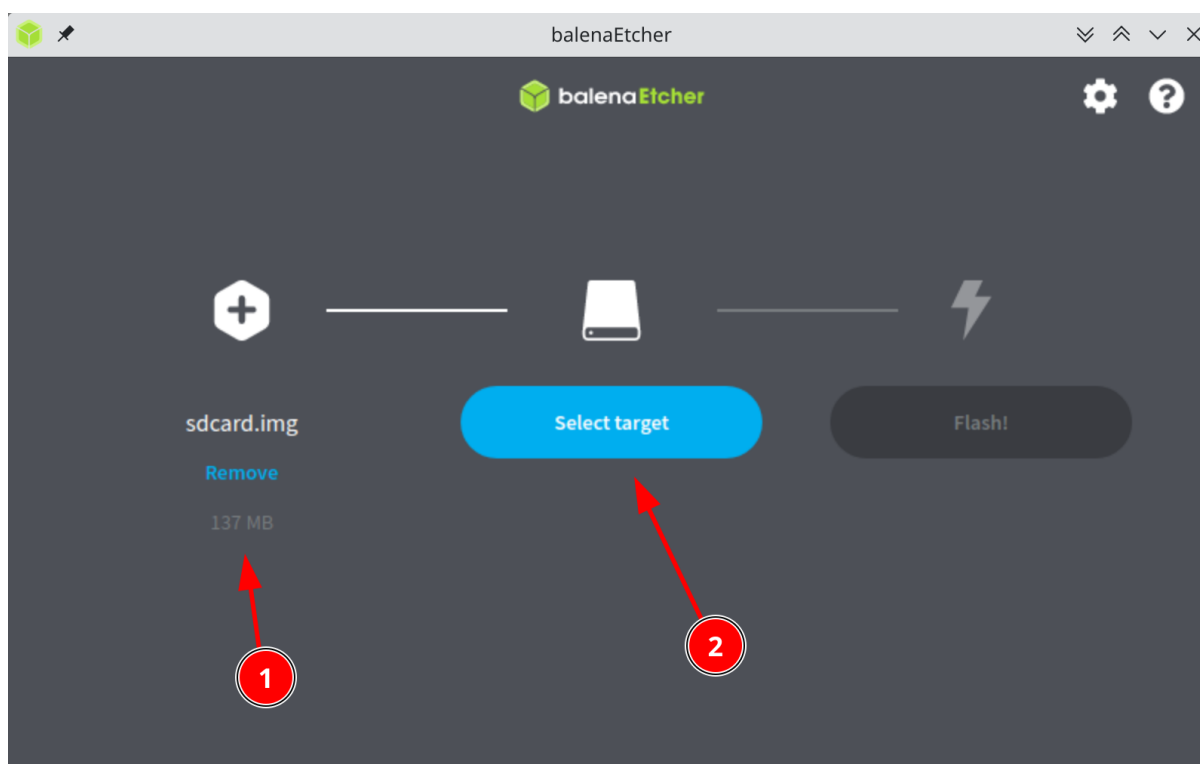


Fig. 5.8: Balena Etcher selecting image

Select Target

1. Select MCC PolarFireSoC_msd as target.
2. Click Select (1) to proceed.

Flash image

1. Click on Flash! to flash the `sdcard.img` on BeagleV-Fire eMMC storage.
2. Second method is suitable for flashing the image using Linux machine via the command line.

```
sudo dd if=output/images/sdcard.img of=/dev/sdX bs=1M status=progress
```

Note: You need to replace `/dev/sdX` with the actual device name of your eMMC. Use tools like `dmesg`, `lsblk`, or `GNOME Disks` before and after exposing your device as a USB to identify the correct device name. Be very careful not to overwrite the wrong drive, as this action is irreversible.

- Once the transfer is complete, type `CTRL+C` to disconnect your device
- Finally boot the new Linux image by typing `boot` or reset your board

Congratulations! with that done you have fully updated BeagleV-Fire board with up to date gateway image on it's PolarFire SoC's FPGA Fabric and linux image on it's eMMC storage.

5.3 Microchip 2025.1 FPGA Tools Installation Guide

Instructions for installing the Microchip FPGA tools on a Ubuntu 20.04 or 22.04 desktop.

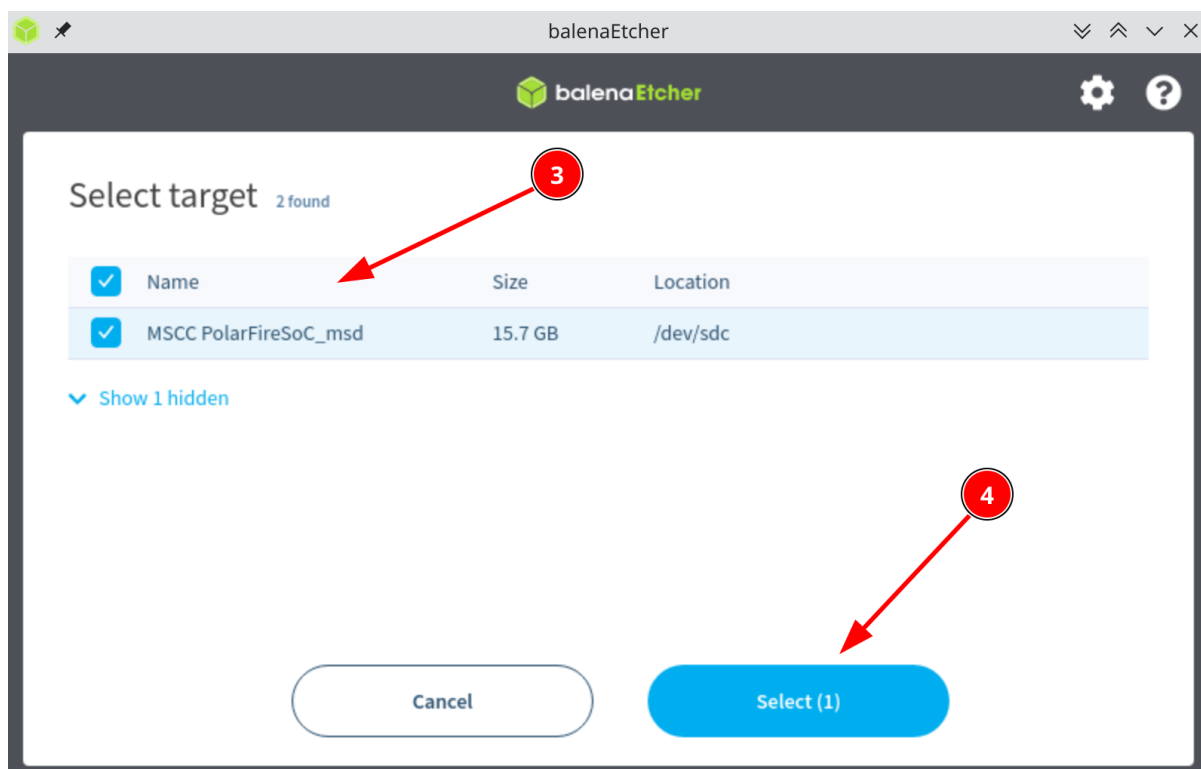


Fig. 5.9: Balena Etcher selecting target

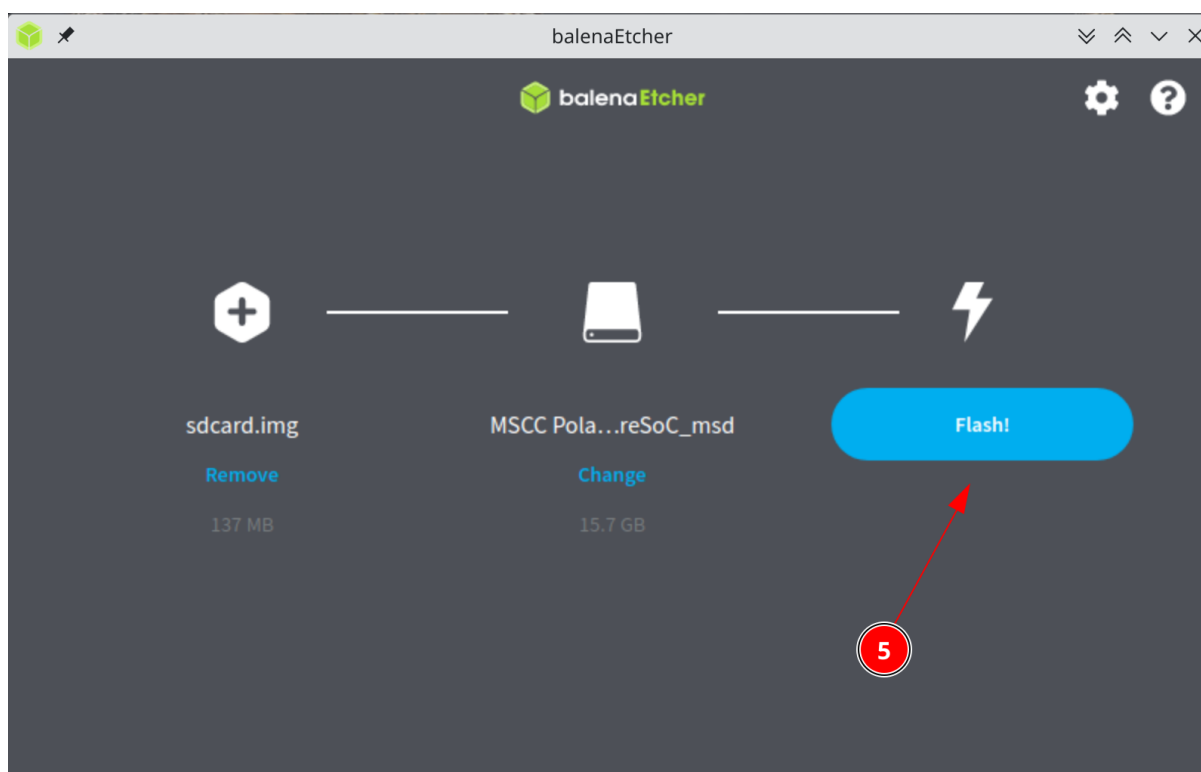


Fig. 5.10: Balena Etcher flashing image

Important: We will be providing instances of Libero that you can run from git.beagleboard.org's gitlab-runners such that you do not need to install the tools on your local machine.

5.3.1 Install Libero SoC

Download the `Microchip.zip` folder into your `/home/$USER/` directory:

`Microchip.zip`

Unzip the downloaded file:

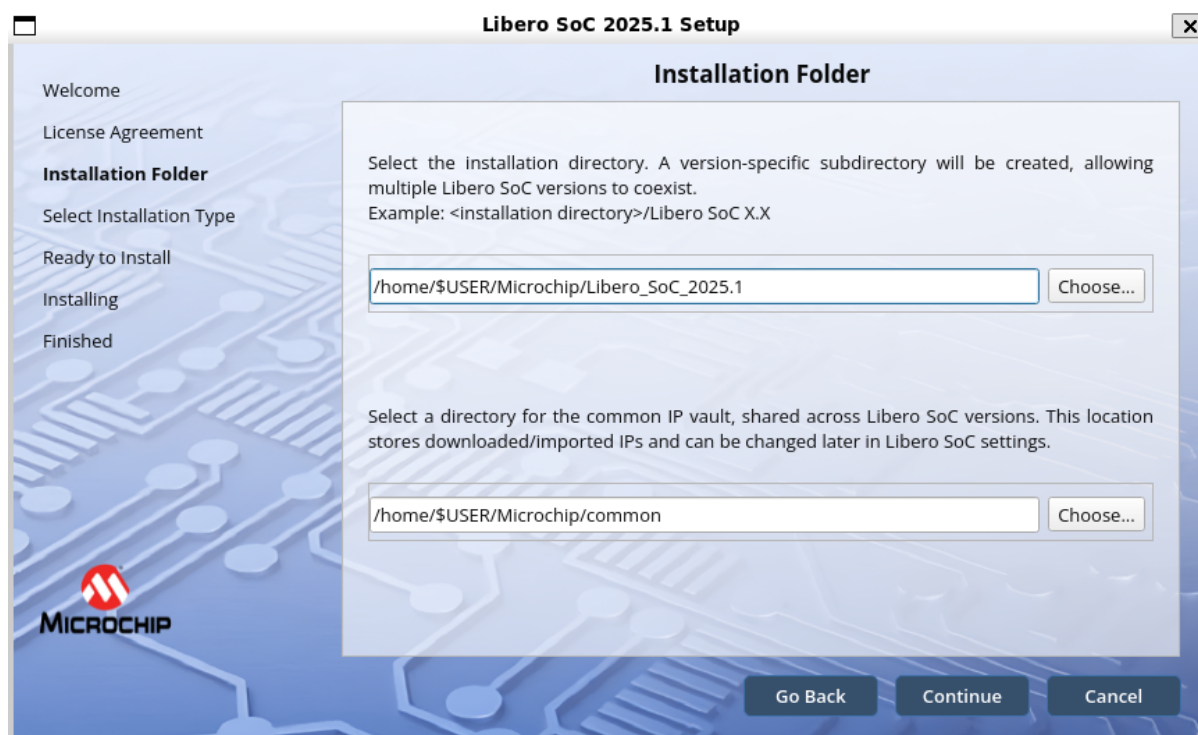
```
unzip Microchip.zip
```

- Download the Libero SoC installer from [Microchip's fpga and soc design tools section](#).

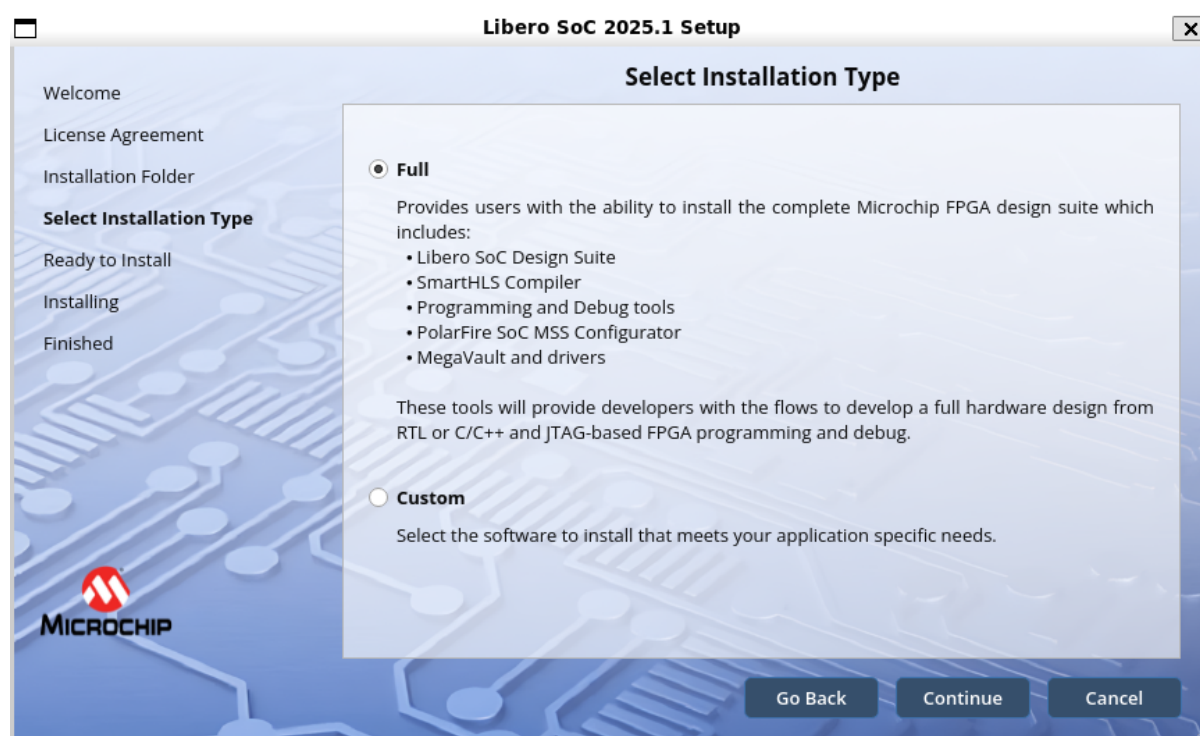
```
unzip libero_soc_2025.1_online_lin.zip
sudo ./Libero_SoC_2025.1_online_lin.sh
```

Important: Install into: `~/Microchip/Libero_SoC_2025.1`

The following images show the options to select during the installation process:



Important: Select the `Full` installation option.



Run the post installation script which will install missing packages:

```
sudo /home/$USER/Microchip/Libero_SoC_2025.1/req_to_install.sh
```

5.3.2 Install SoftConsole 2022.2

- Download the installer from the [Microchip website](#).

```
sudo chmod +x Microchip-SoftConsole-v2022.2-RISC-V-747-linux-x64-installer.  
→run  
./Microchip-SoftConsole-v2022.2-RISC-V-747-linux-x64-installer.run
```

Accept the license, Click Forward, Finish.

Follow the post-installation steps or more conveniently, install the required packages using the streamlined script in `/home/$USER/Microchip/Package-Install`

```
chmod +x install-required-packages.sh  
sudo ./install-required-packages.sh
```

Run the following commands to copy the rules file, enabling non-root user to access FlashPro:

```
cd /home/$USER/Microchip/SoftConsole-v2022.2-RISC-V-747/openocd/share/  
→openocd/contrib  
sudo cp 60-openocd.rules /etc/udev/rules.d  
sudo udevadm trigger
```

5.3.3 Request a Libero Silver license

- Visit [microchip's fpga software products page](#)
- Choose "Libero Silver 1Yr Floating License for Windows/Linux Server" from the list.
- Enter your MAC address and click register.

Note: A MAC address looks something like 12:34:56::78:ab:cd when you use the “ip address” command to find out its value on your Linux machine. However, you need to enter it as 123456abcd in this dialog box.

You will get an email with a License.dat file. Copy it into the ~/Microchip/license directory. Then run the provided license file setup script in the same directory:

```
chmod +x setup-license-file-2025-1.sh
./setup-license-file.sh
```

5.3.4 Setup tools script

Add this to your ~/.bashrc file.

First, type:

```
nano ~/.bashrc
```

Then, copy the following lines to the end of the file:

```
if [ -f "/home/$USER/Microchip/FPGA-Tools-Setup/setup-microchip-2025-1-tools.
→sh" ]; then
. "/home/$USER/Microchip/FPGA-Tools-Setup/setup-microchip-2025-1-tools.sh"
fi
```

To deal with a ca-certificates issue, it is recommended to run the following commands:

```
sudo mkdir -p /etc/pki/tls/certs/
sudo ln -s /etc/ssl/certs/ca-certificates.crt /etc/pki/tls/certs/ca-bundle.
→crt
```

Your directory structure should appear like so:

```
Microchip/
├── common/
├── FPGA-Tools-Setup/
│   ├── setup-microchip-2025-1-tools.sh
│   └── Libero_SoC_2025.1/
│       ├── Libero_SoC/
│       ├── LicenseDaemons/
│       ├── MegaVault/
│       ├── req_to_install.sh
│       └── SmarthHLS/
├── license/
│   ├── License.dat
│   ├── license.log
│   └── setup-license-file-2025-1.sh
├── Package-Install/
│   └── install-required-packages.sh
└── SoftConsole-v2022.2-RISC-V-747/
```

You can now run the BeagleV-fire gateway scripts [BeagleV-fire gateway scripts](#).
and create your own custom design.

5.3.5 License Issues running the gateway script

If you encounter a license checkout error such as:

```
License Checkout Error: Cannot locate license file. Use LM_LICENSE_FILE to
→specify a different license file.Finished
```

Try running the following command to view the processes using the 1702 port:

```
lsof -i :1702
```

You should see output similar to:

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
lmgrd	2582	bb	4u	IPv6	44801	0t0	TCP	*:1702 (LISTEN)
lmgrd	2582	bb	9u	IPv6	52599	0t0	TCP	localhost:1702->localhost:42718 (ESTABLISHED)
lmgrd	2582	bb	10u	IPv6	52602	0t0	TCP	localhost:1702->localhost:42730 (ESTABLISHED)
lmgrd	2582	bb	11u	IPv6	52625	0t0	TCP	localhost:1702->localhost:39020 (ESTABLISHED)
actlmgrd	2584	bb	4u	IPv6	44801	0t0	TCP	*:1702 (LISTEN)
actlmgrd	2584	bb	12u	IPv4	53472	0t0	TCP	localhost:42718->localhost:1702 (ESTABLISHED)
saltd	2585	bb	4u	IPv6	44801	0t0	TCP	*:1702 (LISTEN)
saltd	2585	bb	16u	IPv4	54646	0t0	TCP	localhost:42730->localhost:1702 (ESTABLISHED)
snpslmd	2586	bb	4u	IPv6	44801	0t0	TCP	*:1702 (LISTEN)
snpslmd	2586	bb	58u	IPv4	47824	0t0	TCP	localhost:39020->localhost:1702 (ESTABLISHED)

Kill the `lmgrd` process using the port, in this example, the process ID is 2582:

```
sudo kill -9 2582
```

Now open a new terminal and rerun the gateway script. The issues should be resolved.

5.4 Microchip 2024.2 and Older FPGA Tools Installation Guide

Instructions for installing the Microchip FPGA tools on a Ubuntu 20.04 or Ubuntu 22.04 desktop.

Important: We will be providing instances of Libero that you can run from git.beagleboard.org's gitlab-runners such that you do not need to install the tools on your local machine.

5.4.1 Install Libero

Note: Libero 2023.2, 2024.1 or 2024.2 should work. 2024.2 is used as an example.

Create a folder named Microchip in your /home folder

- Download installer from the [Microchip's fpga and soc design tools section](#).
- Install Libero

```
unzip Libero_SoC_v2024.2_lin.zip
./launch_installer.sh
```

Important: Do not use the default location suggested by the Libero installer. Instead of `/usr/local/Microchip/Libero_SoC_v2024.2` install into `~/Microchip/Libero_SoC_v2024.2`

Run the post installation script which will install missing packages:

```
sudo /home/$USER/Microchip/Libero_SoC_v2024.2/Logs/req_to_install.sh
```

No need to run the FlashPro hardware installation scripts. This will be taken care of as part of the SoftConsole installation.

5.4.2 Install SoftConsole 2022.2

- Download installer from [Microchip website](#).

```
sudo chmod +x Microchip-SoftConsole-v2022.2-RISC-V-747-linux-x64-installer.  
→ run  
  
./Microchip-SoftConsole-v2022.2-RISC-V-747-linux-x64-installer.run
```

Accept the license, Click Forward, Finish.

Perform the post installation steps as described in the html file opened when you click Finish.

Important: Please pay special attention to the “Enabling non-root user to access FlashPro” section of the post-installation instructions. This will actually allow you to program the board using Libero.

5.4.3 Install the Libero licensing daemon

Download the latest 64 bit Licensing Daemons from the [Microchip’s fpga and soc design tools section](#)

- [Linux_Licensing_Daemon_11.19.6.0_64-bit.tar.gz](#)
- [Windows_Licensing_Daemon_11.19.6.0_64-bit.zip](#)

Older Daemon downloads can be found at [Microchip’s daemons section](#)

- [Linux_Licensing_Daemon_11.16.1_64-bit.tar.gz](#)
- [Windows_Licensing_Daemon_11.16.1_64-bit.zip](#)

Copy the downloaded file to the Microchip directory within your home directory and untar it.

```
cd ~/Microchip  
  
tar -xvf Linux_Licensing_Daemon_11.19.6.0_64-bit.tar.gz
```

Install the Linux Standard Base:

```
sudo apt-get update  
  
sudo apt-get -y install lsb
```

5.4.4 Request a Libero Silver license

- Visit [microchip’s fpga software products page](#)
- Choose “Libero Silver 1Yr Floating License for Windows/Linux Server” from the list.
- Enter your MAC address and click register.

Note: A MAC address looks something like 12:34:56::78:ab:cd when you use the “ip address” command to find out its value on your Linux machine. However, you need to enter it as 123456abcd in this dialog box.

You will get an email with a License.dat file. Copy it into the ~/Microchip/license directory. Replace *<put.hostname.here>* in *License.dat* with *localhost* and add *Linux_Licensing_Daemon* as the path to the daemons.

The top of your license file should look something like this after editing. Your daemon files should be located in the Linux_Licensing_Daemon folder inside the Microchip folder.

```
SERVER localhost 001584731680 1702
DAEMON actlmgrd Linux_Licensing_Daemon/actlmgrd
# Starting Libero SOC v2024.2, customers are recommended ...
# DAEMON mgcld Linux_Licensing_Daemon/mgcld
DAEMON salted Linux_Licensing_Daemon/salted
VENDOR snpslmd Linux_Licensing_Daemon/snpslmd
```

5.4.5 Execute tool setup script

Download the script:

Listing 5.1: Libero environment and license setup script

```
#!/bin/bash

# Gets Libero version

dir=$(pwd)
while [[ "$dir" != "/" ]]; do
    for subdir in "$dir"/*; do
        if [[ $subdir =~ Libero_SoC_v([0-9]+\.[0-9]+)/ ]]; then
            Libero_ver="${BASH_REMATCH[1]}"
            break 2
        fi
    done
    dir=$(dirname "$dir")
done

#Set preferred Libero version here if needed
#Libero_ver=2023.2

echo "Using Libero version:" $Libero_ver

# Check if Libero_ver was set; if not, print an error and exit
if [[ -z "$Libero_ver" ]]; then
    echo "Error: No directory found matching the pattern 'Libero_SoC_vXXXX.Y/'"
    return 1
fi

→#=====
# Edit the following section with the location where the following tools are
# installed if they aren't in the default location:
#   - SoftConsole (SC_INSTALL_DIR)
#   - Libero (LIBERO_INSTALL_DIR)
#   - Licensing daemon for Libero (LICENSE_DAEMON_DIR)
→#=====

export SC_INSTALL_DIR=/home/$USER/Microchip/SoftConsole-v2022.2-RISC-V-747
export LIBERO_INSTALL_DIR=/home/$USER/Microchip/Libero_SoC_v$Libero_ver
export LICENSE_DAEMON_DIR=/home/$USER/Microchip/Linux_Licensing_Daemon
export LICENSE_FILE_DIR=/home/$USER/Microchip/license
export SMARTHLS_INSTALL_DIR=$LIBERO_INSTALL_DIR/SmartHLS-$Libero_ver/SmartHLS

→#=====
```

(continues on next page)

(continued from previous page)

```

# The following was tested on Ubuntu 20.04 with:
#   - Libero 2023.2 and 2024.1
#   - SoftConsole 2022.2
# It was also tested on Ubuntu 22.04 with:
#   - Libero 2024.2
#   - SoftConsole 2022.2

→ #=====

#
# SoftConsole
#
export PATH=$PATH:$SC_INSTALL_DIR/riscv-unknown-elf-gcc/bin
export FPGENPROG=$LIBERO_INSTALL_DIR/Libero/bin64/fpgenprog

#
# Libero
#
export PATH=$PATH:$LIBERO_INSTALL_DIR/Libero/bin:$LIBERO_INSTALL_DIR/Libero/
→ bin64
export PATH=$PATH:$LIBERO_INSTALL_DIR/SynplifyPro/bin
export PATH=$PATH:$LIBERO_INSTALL_DIR/ModelSimPro/modeltech/linuxacoem
export PATH=$PATH:$SMARTHLS_INSTALL_DIR/bin
export PATH=$PATH:$SMARTHLS_INSTALL_DIR/swtools/binutils/riscv-gnu-toolchain/
→ bin

export LOCALE=C
export LD_LIBRARY_PATH=/usr/lib/i386-linux-gnu:$LD_LIBRARY_PATH

#
# Libero License daemon
#
export LM_LICENSE_FILE=1702@localhost
export SNPSLMD_LICENSE_FILE=1702@localhost

$LICENSE_DAEMON_DIR/lmgrd -c $LICENSE_FILE_DIR/License.dat -l $LICENSE_FILE_
→ DIR/license.log

```

setup-microchip-2024-2-and-older-tools.sh

Details:

You can create a folder named FPGA-Tools-Setup and store the file there, although this is not required, as long as it is inside the Microchip folder.

You shouldn't need to edit the script, as long as you have installed Libero inside a folder that follows the Libero_SoC_vXXXX.X format, or if you have multiple Libero versions installed and want to select a preferred one to use.

Source the script:

```

sudo chmod +x setup-microchip-tools.sh

. ./setup-microchip-tools.sh

```

Important: Do not forget the leading dot. It matters. You will need to run this every time you restart your machine.

Optionally, add this to the end of your ~/.bashrc file to avoid running it each time on startup.

First, open ~/.bashrc:

```
nano ~/.bashrc
```

Then, add the following lines at the end:

```
cd /home/$USER/Microchip/  
./setup-microchip-tools.sh
```

You can then start Libero to open an existing Libero project.

```
libero
```

However you will more than likely want to use Libero to run a TCL script that will build a design for you.

```
libero SCRIPT:BUILD_A_DESIGN.tcl
```

5.5 Gateway Design Introduction

The PolarFire SoC device used on BeagleV-Fire is an SoC FPGA which includes a RISC-V processors subsystem and a PolarFire FPGA on the same die. The gateway configures the Microprocessor subsystem's hardware and programs the FPGA with digital logic allowing customization of the use of BeagleV-Fire connectors.

5.5.1 Gateway Architecture

The diagram below is a simplified overview of the gateway's structure.

The overall gateway is made-up of several blocks, some of them interchangeable. These blocks are all clocked and reset by another "Clock and Resets" block not showed in the diagram for clarity. A 125MHz, and a 160MHz clock are provided for use by the gateway blocks.

Each gateway block is associated with one of BeagleV-Fire's connectors.

All gateway blocks have an AMBA APB target interface for software to access control and status registers defined by the gateway to operate digital logic defined by the gateway block. This is the software's control path into the gateway block.

Some gateway blocks also have an AMBA AXI target and/or source interfaces. The AXI interfaces are typically used to move high volume of data at high throughput in and out of DDR memory. For example, the M.2 gateway uses these interfaces to transfer data in and out of its PCIe root port.

Cape Gateway

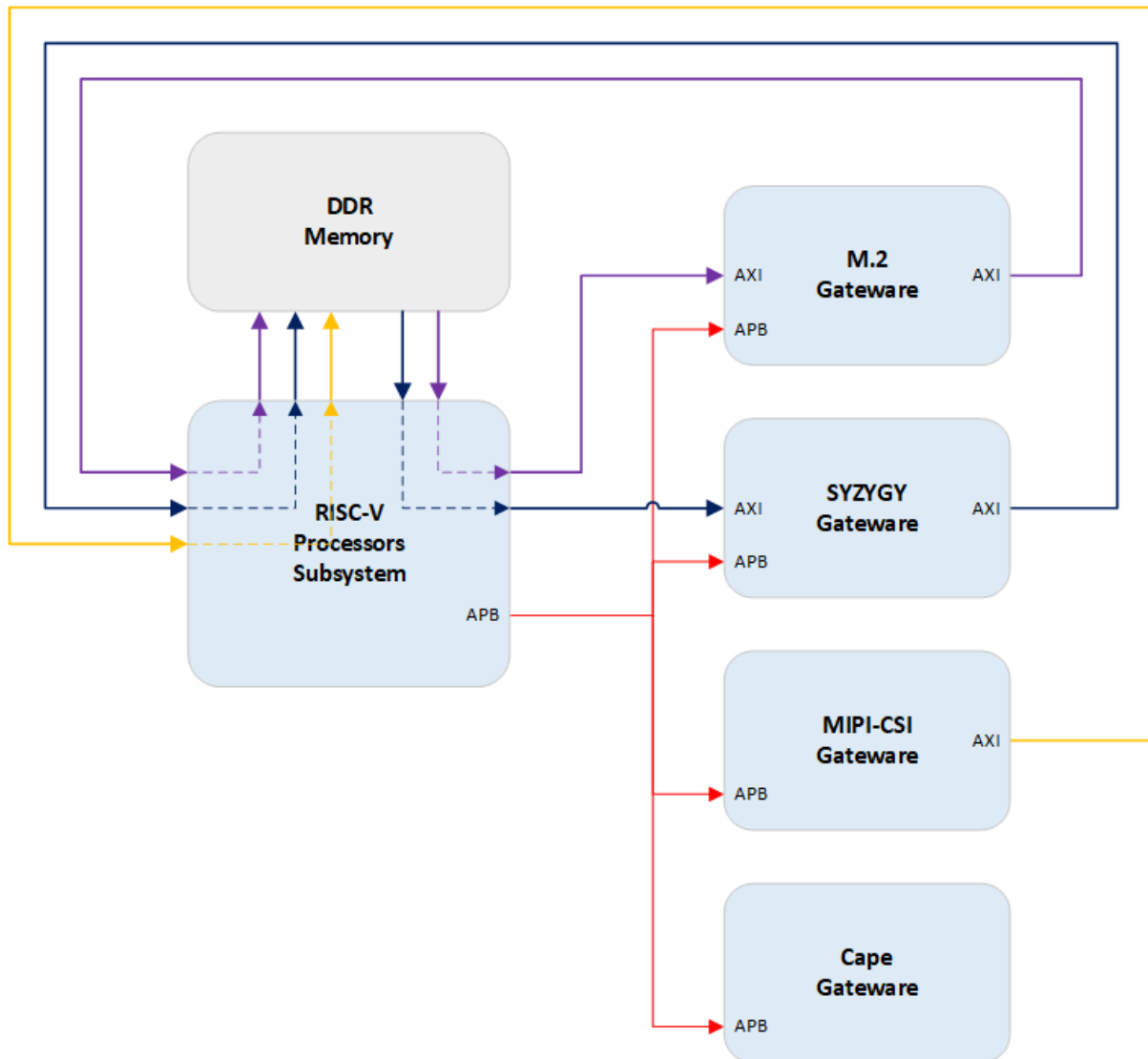
The cape gateway handles the P8 and P9 connectors signals. This is where support for specific capes is implemented.

This is a very good place to start learning about FPGA and how to customize gateway.

SYZGY Gateway

The SYZGY gateway handles the high-speed connector signals. This connector includes:

- up to three transceivers capable of 12.7Gbps communications
- One SGMII interface
- 10 high-speed I/Os
- Clock inputs



There is a lot of fun that can be had with this interface given its high-speed capabilities.

Please note that only two transceivers can be used when the M.2 interface is enabled.

MIPI-CSI Gateway

The MIPI gateway handles the signals coming from the camera interface.

Gateway for the MIPI-CSI interface is Work-In-Progress.

M.2 Gateway

The M.2 gateway implements the PCIe interface used for Wi-Fi modules. It connects the processor subsystem to the PCIe controller associated with the transceivers bank.

There is limited fun to have here. You either include this block or not in your bitstream.

The M.2 gateway uses one of the four available 12.7 Gbps transceivers. Only two out of the three SYZYGY transceivers can be used when the M.2 is included in the bitstream. This gateway needs to be omitted from the bitstream if you want to use all three 12.7Gbps transceivers on the SYZYGY high-speed connector.

RISC-V Processors subsystem

The RISC-V Processors Subsystem also includes some gateway mostly dealing with exposing AMBA bus interfaces for the other gateway blocks to attach to. It also handles immutable aspects of the gateway related to how some PolarFire-SoC signals are used to connect BeagleV-Fire peripherals such as the ADC and EEPROM. As such the RISC-V Processors Subsystem gateway is not intended to be customized.

5.6 How to retrieve BeagleV-Fire's gateway version

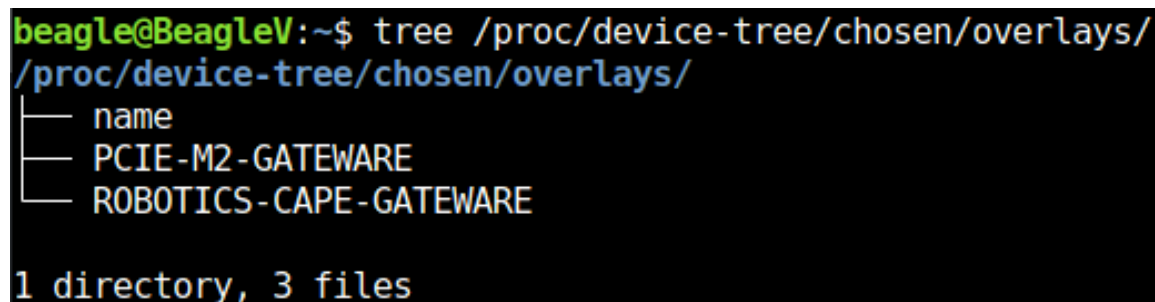
There are two methods to find out what gateway is programmed on a board.

5.6.1 Device Tree

The device tree overlays contains the list of gateway blocks included in the overall gateway design. You can retrieve that information using the following command:

```
tree /proc/device-tree/chosen/overlays/
```

This should give an output similar to the one below.



```
beagle@BeagleV:~$ tree /proc/device-tree/chosen/overlays/
/proc/device-tree/chosen/overlays/
├── name
├── PCIE-M2-GATEWARE
└── ROBOTICS-CAPE-GATEWARE

1 directory, 3 files
```

The gateway version can be retrieved by reading one of the overlay files. For example, the command:

```
cat /proc/device-tree/chosen/overlays/ROBOTICS-CAPE-GATEWARE
```

```
beagle@BeagleV:~$ cat /proc/device-tree/chosen/overlays/ROBOTICS-CAPE-GATEWARE
BVF-0.3.0-5-g3e0d3380beagle@BeagleV:~$
```

should result in:

where the result of a “git describe” command on the gateway repository is displayed. This provides the most recent tag on the gateway repository followed by information about additional commits if some exist. In the example above, the gateway was created from a gateway repository hash 3e0d338 which is 5 commits more recent than tag BVF-0.3.0.

5.6.2 Bootloader messages

The Hart Software Services display the gateway design name and design version retrieved from the FPGA at system start-up.

```
[5.528316] Design Info:
        Design Name: DEV_ROBOTICS_3E0D338F3C2574145
        Design Version: 02.00.1
```

The design name is the name of the build option selected when using the bitstream-builder to generate the bitstream. The number at the end of the design name is the hash of the gateway repository used to build the bitstream.

The design version is specified as part of the bitstream-builder build configuration option.

Please note that design name “BVF_GATEWARE” indicates that the bitstream used to program the board was generated directly from the gateway repositories scripts and not the bitstream-builder. You might see this when customizing the gateway. Seeing “BVF_GATEWARE” as the design name should be a warning sign that there is a disconnect between the hardware and software on your board.

5.7 Building Linux for BeagleV-Fire using Buildroot

5.7.1 Introduction

Buildroot is a simple, efficient, and user-friendly tool for creating custom embedded Linux systems through cross-compilation.

This document provides a guide for building and flashing a Linux image on the BeagleV-Fire board with Buildroot. It outlines the process for compiling the image, writing image to the eMMC, and booting the new operating system.

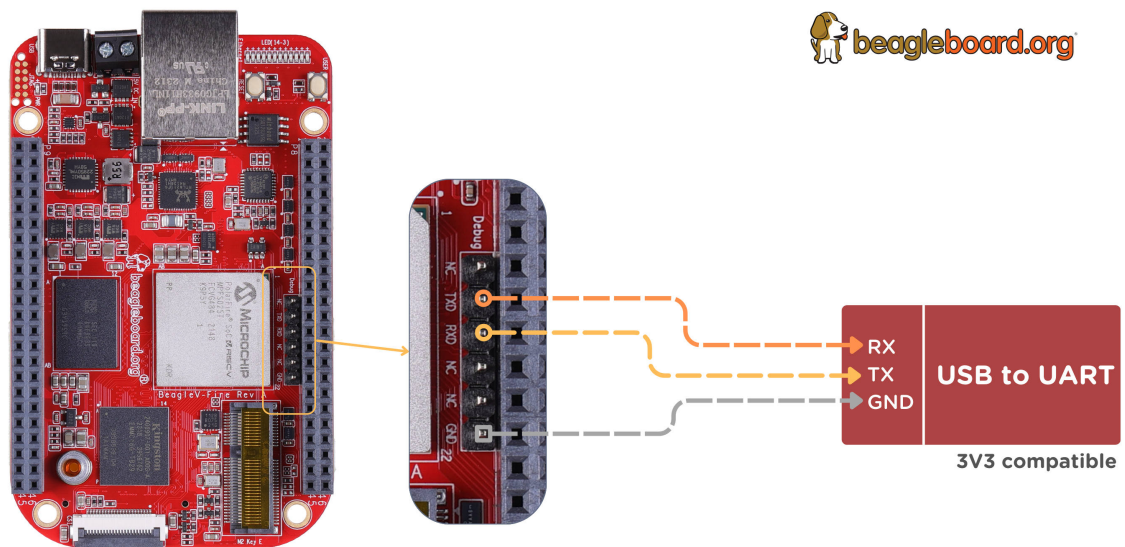
Hardware requirements:

1. BeagleV-Fire board
2. USB-C cable
3. 3.3v USB to UART bridge

Connect BeagleV-Fire UART debug port using 3.3v USB to UART bridge.

Software requirements:

Download [Buildroot](#) repository from GitHub.



5.7.2 Start Building

Note: The following steps are intended for a Linux operating system.

To build and flash Linux image using Buildroot,

Step 1. Navigate to the Buildroot directory

```
cd buildroot
```

Step 2. Configure the Build for BeagleV-Fire

Configure the build by selecting the default board configuration:

```
make beaglev_fire_defconfig
```

Step 3. Customise the build (Optional)

If you need to customize the build, use the following command:

```
make menuconfig
```

Step 4: Start the build process

```
make
```

Note: The build process can take 20-30 minutes for a clean build.

Step 5. Locate the build image

Once the build is complete, the Linux image will be saved as *sdcard.img* in the */output/images/* directory. The directory structure will look something like this:

```
$ ls output/images/
boot.scr          boot.vfat.bmap    dts/              Image.gz          mpfs_
→ icicle/         mpfs_icicle.its  rootfs.cpio       rootfs.tar        sdcard.img
→ sdcard.img.gz
boot.vfat         boot.vfat.gz      Image             microchip/        mpfs_
→ icicle.itb     payload.bin      rootfs.cpio.gz    sdcard.bmap       sdcard.img.
→ bmap          u-boot.bin
```

Step 6. Flash the Image to BeagleV-Fire's eMMC

- Restart the board and halt the HSS (Hart Software Services) by pressing any key
- In the HSS command line interface, type *usbdmcs* to expose the eMMC as a USB mass storage device using the USB-C connector.
- If successful, a message saying "USB Host connected" will be displayed
- Now, copy the image from local machine to BeagleV-Fire's eMMC

```
sudo dd if=output/images/sdcard.img of=/dev/sdX bs=1M
```

Note: You need to replace */dev/sdX* with the actual device name of your eMMC. Be very careful not to overwrite the wrong drive, as this action is irreversible.

- Once the transfer is complete, type *CTRL+C* to disconnect your device
- Finally boot the new Linux image by typing *boot* or reset your board

Detailed description of this step is mentioned in [Flashing eMMC](#) section.

5.8 Gateware Full Build Flow

5.8.1 Introduction

BeagleV-Fire gateware is made up of several components:

- Digital design for the FPGA fabric.
- Microprocessor Subsystem (MSS) configuration containing MSS configuration register values.
- A zero stage bootloader (HSS).
- A set of device tree overlays describing the content of the FPGA fabric.

The FPGA's digital design is a combination of:

- HDL/Verilog source code
- TCL scripts configuring IP blocks
- TCL scripts stitching IP blocks together
- Microprocessor Subsystem (MSS) configuration describing the MSS port list
- Pin, placement and timing constraints

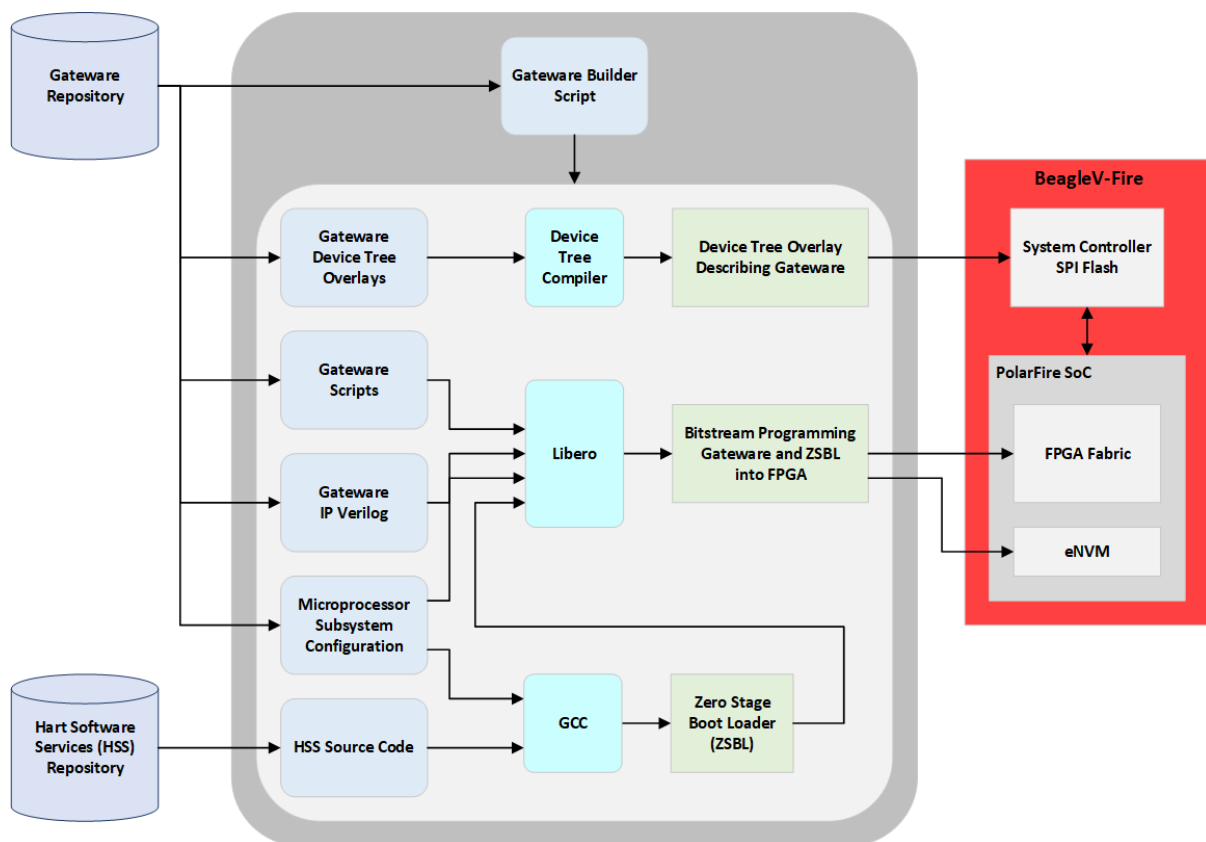
The Hart Software Service (HSS) zero stage bootloader

- Configures the PolarFire SoC chip.
- Retrieves the next boot stage from eMMC and hand-over to the next boot stage (e.g. u-boot)
- Makes the board appear as a USB mass-storage for populating the eMMC with secondary boot-loader and operating system image.

The chip configuration applied by the HSS includes the configuration of:

- Clocks
- Memory controllers
- IOs
- Transceivers

Of course all these components need to be in synch with each other for the system to work properly. This is the reason for using a gateway build system rather than building and tracking each component individually.



5.8.2 Programming artifacts

The gateway builder for BeagleV-Fire produces two programming artifacts:

- A bitstream containing the FPGA fabric and eNVM programming
- A device tree overlay describing the FPGA content.

These two artifacts are packaged differently depending on the programming method. They are merged into a single programming file for DirectC (.dat) and FlashPro Express (.job). They are kept separate for Linux programming (mpfs_bitstream.spi and mpfs_dtbo.spi).

5.8.3 Programming BeagleV-Fire with new gateway

There are several methods possible for programming the BeagleV-Fire with new gateway:

- Linux script executed on the BeagleV-Fire board.
- Running DirectC on another single board computer
- Using Microchip's FlashPro Express

Linux script

This is the recommended approach. It does not require any additional hardware. Simply run the script located in `/usr/share/beagleboard/gateway`. You should use this method unless you have soft-bricked your BeagleV-Fire.

DirectC

This approach uses a single board computer (SBC) connected to the BeagleV-Fire JTAG port. The SBC bit-bangs the FPGA programming protocol over GPIOs. This approach is only required for recovering a soft-bricked BeagleV-Fire.

FlashPro Express

This approach uses Microchip's FlashPro Express programming software and a FlashPro6 JTAG programmer. I would recommend using the Linux script even if you are an existing Microchip FPGA user with all the tools. This approach makes most sense when doing bare metal software development and already have a FlashPro programmer and don't care about device tree overlays.

5.9 Gateway TCL Scripts Structure

This document describes the structure of the gateway TCL scripts. It is of interest to understand how to extend or customize the gateway.

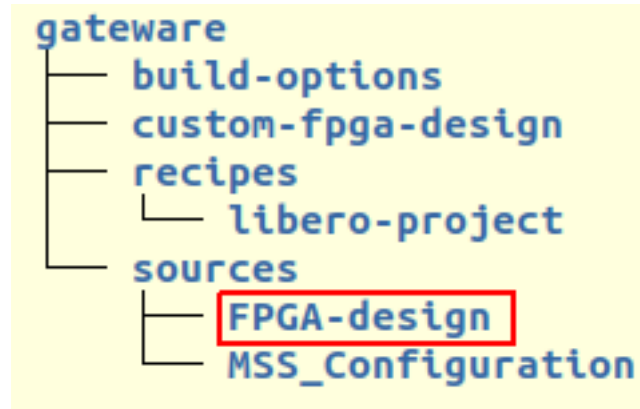
The [Libero SoC TCL Command Reference Guide](#) describes the TCL command used in the gateway scripts.

5.9.1 Gateway Project

The gateway project is made up of:

- TCL scripts
- HDL/Verilog source code
- IO pin constraints
- Placement constraints
- Device tree overlays

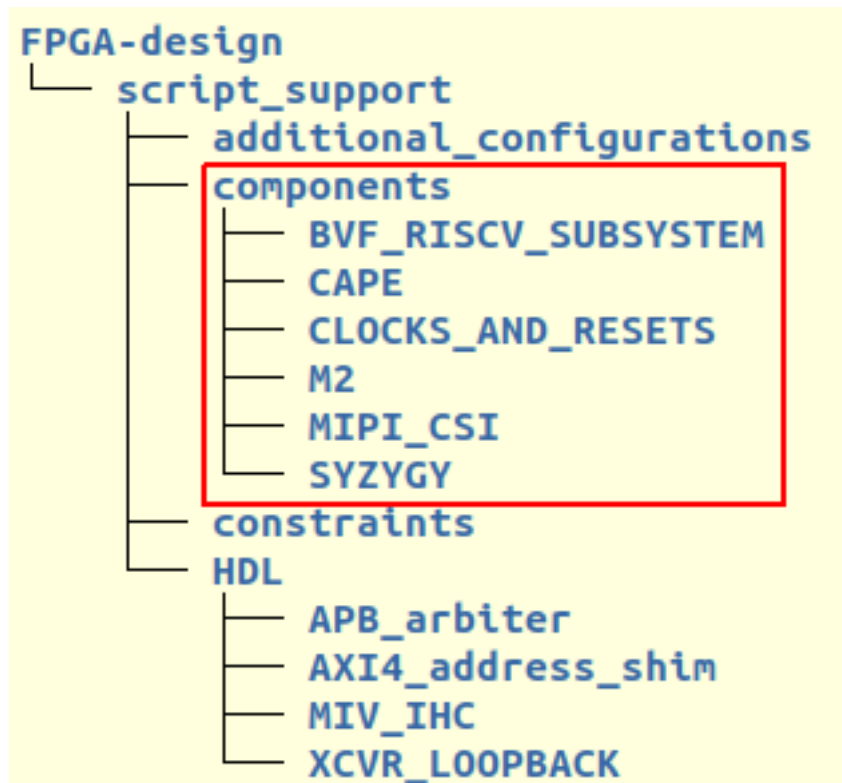
All these files are found in the `FPGA-design` directory.



5.9.2 Gateway Components

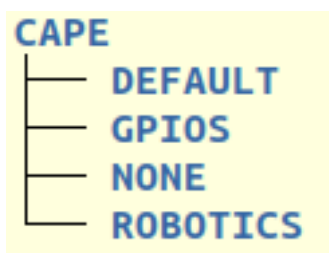
The gateway is organized into 6 components:

- Clocks and reset control
- A base RISC-V microprocessor subsystem
- Cape interface
- M.2 interface
- MIPI camera interface
- SYZYGY high speed interface



5.9.3 Gateway Build Options

Each interface component may have a number of build options. For example, which cape will be supported by the generated gateway.



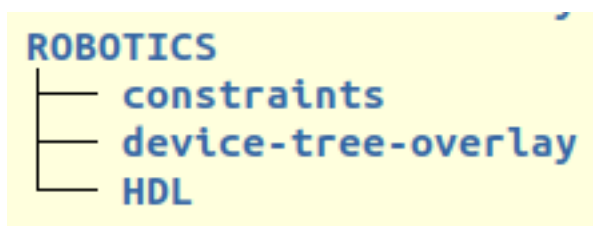
The name of the directories within the component's directory are the option names passed to the top Libero BUILD_BVF_GATEWATE.tcl script. These directory names are the option name specified in the bitstream builder's build option YAML files.

The gateway is extended or customized by creating additional directories within the component directory of interest. For example, add a MY_CUSTOM_CAPE directory under the CAPE directory to add a gateway build option to support a custom cape.

5.9.4 Gateway Component Directories

The component directory contains subdirectories for:

- Constraint files
- Device tree overlay
- Optional HDL/Verilog source code



Gateway TCL Scripts

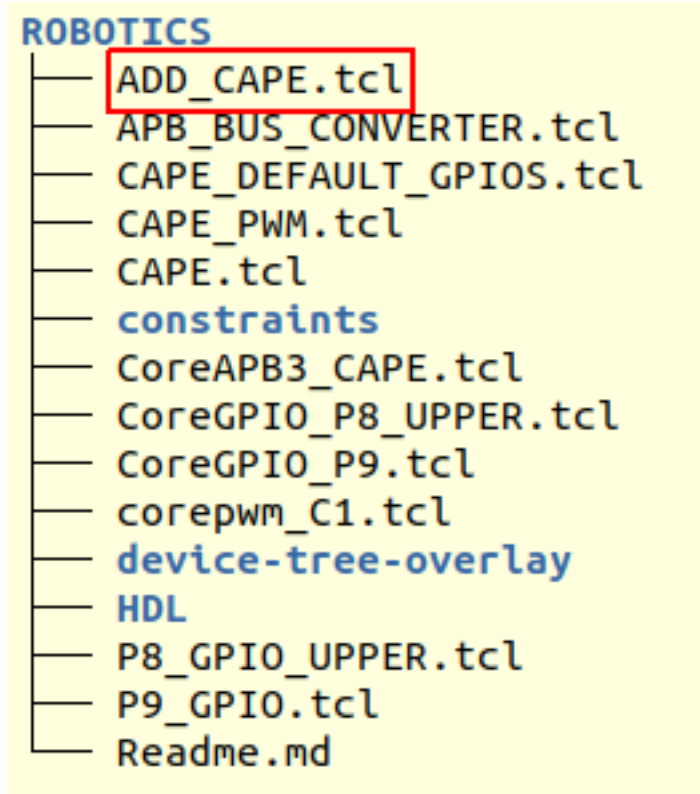
The component directory contains the TCL scripts executed by Libero to generate the gateway. The TCL script framework executes a hand-crafted ADD_<COMPONENT_NAME>.tcl script which instantiates the component and stitches it to the base RISC-V subsystem and top level IOs. The other TCL scripts are typically IP configuration scripts and SmartDesign stitching scripts.

5.9.5 Opening the gateway as a libero project

It can be slightly difficult to explore the gateway design through the TCL files. To inspect the gateway design in detail easily, you can open the gateway as a Libero project. This is done by running the following command in the gateway directory:

```
python build-bitstream.py ./build-options/default.yaml # build option_
↪ depending on the gateway
```

You will need to have all microchip tools installed and the environment variables set up correctly. Refer to the [Microchip tools installation guide](#) for information on how to install these tools.



```
ROBOTICS
├── ADD_CAPE.tcl
├── APB_BUS_CONVERTER.tcl
├── CAPE_DEFAULT_GPIOS.tcl
├── CAPE_PWM.tcl
├── CAPE.tcl
├── constraints
├── CoreAPB3_CAPE.tcl
├── CoreGPIO_P8_UPPER.tcl
├── CoreGPIO_P9.tcl
├── corepwm_C1.tcl
├── device-tree-overlay
├── HDL
├── P8_GPIO_UPPER.tcl
├── P9_GPIO.tcl
└── Readme.md
```

5.10 Customize BeagleV-Fire Cape Gateway Using Verilog

This document describes how to customize gateway attached to BeagleV-Fire's cape interface using Verilog as primary language. The methodology described can also be applied when using other HDL languages.

It will describe:

- How to generate programming bitstreams without requiring the installation of the Libero FPGA toolchain on your development machine.
- How to use the cape Verilog template
- How to use the git.beagleboard.org CI infrastructure to generate programming bitstreams for your custom gateway

Steps:

1. Fork BeagleV-Fire gateway repository on git.beagleboard.org
2. Create a custom gateway build option
3. Rename a copy of the cape gateway Verilog template
4. Customize the cape's Verilog source code
5. Commit and push changes to your forked repository
6. Retrieve the forked repositories artifacts
7. Program BeagleV-Fire with your custom bitstream

5.10.1 Fork BeagleV-Fire Gateway Repository

Important: All new users need to be manually approved to protect from BOT spam. You will not be able to fork the Gateway Repository until you have been approved. A request to [the forum](#) may expedite the process.

Navigate to BeagleV-Fire's [gateway source code repository](#).

Click on the Forks button on the top-right corner.



Fig. 5.11: BeagleV-Fire gateway repo fork button

On the Fork Project page, select your namespace and adjust the project name to help you manage multiple custom gateway (e.g. my-lovely-gateway). Click the Fork project button.

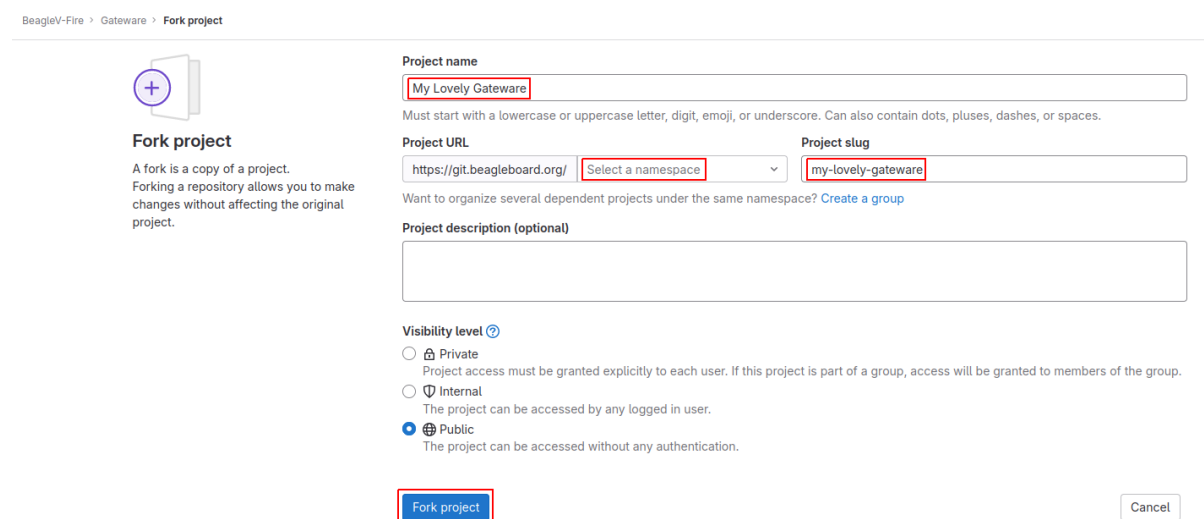


Fig. 5.12: Create gateway fork

Clone the forked repository

```
git clone git@git.beagleboard.org:<MY-NAMESPACE>/my-lovely-gateway.git
```

Where <MY-NAMESPACE> is your Gitlab username or namespace.

5.10.2 Create A Custom Gateway Build Option

BeagleV-Fire's gateway build system uses "build configuration" YAML files to describe the combination of gateway components options that will be used to build the gateway programming bitstream. You need to create one such file to describe to the gateway build system that you want your own custom gateway to be built. You need to have one such file describing your gateway in directory `custom-fpga-design`.

Let's modify the `./custom-fpga-design/my_custom_fpga_design.yaml` build configuration file to specify that your custom cape gateway should be included in the gateway bitstream. In this instance will call our custom cape gateway `MY_LOVELY_CAPE`.

```
HSS:
  type: git
  link: https://github.com/polarfire-soc/hart-software-services.git
  branch: master
  make_clean: 1
gateway:
  type: sources
  build-args: "M2_OPTION:NONE CAPE_OPTION:MY_LOVELY_CAPE" # ?
```

① On the gateway build-args line, replace VERILOG_TUTORIAL with MY_LOVELY_CAPE.

Note: The **custom-fpga-design** directory has a special meaning for the Beagleboard Gitlab CI system. Any build configuration found in this directory will be built by the CI system. This allows generating FPGA programming bitstreams without the requirement for having the Microchip FPGA toolchain installed on your computer.

5.10.3 Rename A Copy Of The Cape Gateway Verilog Template

Move to the cape gateway source code

```
cd my-lovely-gateway/sources/FPGA-design/script_support/components/CAPE
```

Create a directory that will contain your custom cape gateway source code

```
mkdir MY_LOVELY_CAPE
```

Copy the cape Verilog template

```
cp -r VERILOG_TEMPLATE/* ./MY_LOVELY_CAPE/
```

5.10.4 Customize The Cape's Verilog Source Code

You will only need to change the content of `ADD_CAPE.tcl` if you want to modify how the cape interfaces with the rest of the gateway (RISC-V processor subsystem, clock and reset blocks).

Tip:

Any Verilog file (*.v) you put in the HDL subdirectory will automatically be pulled into the project by the build system.

Manual intervention is no longer required.

Customize The Cape's Verilog source code

We will add a simple Verilog source file, `blinky.v`, in the `MY_LOVELY_CAPE/HDL` directory. Code below:

```

`timescale 1ns/100ps

module blinky (
    input  clk,
    input  resetn,
    output blink
);

reg [22:0] counter;

assign blink = counter[22];

always @(posedge clk, negedge resetn)
begin
    if(~resetn)
    begin
        counter <= 23'b0;
    end
    else
    begin
        counter <= counter + 23'b1;
    end
end

endmodule

```

Let's connect the blinky Verilog module within the cape by editing the `CAPE.v` file (we're still in the HDL subdirectory).

Add the instantiation of the blinky module:

```

//-----P9_41_42_IOPADS
P9_41_42_IOPADS P9_41_42_IOPADS_0 (
    // Inputs
    .GPIO_OE  ( GPIO_OE_const_net_3 ),
    .GPIO_OUT ( GPIO_OUT_const_net_3 ),
    // Outputs
    .GPIO_IN  (   ),
    // Inouts
    .P9_41    ( P9_41 ),
    .P9_42    ( P9_42 )
);

//-----blinky
blinky blinky_0(
    .clk      ( PCLK ),           // ②
    .resetn   ( PRESETN ),        // ③
    .blink    ( BLINK )           // ④
);

endmodule

```

- ① Create a blinky module instance called `blinky_0`.
- ② Connect the clock using the existing `PCLK` wire.
- ③ Connect the reset using the existing `PRESETS` wire.
- ④ Connect the blinky's `blink` output using the `BLINK` wire. This `BLINK` wire needs to be declared.

Add the `BLINK` wire:

```

wire PCLK;
wire PRESETN;

```

(continues on next page)

(continued from previous page)

```
wire          BLINK; // ?
wire [31:0]    APB_SLAVE_PRDATA_net_0;
wire [27:0]    GPIO_IN_net_1;
```

① Create a wire called BLINK.

The BLINK wire will be used to connect the blinky module's output to a top level output connected to a LED. Do you see where this is going?

Now for the complicated part. We are going to change the wiring of the bi-directional buffers controlling the cape I/Os including the user LEDs.

The original code populates two 43 bit-wide vectors, for controlling the output-enable and output values of the P8 cape connector I/Os.

The bottom 28 bits being controlled by the microprocessor subsystem's GPIO block; those are the ones we want.

```
//-----
// Concatenation assignments
//-----
assign GPIO_OE_net_0 = { 16'h0000 , GPIO_OE };
assign GPIO_OUT_net_0 = { 16'h0000 , GPIO_OUT };
```

We are going to hijack the 6th I/O with our blinky's output:

```
//-----
// Concatenation assignments
//-----
assign GPIO_OE_net_0 = { 16'h0000, GPIO_OE[27:6], 1'b1, GPIO_OE[4:0] };
→ // ?
assign GPIO_OUT_net_0 = { 16'h0000, GPIO_OUT[27:6], BLINK, GPIO_OUT[4:0] };
→ // ?
```

① Tie high the output-enable of the 6th bit to constantly enable that output.

② Control the 6th output from the blink module through the BLINK wire.

Edit The Cape's Device Tree Overlay

You should always have a device tree overlay associated with your gateway even if there is limited control from Linux.

The device tree overlay is very useful to identify which gateway is currently programmed on your BeagleV-Fire.

```
/dts-v1/;
/plugin/;

&{/chosen} {
    overlays {
        MY-LOVELY-CAPE-GATEWARE = "GATEWARE_GIT_VERSION"; // ?
    };
};
```

① Replace VERILOG-CAPE-GATEWARE with MY-LOVELY-CAPE-GATEWARE.

This change will result in MY-LOVELY-CAPE-GATEWARE being visible in /proc/device-tree/chosen/overlays at run-time, allowing to check that my lovely gateway is successfully programmed on BeagleV-Fire.

5.10.5 Commit And Push Changes To Your Forked Repository

Move back up to the root directory of your gateway project. This is the my-lovely-gateway directory in our current example.

Add the my-lovely-gateway/sources/FPGA-design/script_support/components/CAPE/MY_LOVELY_CAPE directory content to your git repository.

```
git add sources/FPGA-design/script_support/components/CAPE/MY_LOVELY_CAPE/
```

Commit changes to my-lovely-gateway/custom-fpga-design/my_custom_fpga_design.yaml

```
git commit -m "Add my lovely gateway."
```

Push changes to your beagleboard Gitlab repository:

```
git push
```

5.10.6 Retrieve The Forked Repositories Artifacts

Navigate to your forked repository. Click Pipelines in the left pane then the Download Artifacts button on the right handside.

Select build-job:archive. This will result in an artifacts.zip file being downloaded.

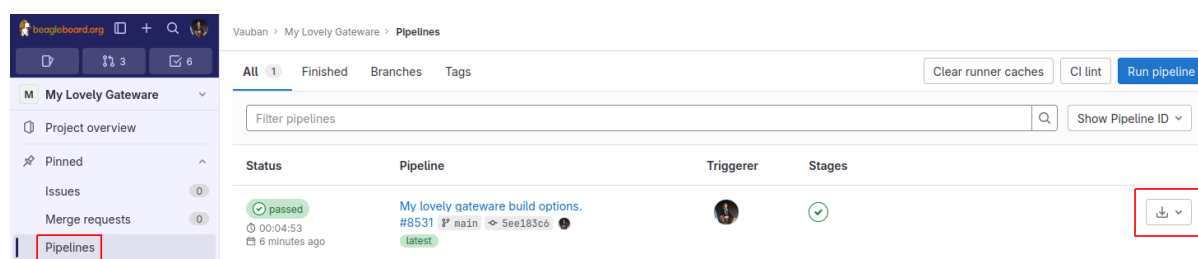


Fig. 5.13: gateway pipeline

5.10.7 Program BeagleV-Fire With Your Custom Bitstream

Unzip the downloaded artifacts.zip file. Go to the gateway-builds-tester/artifacts/bitstreams directory:

```
cd gateway-builds-tester/artifacts/bitstreams
```

On your Linux host development computer, use the scp command to copy the bitstream to BeagleV-Fire home directory,

replacing <IP_ADDRESS> with the IP address of your BeagleV-Fire.

```
scp -r ./my_custom_fpga_design beagle@<IP_ADDRESS>:/home/beagle/
```

On BeagleV-Fire, initiate the reprogramming of the FPGA with your gateway bitstream:

```
sudo /usr/share/beagleboard/gateway/change-gateway.sh ./my_custom_fpga_  
→design
```

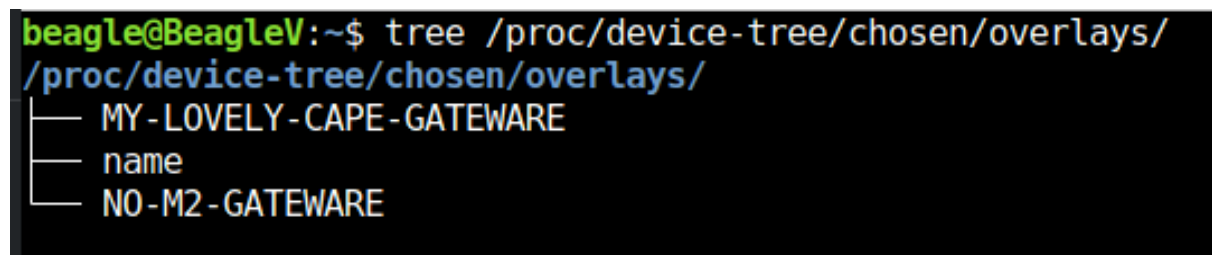
Wait for a couple of minutes for the BeagleV-Fire to reprogram itself.

You will see the 6th user LED flash once the board is reprogrammed.

That's the Verilog you added blinking the LED.

On BeagleV-Fire, You can check that your gateway was loaded using the following command to see the device tree overlays:

```
tree /proc/device-tree/chosen/overlays/
```



```
beagle@BeagleV:~$ tree /proc/device-tree/chosen/overlays/  
/proc/device-tree/chosen/overlays/  
├── MY-LOVELY-CAPE-GATEWARE  
│   └── name  
└── NO-M2-GATEWARE
```

Fig. 5.14: gateway lovely overlay

5.11 How to use PicoRV Softcore on BeagleV-Fire

5.11.1 Introduction

The PicoRV Softcore is a 32-bit RISC-V CPU subsystem on the BeagleV-Fire's FPGA fabric, functionally equivalent to the PRU subsystem on the BeagleBone Black. The core is designed for low-latency I/O operations and offloading smaller tasks like PWM, LEDs, etc .

5.11.2 Prerequisites

To be able to use the softcore component, you'll need to have following packages installed on your host computer:

- gcc-riscv64-unknown-elf
- picolibc-riscv64-unknown-elf

Note: These packages are used for compiling the code for the softcore.

For debian-based systems like ubuntu, you can use following commands:

```
sudo apt install gcc-riscv64-unknown-elf picolibc-riscv64-unknown-elf
```

If you are using any other linux distro these packages can be downloaded from respective package managers. But, it is highly recommended to use docker for accurate results.

Docker installation steps.

Once you have docker installed:

1. Start a container with ubuntu image.
2. Install the required toolchain using above mentioned command (remember not to use 'sudo' in docker).

```
Sudo docker run -it ubuntu:latest
```

After installing the necessary tools, you are all set for using the Softcore.

5.11.3 How to use the Softcore

1. Boot the softcore with firmware

Step 1 Navigate to `sources/FPGA-design/script_support/components/SOFTCORE/PICO_RISCV/firmware` in the BeagleV-Fire repository.

Step 2 Edit the `firmware.c` file by adding your code in the main function. You can create your own custom file but remember to use `firmware.h` file for pre-defined functions to access gpios.

Step 3 Now run the `generatehex.sh` to compile the code. (remember to edit the file name in the script if you are using custom file)

```
. ./generatehex.sh
```

If you are using docker clone the repository on your docker container or use docker copy, then compile the program and use docker copy to obtain the hex file.

To copy file from local system to docker container:

```
docker cp ./path/to/file CONTAINER:/path/to/directory
```

To copy file from docker container to local system:

```
docker cp CONTAINER:/path/to/file /path/to/directory
```

To check the name of docker 'CONTAINER':

```
docker ps -a
```

This will list all the running and stopped containers To restart a stopped container, use:

```
sudo docker start name_of_container -i
```

Step 4 Select PicoRV build option for bitstream generation

Once the code is compiled, build the bitstream using the 'build-bitstream.py' script.

```
python build-bitstream.py ./build-options/picorv-softcore.yaml
```

Note: The Gitlab CI does not support the softcore code compilation yet, so the bitstream must be generated locally.

Step 5 Program BeagleV-Fire With Your [Custom Bitstream](#).

2. Programming the softcore in run-time

Step 1

- Program the softcore with custom bitstream with picorv-softcore build-option.
- Follow the steps mentioned in previous section.

Step 2

- Edit and compile your new code in your local machine.(BeagleV-Fire does not support on-board compilation for softcore programs yet, this feature will be added soon.)
- Follow Steps 2 and 3 in previous section for compiling your new program.

Step 3 Copy the firmware directory from BeagleV-Fire repository to your board.

```
sudo scp -r path/to/firmware beagle@192.168.7.2:/home/beagle
```

Step 4

- Overwriting softcore's program memory.
- Compile the 'AXI_test.c' file using gcc and execute it.

```
gcc -o AXI_test AXI_test.c
sudo ./AXI_teste
```

- This will open a interactive menu with various option that you can try out and access the program memory of the softcore.
- Here we will select *option 2* to upload the hex file we just compiled to the softcore.

Note: This compilation in step 2 is to generate a hex file to program the Softcore, while the compilation in Step 3 is performed to command the Linux system on-board to access the program memory of the Softcore.

Note: The filename in **AXI_test.c** for the hex file is set as `firmware.hex`, so make sure the name of the hex file you compiled is the same else change the filename pointer in **AXI_test.c**.

Repository

PicoRV softcore builder.

5.12 How to build the BeagleV-Fire Gateway on Windows

5.12.1 Introduction

The BeagleV-Fire gateway builder is a Python script that builds both the PolarFire SoC HSS bootloader and Libero FPGA project into a single programming bitstream. It uses a list of repositories/branches specifying the configuration of the BeagleV-Fire to build.

5.12.2 Prerequisites

Tools

To be able to use the bitstream builder on Windows, you will need to install the following tools:

- Msys2-Mingw
- Make
- wsl

Please follow the installation instructions for Msys2 available at <https://www.msys2.org/wiki/MSYS2-installation/>

When installing *make* in your msys2 terminal you're recommended to use the default command

```
pacman -S make
```

For those requiring a specific version of *make*, refer to the porting guide at <https://www.msys2.org/wiki/Porting/>

```
pacman -S <target>-make
```

Ensure that the *Msys2* bin path (e.g., *C:\msys64\usrbin*) is added to your system's environment variable PATH.

To enable and install WSL, follow these steps:

- Search for "Turn Windows features on or off" in the Windows start menu.
- Select "Windows Subsystem for Linux" and click OK.
- Open a command prompt as an administrator and execute:

```
wsl.exe --install
```

After installing the necessary tools, proceed to the repository and follow the instructions in the README to build the bitstream on Windows

Repository

Access the BeagleV-Fire gateway builder repository at <https://openbeagle.org/cyril-jean/gateway-maintenance/>

Note: If you encounter an end-of-line error (CRLF/LF) during the build process, change the local Git configuration *core.autocrlf* to false and clone the repository again

```
git config --global core.autocrlf false
```

Note:

- Should the build fail due to an unrecognized Python package, despite the package being installed, it may be due to multiple Python/pip versions. Reinstall the package using.

```
python -m pip install <package-name>
```

- Verify that the LM_LICENSE_FILE environment variable includes licenses for all required programs to avoid silent errors during the build process
-

5.13 Exploring Gateway Design with Libero

In this demonstration, we'll be exploring the BeagleV-Fire gateway in the [Libero Design Suite](#), making changes to the default gateway. This demo will serve as an introduction to the design tool, an alternative method for developing gateway.

5.13.1 Prerequisites

The prerequisites required for creating the Libero project locally are:

1. Microchip design tools: Refer to the document here for [installation instructions](#) of microchip FPGA tools.
2. Python requirements for gateway build scripts:

```
pip3 install gitpython  
pip3 install pyyaml
```

3. Build requirements:

```
sudo apt install device-tree-compiler
```

Tip: For convenience, you can install a python command alias like so:

```
sudo apt install python-is-python3
```

This is optional, but remember to use `python3` in later command examples if you don't.

4. **Environment variables:** The following environment variables are required for compilation:

- SC_INSTALL_DIR
- FPGENPROG
- LIBERO_INSTALL_DIR
- LM_LICENSE_FILE

A script is provided for setting up these variables in the [fpga tools installation](#) section.
An example script for setting up the environment is available [here](#).

5. It is highly recommended to go through the [Customize BeagleV-Fire Cape Gateway Using Verilog](#) tutorial to understand the basics of the gateway structure.

5.13.2 Cloning and Building the Gateway

First, we must source the environment to include the microchip tools.

```
source /path/to/microchip/fpga/tools/setup-microchip-tools.sh
```

Next, we'll clone the gateway repository to get a local copy of the project.

```
git clone https://openbeagle.org/beagle-v-fire/gateway.git
cd gateway
```

We can then use the `build-bitstream.py` script to generate a Libero project for us, where we can start making our changes.

Important:

Make sure to source the microchip setup script before starting the next command. This is required every time you open a new terminal.

```
python build-bitstream.py ./build-options/default.yaml # exploring the
↳ default gateway
```

This should start a big log stating the compilation of the project. First, the device tree overlays are compiled, which contain information for linux about the gateway.

Next, the Hart Software Services (HSS) is compiled. This acts as a zero-stage bootloader, configuring the Polarfire SoC and allowing services like loading the next stage bootloader and flashing the eMMC of the board.

Then the libero project generating is started. Here, TCL scripts inside the `sources` directory are executed, starting with the `BUILD_BVF_GATEWARE.tcl` script. This stitches each HDL module, IP, hardware configuration together in the gateway.

Once bitstream generation is completed, the Libero project is ready to be opened. Start Libero on the same terminal in linux, like so:

```
libero &
```

or from the start menu in Windows, and open the project file by pressing CTRL+O and selecting the generated project as `gateway/work/libero/BVF_GATEWARE_025T.prjx`.

5.13.3 Exploring The Design

Let the IDE load everything, and then you're all set to browse around! You can go to the Design Hierarchy view to look at all Smart Design components. Here, all the gateway components are listed in block-like views. Double click the `DEFAULT_*****` option in the hierarchy to have a look at the whole gateway. You should also be able to see the cape, M.2 interface and the RISC-V subsystem modules. These modules are explained in [Gateway Introduction](#).

5.13.4 Adding Custom HDL

Once you're done exploring, we can start by adding our first HDL to the design.

Create a new HDL file through the menu bar, and name it `blink`.

Once created, you can find the HDL file under the User HDL Source Files heading in the Design Hierarchy.

Next, add this code to the file:

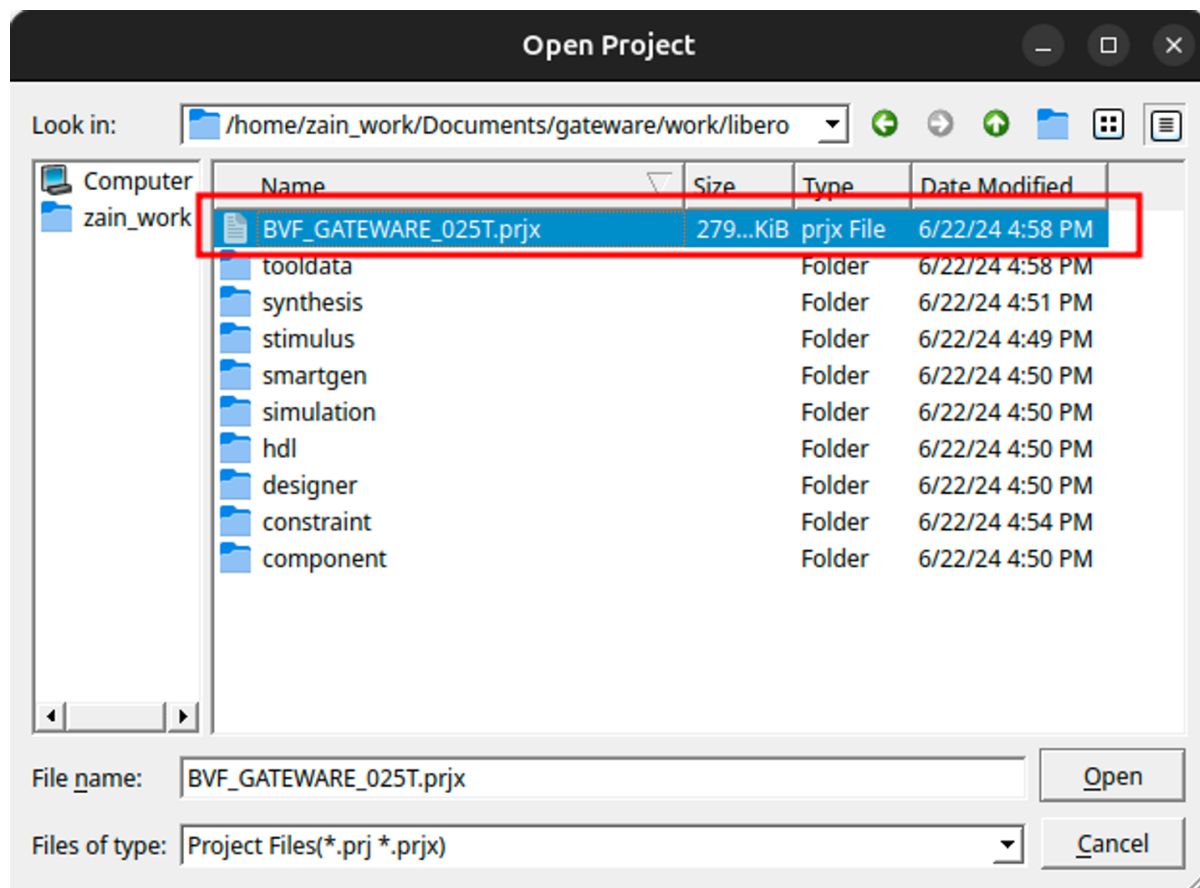


Fig. 5.15: Libero project location

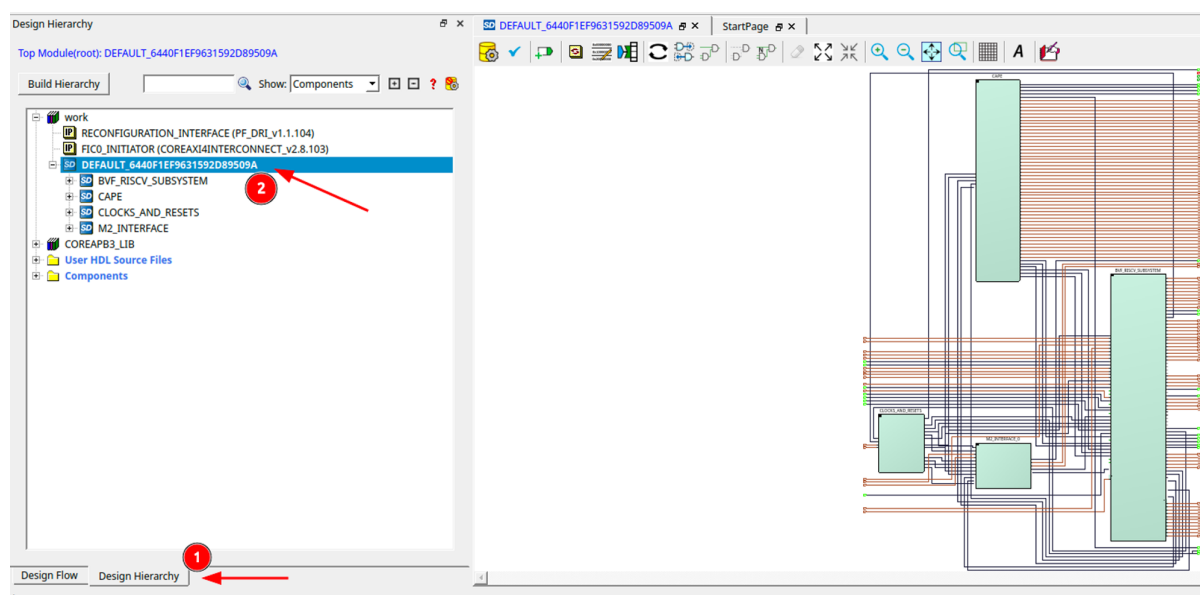


Fig. 5.16: Libero gateway overview

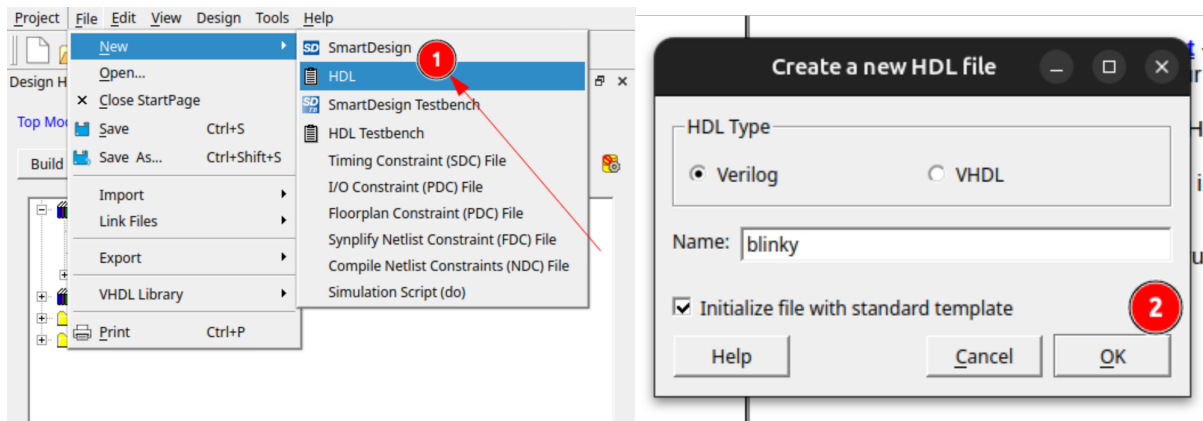


Fig. 5.17: Adding new HDL

```

`timescale 1ns/100ps
module blinky(
input    clk,
input    reseth,
input [27:0] gpio_out,
input [27:0] gpio_enable,
output [27:0] modified_gpio,
output [27:0] modified_gpio_enable
);

reg [22:0] counter;
assign modified_gpio = {gpio_out[27:6], counter[22], gpio_out[4:0]};
assign modified_gpio_enable = {gpio_enable[27:6], 1'b1, gpio_enable[4:0]};

always@(posedge clk or negedge reseth)
begin
    if(~reseth)
        begin
            counter <= 23'h0;
        end
    else
        begin
            counter <= counter + 23'b1;
        end
    end
end
endmodule

```

After saving it, press the **Build Hierarchy** button in the Design Hierarchy window to refresh it, and bring the added HDL to the work directory.

Right click on it to select the “Create Core from HDL...” option.

Press **No** on the dialog that follows since we’ve described the ports completely in our HDL.

Now, double click the **CAPE** design under the **DEFAULT_***** smart design, to have a look at what’s in the cape.

Drag and drop the **blinky** file appearing in the work section into the cape design.

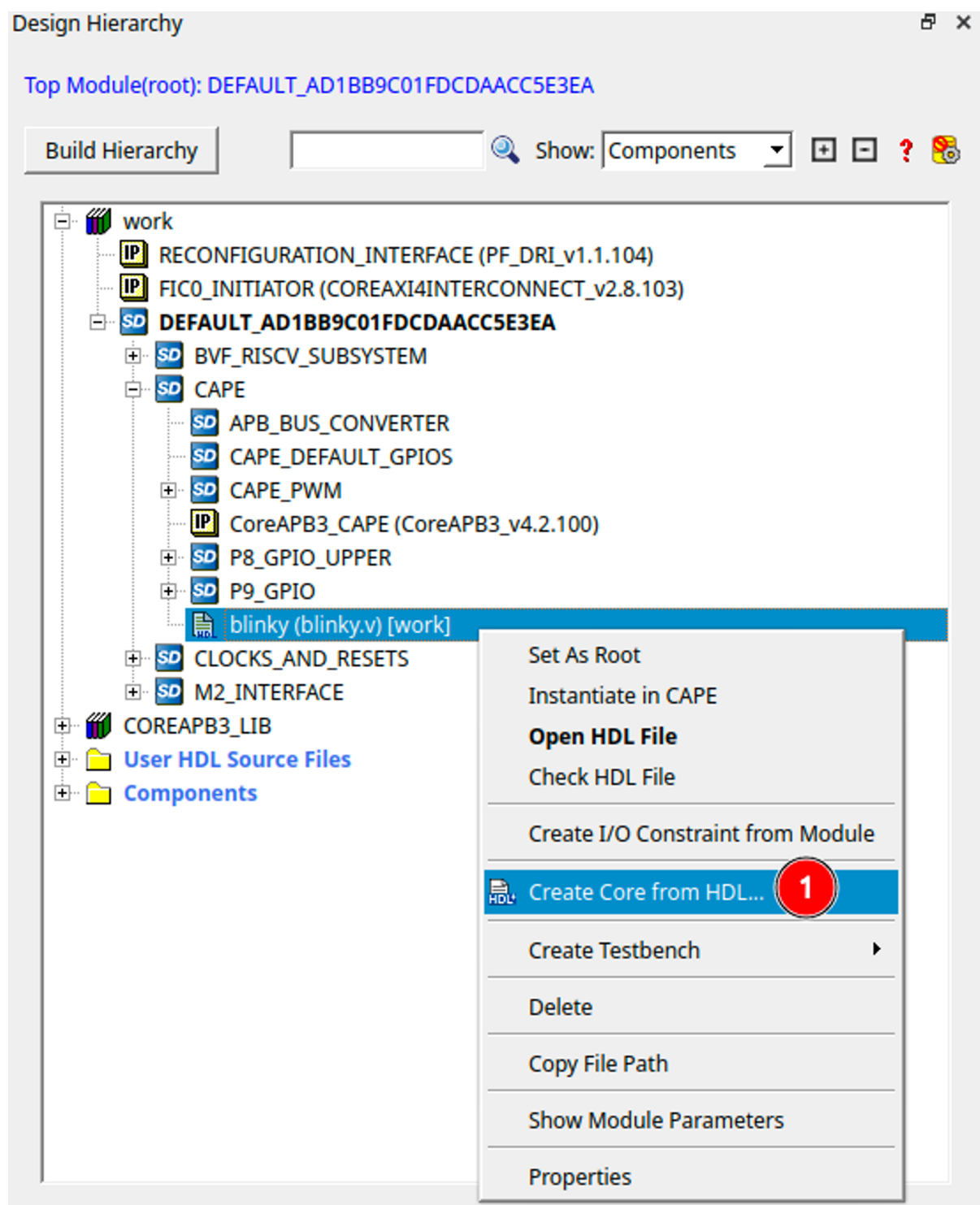


Fig. 5.18: Create core from HDL

Finally, connect the CLK and the RESET pins to the PCLK and the PRESETN pins below in the cape. You can use the **compress layout** button in the toolbar to make the design neat once you're done connecting the wires.

Go ahead and save the CAPE file.

You can also verify the design by pressing the checkmark icon in the editor toolbar.

Now, it's time to export our design back to the gateway repository.

Tip: You will hear **Exporting** and **Backannotation** used interchangeably; it's the same thing.

5.13.5 Exporting The Design

Exporting the Cape

The SmartDesigns you have changed should show an "i" icon in front of them indicating that they need to be regenerated.

First, regenerate the designs by right clicking on them and selecting "Generate Component".

Rebuild the Hierarchy too as we've done before.

Next, right-click on the cape and select "Export Component Description (TCL)" to export it as a script which can be used in the gateway repository.

I suggest creating an export directory where you can temporarily store the exported gateway files before getting them into the repository.

Important: You **must** make sure your path exist, because Libero does not currently tell you if the export is successful or not.

Now, simply copy it into the gateway at the following path.

```
cp ~/export/gateway/CAPE.tcl ~/gateway/sources/FPGA-design/script_support/  
↪components/CAPE/DEFAULT/
```

Tip:

You will find that sometimes Libero can export a TCL file that's quite different from what you started out with, even when you've only made *relatively* small changes.

It is worth spending a little time to look at the changes with a tool like git diff (VS Code works really well too), to make sure Libero didn't just run off on a tangent with your code.

Exporting The HDL

To add new HDL to the gateway repository, first we need to copy it to the HDL directory at `sources/FPGA-design/script_support/components/CAPE/DEFAULT/HDL`.

You can do that by just creating a folder named blinky inside and copying the HDL to it.

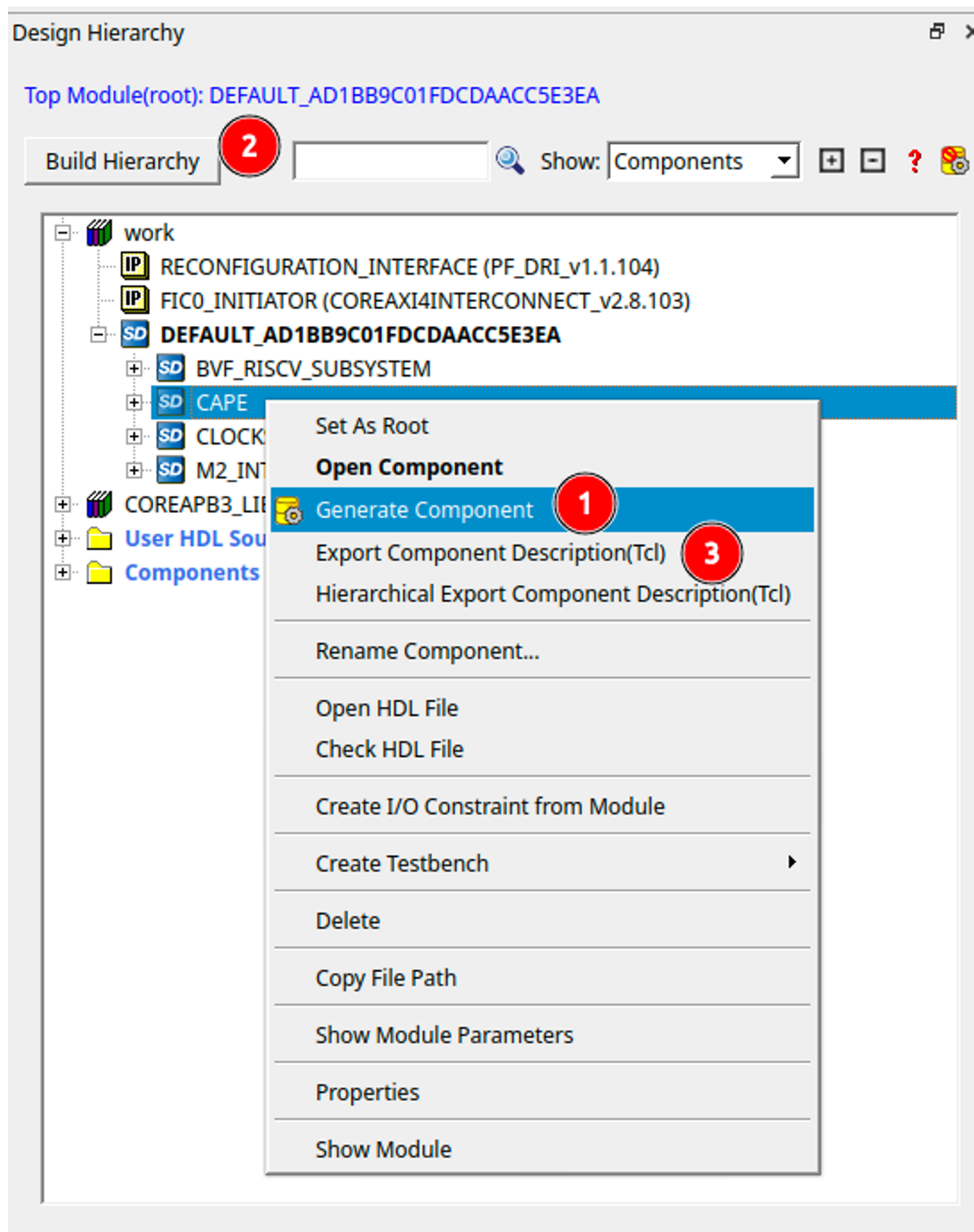


Fig. 5.21: Regenerate designs

```
mkdir ~/gateway/sources/FPGA-design/script_support/components/CAPE/DEFAULT/  
↪HDL
```

```
cp ~/gateway/work/libero/hdl/blinky.v ~/gateway/sources/FPGA-design/script_  
↪support/components/CAPE/DEFAULT/HDL/
```

Now, to add the TCL script to import this design for the CAPE scripts,
we can export the script by right-clicking on the HDL file in the Design Hierarchy and select Export
Component Description.

Now, copy this exported file to our gateway's CAPE directory at *gateway/sources/FPGA-design/script_support/components/CAPE/DEFAULT/* like so:

```
cp blinky.tcl ~/gateway/sources/FPGA-design/script_support/components/CAPE/  
↪DEFAULT/blinky_config.tcl
```

Then we make sure it's getting sourced by adding the following to `ADD_CAPE.tcl`:

Verify your script as above, save it and now you're good to compile your project!

Important:

Make sure you close Libero at this point.

If you don't, `build-bitstream.py` **will fail** to properly checkout the required licenses.

Now is a good time to check in your changes to git:

```
cd ~/gateway  
git add ./sources/FPGA-design/script_support/components/CAPE/DEFAULT/CAPE.tcl  
git add ./sources/FPGA-design/script_support/components/CAPE/DEFAULT/ADD_  
↪CAPE.tcl  
git add ./sources/FPGA-design/script_support/components/CAPE/DEFAULT/blinky_  
↪config.tcl  
git add ./sources/FPGA-design/script_support/components/CAPE/DEFAULT/HDL/  
↪blinky.v  
git clean -df
```

5.13.6 Final Verification

Go ahead and run the python script to build the gateway and verify your changes:

```
python build-bitstream.py ./build-options/default.yaml
```

If at any point the compilation fails, you can debug the script at the mentioned line.

If it compiles successfully, it will mention it by saying:

```
The Execute Script command succeeded.  
The BVF_GATEWARE_025T project was closed.
```

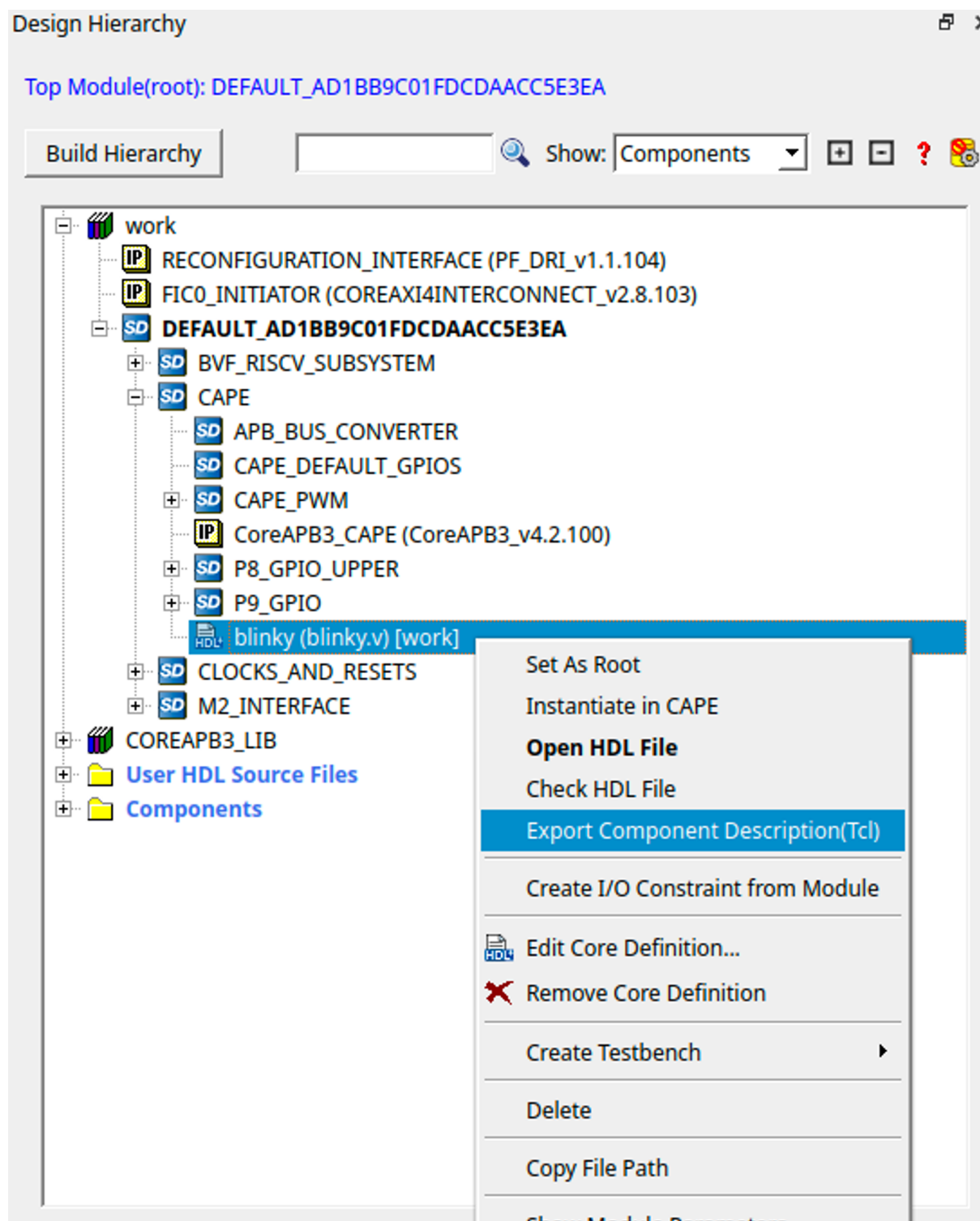


Fig. 5.22: Export HDL

```
sources > FPGA-design > script_support > components > CAPE > DEFAULT >  ADD_CAPE.tcl
1  puts "=====Add cape option: DEFAULT====="
2
3  #-----
4  # Build cape's submodules
5  #-----
6  ::safe_source script_support/components/CAPE/shared/APB_BUS_CONVERTER.tcl
7  ::safe_source script_support/components/CAPE/shared/CoreAPB3_CAPE.tcl
8  ::safe_source script_support/components/CAPE/shared/CoreGPIO_P8_UPPER.tcl
9  ::safe_source script_support/components/CAPE/shared/P8_GPIO_UPPER.tcl
10 ::safe_source script_support/components/CAPE/shared/CoreGPIO_P9.tcl
11 ::safe_source script_support/components/CAPE/shared/P9_GPIO.tcl
12 ::safe_source script_support/components/CAPE/shared/CAPE_DEFAULT_GPIOS.tcl
13 ::safe_source script_support/components/CAPE/shared/CorePWM_C1.tcl
14 ::safe_source script_support/components/CAPE/shared/CAPE_PWM.tcl
15 ::safe_source script_support/components/CAPE/$cape_option/blink_config.tcl
16 ::safe_source script_support/components/CAPE/$cape_option/CAPE.tcl
17
```

With a little luck, the script completes successfully and you can now send your changes onto your gateway repository fork, download the artifacts after compilation, and program the gateway using the `change_gateway.sh` script.

Tip: For a more direct route you can copy the generated bitstream straight to your Beagle and try the result immediately:

```
scp -r ./bitstream beagle@<ip or name here>:
```

On the beagle, use:

```
sudo /usr/share/beagleboard/gateway/change-gateway.sh ./bitstream
```

Have fun!

5.14 Simulating Gateway Design with Libero

In this demonstration, we will have a look at simulating the gateway design in Libero. Through simulations, one can verify the functionality of the design before implementing it on the hardware. The simulation is done using the ModelSim simulator, which is integrated with Libero. The gateway design that we will simulate is the blinky LED design, present in the `VERILOG_TUTORIAL` gateway option.

5.14.1 Prerequisites

1. Libero SoC 2022 or later. You can follow this guide to install Libero SoC: [Microchip 2025.1 FPGA Tools Installation Guide](#).
2. A copy of the [gateway repository](#).
3. The `setup_microchip_tools.sh` file, to start the license server and set the environment variables. The setup of this script is also covered in the installation guide.

5.14.2 Setting up ModelSim

Modelsim requires certain libraries to be present in the system, which might not be installed by default. You can check if ModelSim is working by:

```
source setup_microchip_tools.sh
$LIBERO_INSTALL_DIR/ModelSimPro/linuxacoem/vsim
```

If ModelSim is not working due to missing libraries, you might see an error message like:

```
vsim: error while loading shared libraries: libXft.so.2: cannot open shared_
↳ object file: No such file or directory
```

To fix this, you can install the required libraries using the following command:

```
sudo apt-get install libxft2 libxft2:i386 lib32ncurses5
```

If you still cannot run ModelSim, you can try fixes from this [guide](#).

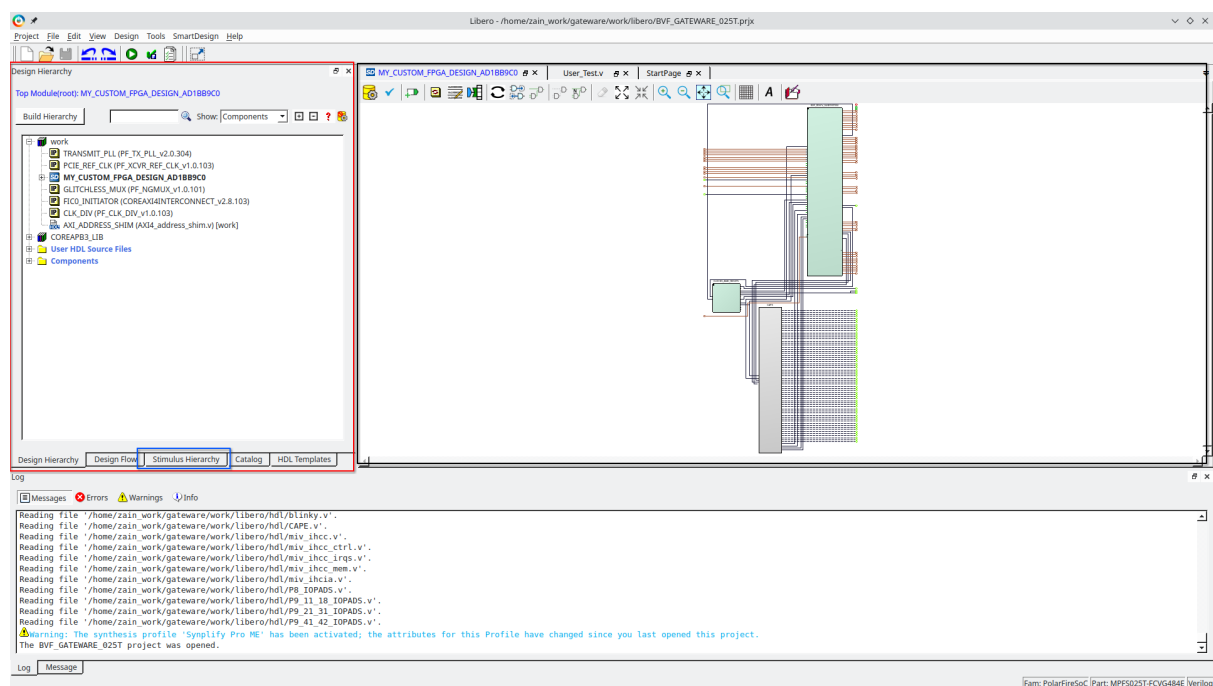
5.14.3 Simulating the Blinky LED Design

To start with the simulation, we must first compile the gateway design. The blinky LED design is present in the VERILOG_TUTORIAL gateway option. To compile the design, follow the steps below:

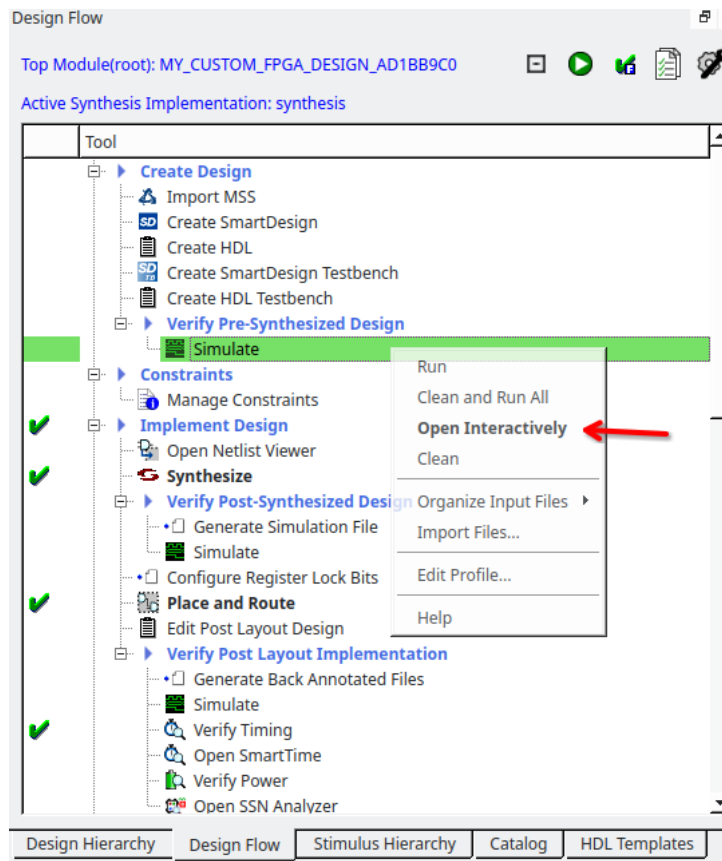
1. First, compile the gateway design using the following command:

```
cd gateway
python build-bitstream.py custom-fpga-design/my_custom_fpga_design.yaml
```

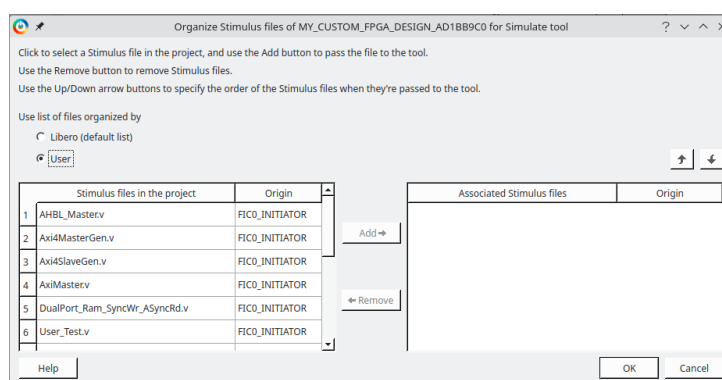
1. Once the design is compiled, open the Libero SoC software and select the created project. You can find the project in the `work/libero` directory.
2. Once the project is opened, your window should look something like this. In front of you will be the overview of the gateway design, and on the left you will have multiple tabs showing the design hierarchy, design flow, etc.



1. For the simulation, testbench files will need to be set up, which will be used to simulate the design. This can be done from the stimulus hierarchy. However, for this example we will be using the default testbench files provided by libero.
2. To set up the simulation, go to the Design Flow tab and right click on the Simulate button to select Open Interactively.



1. Before starting modelsim, Libero will ask you to add any additional files that you want to include in the simulation. For now, let's go with the ones that came with the design and its IPs.

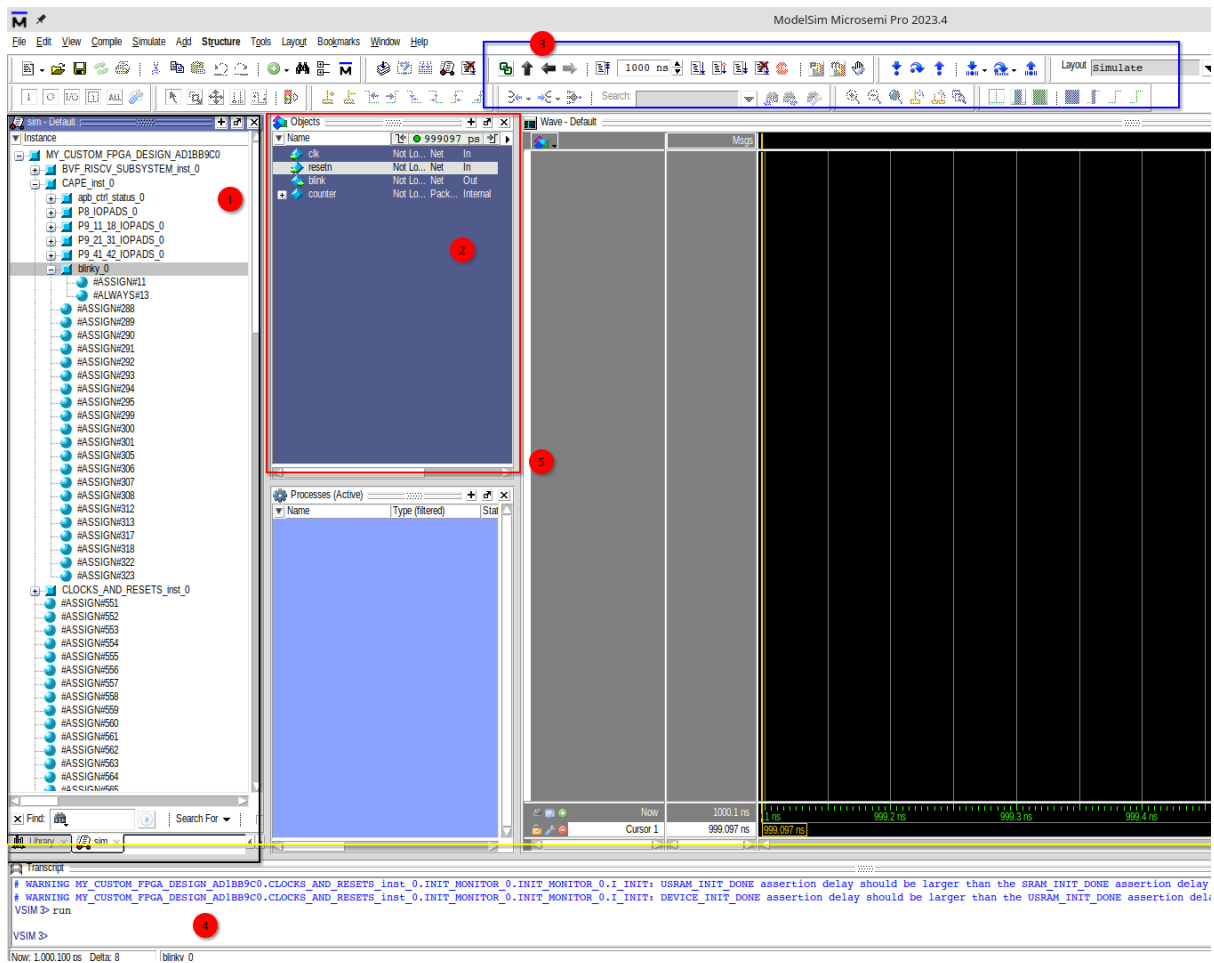


1. Once the simulation is started, you will see the ModelSim window open up.

5.14.4 Exploring ModelSim and Running the simulations

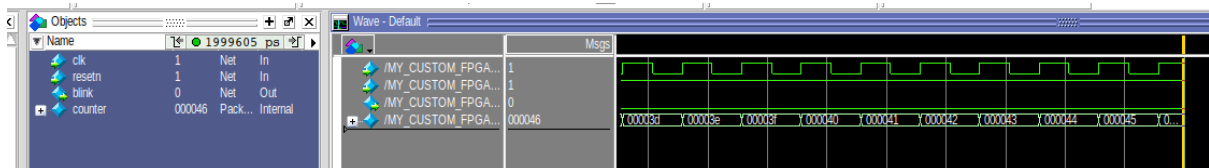
Looking at the modelsim window, there are four main sections to look at:

1. The top left section shows the design hierarchy. This is where you can see the design modules and their instances.



2. The section beside the design hierarchy is the object hierarchy. This shows the objects in the design, including the signals and variables.
3. At the top, you should see the simulation toolbar. This is where you can run the simulations, add break-points, etc.
4. At the bottom, you should see the transcript window. This is where you can see the simulation logs. This also acts as a command line interface for ModelSim.
5. The far right section is the waveform window. This is where you can see the waveforms of the signals in the design.

You can add signals to the waveform window by right clicking on the signal in the object hierarchy and selecting **Add to Wave**. Once added, you can run the simulation by clicking on the **Run** button in the simulation toolbar. The simulation will run for a few nano seconds as specified in the toolbar beside the **Run** button.



Once the simulation is complete, you can see the waveforms of the signals in the waveform window.

If you want to automate addition of signals to the waveform, you see the output of each GUI command in the transcript window. You can use these commands to automate the process. Just put the commands in a file with a `.do` extension and run the file using the `do` command in the transcript window.

An example of a `.do` file is shown below:

```
add wave -noupdate /tb_top/clk
add wave -noupdate /tb_top/rst
add wave -noupdate /tb_top/led

run 100 ns
```

A default `run.do` script is created at the following path - `work/libero/simulation` - when a simulation is run. You can use this file as a starter file for creating your own scripts as well as for understanding how the initial simulation is set-up.

Good luck with your simulations!

5.15 Comms Cape Gateway for BeagleV-Fire

The comms cape provides an array of communication protocols including

- one RS485,
- one CAN,
- two analog 4-20 mA current loops,
- two 3A 50V interfaces allowing the control of high current loads.

5.15.1 Cape schematics, layout, and mechanicals

For the schematics, layout, and mechanicals of the cape, please refer to `industrial-comms-cape`.

5.15.2 Usage

Firstly, the comms cape gateway must be compiled and updated on the beagleV-Fire. This can be done by using the *build-bitstream.py* script in the gateway repository with the *cape_comms.yaml* build option file.

```
python3 build-bitstream.py build-options/cape_comms.yaml
```

CAN

RS485

The RS485 interface is connected to UART4 on the BeagleV-Fire. It can be accessed using */dev/bone/uart/4* in Linux.

Sink drivers

The sink drivers are connected to the P9_15 and P9_23 GPIOs. They can be controlled by writing to the GPIOs by:

```
echo 425 > /sys/class/gpio/export
echo 431 > /sys/class/gpio/export
echo out > /sys/class/gpio/gpio425/direction
echo out > /sys/class/gpio/gpio431/direction
echo 1 > /sys/class/gpio/gpio425/value
echo 1 > /sys/class/gpio/gpio431/value
```

Current loops

The current loops are connected to the ADC inputs of the BeagleV-Fire at pins P9_35 and P9_36. They can be accessed once QSPI is enabled in the comms cape device tree overlay. The current loops can be read by:

```
cat /sys/bus/iio/devices/iio:device0/in_voltage5_raw #Current Loop A
cat /sys/bus/iio/devices/iio:device0/in_voltage6_raw #Current Loop B
```

Voltage to current conversion will have to be done in software.

5.15.3 Pinout

The full pinout for the cape interface spec can be found [here](#). You can also see the pinout below, refer to the last column for comms-cape specific pins.

P8 Header

Signal	Control	Irq #	Description
P8_1	n/a	n/a	GND
P8_2	n/a	n/a	GND
P8_3	MSS GPIO_2[0]	53	User LED 0
P8_4	MSS GPIO_2[1]	53	User LED 1
P8_5	MSS GPIO_2[2]	53	User LED 2
P8_6	MSS GPIO_2[3]	53	User LED 3
P8_7	MSS GPIO_2[4]	53	User LED 4
P8_8	MSS GPIO_2[5]	53	User LED 5

continues on next page

Table 5.1 – continued from previous page

Signal	Control	Irq #	Description
P8_9	MSS GPIO_2[6]	53	User LED 6
P8_10	MSS GPIO_2[7]	53	User LED 7
P8_11	MSS GPIO_2[8]	53	User LED 8
P8_12	MSS GPIO_2[9]	53	User LED 9
P8_13	core_pwm[1] @ 0x41500000	n/a	PWM_2:1
P8_14	MSS GPIO_2[11]	53	User LED 11
P8_15	MSS GPIO_2[12]	53	GPIO
P8_16	MSS GPIO_2[13]	53	GPIO
P8_17	MSS GPIO_2[14]	53	GPIO
P8_18	MSS GPIO_2[15]	53	GPIO
P8_19	core_pwm[0] @ 0x41500000	n/a	PWM_2:0
P8_20	MSS GPIO_2[17]	53	GPIO
P8_21	MSS GPIO_2[18]	53	GPIO
P8_22	MSS GPIO_2[19]	53	GPIO
P8_23	MSS GPIO_2[20]	53	GPIO
P8_24	MSS GPIO_2[21]	53	GPIO
P8_25	MSS GPIO_2[22]	53	GPIO
P8_26	MSS GPIO_2[23]	53	GPIO
P8_27	MSS GPIO_2[24]	53	GPIO
P8_28	MSS GPIO_2[25]	53	GPIO
P8_29	MSS GPIO_2[26]	53	GPIO
P8_30	MSS GPIO_2[27]	53	GPIO
P8_31	core_gpio[0] @ 0x41100000	126	GPIO
P8_32	core_gpio[1] @ 0x41100000	127	GPIO
P8_33	core_gpio[2] @ 0x41100000	128	GPIO
P8_34	core_gpio[3] @ 0x41100000	129	GPIO
P8_35	core_gpio[4] @ 0x41100000	130	GPIO
P8_36	core_gpio[5] @ 0x41100000	131	GPIO
P8_37	core_gpio[6] @ 0x41100000	132	GPIO
P8_38	core_gpio[7] @ 0x41100000	133	GPIO
P8_39	core_gpio[8] @ 0x41100000	134	GPIO
P8_40	core_gpio[9] @ 0x41100000	135	GPIO
P8_41	core_gpio[10] @ 0x41100000	136	GPIO
P8_42	core_gpio[11] @ 0x41100000	137	GPIO
P8_43	core_gpio[12] @ 0x41100000	138	GPIO
P8_44	core_gpio[13] @ 0x41100000	139	GPIO
P8_45	core_gpio[14] @ 0x41100000	140	GPIO
P8_46	core_gpio[15] @ 0x41100000	141	GPIO

P9 Header

Signal	Control	Irq #	Description
P9_1	n/a	n/a	GND
P9_2	n/a	n/a	GND
P9_3	n/a	n/a	VCC 3.3V
P9_4	n/a	n/a	VCC 3.3V
P9_5	n/a	n/a	VDD 5V
P9_6	n/a	n/a	VDD 5V
P9_7	n/a	n/a	SYS 5V
P9_8	n/a	n/a	SYS 5V
P9_9	n/a	n/a	NC
P9_10	n/a	n/a	SYS_RSTN

continues on next page

Table 5.2 – continued from previous page

Signal	Control	Irq #	Description
P9_11	MMUART4	94	UART4 RX <— For RS485
P9_12	core_gpio[1] @ 0x41200000	143	GPIO
P9_13	MMUART4	94	UART4 TX <— For RS485
P9_14	core_pwm[0] @ 0x41400000	n/a	PWM_1:0
P9_15	core_gpio[4] @ 0x41200000	146	GPIO
P9_16	core_pwm[1] @ 0x41400000	n/a	PWM_1:1
P9_17	MSS SPI0	54	SPI0 CS
P9_18	MSS SPI0	54	SPI0 MOSI
P9_19	MSS I2C0	58	I2C0 SCL
P9_20	MSS I2C0	58	I2C0 SDA
P9_21	MSS SPI0	54	SPI0 MISO
P9_22	MSS SPI0	54	SPI0 SCLK
P9_23	core_gpio[10] @ 0x41200000	152	GPIO
P9_24	CAN_1_RXBUS	n/a	CAN RX <— For CAN
P9_25	CAN_1_TX_EBL	154	CAN TX EBL
P9_26	CAN_1_TXBUS	n/a	CAN TX
P9_27	core_gpio[14] @ 0x41200000	156	GPIO
P9_28	MSS SPI1	55	SPI1 CS
P9_29	MSS SPI1	55	SPI1 MISO
P9_30	core_gpio[17] @ 0x41200000	159	GPIO
P9_31	MSS SPI1	55	SPI1 SCLK
P9_32	n/a	n/a	VDD ADC
P9_33	n/a	n/a	ADC input 4
P9_34	n/a	n/a	AGND
P9_35	n/a	n/a	ADC input 6
P9_36	n/a	n/a	ADC input 5
P9_37	n/a	n/a	ADC input 2
P9_38	n/a	n/a	ADC input 3
P9_39	n/a	n/a	ADC input 0
P9_40	n/a	n/a	ADC input 1
P9_41	core_gpio[19] @ 0x41200000	161	GPIO
P9_42	core_pwm[0] @ 0x41000000	n/a	PWM_0:0
P9_43	n/a	n/a	GND
P9_44	n/a	n/a	GND
P9_45	n/a	n/a	GND
P9_46	n/a	n/a	GND

5.16 Accessing APB and AXI Peripherals Through Linux

5.16.1 AXI

AXI is part of the [ARM AMBA](#) (Advanced Microcontroller Bus Architecture) protocol family.

It is designed for high-performance, high-frequency system-on-chip (SoC) designs.

AXI provides high-speed data transfer with minimal latency and is widely used in various applications, including high-end embedded systems and complex digital circuits.

5.16.2 APB

APB is also part of the ARM AMBA protocol family, designed for low-power and low-latency communication with peripheral devices.

It is simpler and lower performance compared to AXI, making it suitable for slower peripheral devices. An APB peripheral also consumes less resources on the FPGA fabric compared to an AXI peripheral.

5.16.3 Accessing AXI and APB Peripherals from Linux

To access AXI and APB peripherals from Linux, memory-mapped I/O (MMIO) is commonly used.

This involves mapping the physical addresses of the peripherals into the virtual address space of a user-space application.

The following sections demonstrate how to access APB peripherals using the Linux `/dev/mem` interface and AXI peripherals using the UIO (Userspace I/O) framework.

Note: The codes for accessing the interfaces are available in the snippets here: [APB Interfaces](#) and [AXI Interfaces](#)

APB Interfaces

The MSS includes fabric interfaces for interfacing FPGA fabric with the CPU Core Complex.

It provides one 32-bit APB master interface, FIC3, and can be connected to a slave in the fabric.

Design Details For this example, you can try to write to the APB slave present in the Verilog Tutorial Cape gateway. Select the gateway by changing *custom-fpga-design/my_custom_fpga_design.yaml* to include `VERILOG_TUTORIAL` as the cape option.

The APB Slave has two registers, one read-only register at `0x00`, one read-write register at `0x10`, and a status register containing the last read value at `0x20`.

Having a look at the design, we can see that the APB slave is connected with a CoreAPB3 interconnect, which assigns it the `0xXX10_0000` address, the top two bits being ignored.

Tracing to the master connected with the CoreAPB3 device, we can see that another interconnect is present, which gives our slave the `0xX100_0000` address.

The polarfire technical manual shows that FIC3 peripherals can start from the `0x4000_0000` address.

Therefore, the final address of our APB slave becomes `0x4110_0000`.

Now, we shall access this address through a memory-mapped interface in Linux.

Important:

The following paragraphs will present to you several ways to test APB/AXI traffic.

Normally, this isn't harmful, but reading/writing to addresses with no gateway behind it **will** lead to you stalling a CPU.

In rare cases, this stalling of a CPU *can* lead to loss of content on your eMMC, so please make sure you have a known good backup!

For more information about the stalls, please read the section [on issues faced with the interfaces](#).

Accessing the Interface There are two ways to access such registers. One can use the *devmem2* utility or write a C program for accessing the memory region. The first method is quite simple.

1. To read from a register:

```
sudo devmem2 0x41100000 w
```

2. To write to a register:

```
sudo devmem2 0x41100010 w 0x1
```

In the second method, we can use the `/dev/mem` interface to access the registers inside the APB Slave. Here is an example C program which demonstrates this:

AXI Interfaces

The MSS includes three 64-bit AXI FICs out of which FIC0 is used for data transfers to/from the fabric. FIC0 is connected as both master and slave. For usage of AXI peripherals, an example is also provided by microchip in their [Polarfire SoC Linux examples](#). The example here takes reference from the [AXI LSRAM example](#).

Design Details

A simple design can be created by first connecting the FIC0 Initiator from the MSS to a [CoreAXI4Interconnect](#). Now, you can connect an AXI slave to this interconnect. We will be using the Polarfire AXI LSRAM.

Both the CoreAXI4Interconnect and the PF AXI LSRAM will have to be configured.

The AXI ID Width of both the modules will have to be matched, as well as the address space of the only slave will have to be configured.

In this example, LSRAM gets an address of *0x6000 0000* to *0x6000 ffff*, and the AWID is kept at 9 bits.

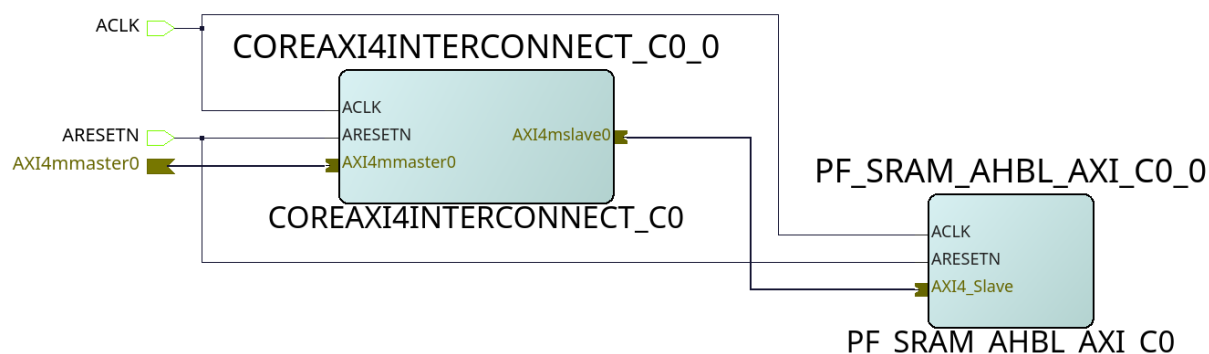


Fig. 5.23: AXI LSRAM slave (example design)

Finally, an entry will be added to the device tree to make a UIO device point to our LSRAM's memory region.

```
&{/} {
    fabric-bus@40000000 {
        fpgalsram: uio@60000000 {
            compatible = "generic-uio";
            linux,uio-name = "fpga_1sram"; // mandatory for program. If
changed, please update program as well.
            reg = <0x0 0x60000000 0x0 0x1000>;
            status = "enabled";
        };
    };
};
```

Once the gateway is compiled, we can access the memory-mapped interface by the same methods, and by the UIQ device as well.

1. Using devmem2:

```
sudo devmem2 0x60000000 w # for read
sudo devmem2 0x60000000 w 0x1 # for write
```

1. Using the UIO device:

Issues that can be faced when using an improperly configured AXI/APB interface

A CPU stall can be faced when accessing the FIC interfaces without any slaves connected to the memory region being accessed. Your BVF will stop responding if connected to SSH, and on serial you will see the following kernel messages:

```
[ 24.110099] rcu: INFO: rcu_sched detected stalls on CPUs/tasks:
[ 24.116041] rcu:      0-...0: (1 GPs behind) idle=e00c/0/0x1 softirq=40/41
→fqs=2626
[ 24.123377]      (detected by 3, t=5255 jiffies, g=-1131, q=9 ncpus=4)
[ 24.129573] Task dump for CPU 0:
[ 24.132810] task:swapper/0      state:R  running task      stack:0
→pid:0      ppid:0      flags:0x00000008
[ 24.142757] Call Trace:
[ 24.145213] [<ffffffff80a67ba0>] __schedule+0x27c/0x834
```

If this happens, please double check your design. Specifically, check the address configured for the slaves, the AXI ID wire width and other AXI parameters.

In any case, this state is virtually impossible to recover from gracefully, so the **reset** button may be your last resort.

5.17 Change the MSS configuration

With the introduction of support for multiple MSS configurations in the same builder, the MSS part was modularized to abstract away some of the intricacies to ease the price-of-admittance for people just getting started with PolarFire® architecture.

5.17.1 Quick recap

As seen in [Design & specifications](#), PolarFire® is split into two distinct parts:

- Fabric: This is where you put your logic design.
- MSS: This is the hard processor and peripherals.

This last part is the subject of today's discussion.

MSS module

We are leveraging two of `git`'s built-in features to compartmentalize the MSS as much as possible:

- Bundle: Creates a nice and compact copy of the Repository for easy distribution.
- Branches: Eliminates the need for a myriad of same-but-not-quite config files.

If you haven't encountered `git` bundles before, please refer to its documentation.

Unrolling the bundle is as easy as:

```
git clone -b qspi sources/FPGA-design/mss.bundle [your-workdir-here]
```

where `qspi` signifies the particular variant you want to work on.

Note: You don't *need* to specify a branch; `git` will check out the latest one if you don't.

Navigate to the cloned directory and do a:

```
git log --all --decorate --oneline --graph
```

and you'll see something like this:

```
* f3228ed (origin/default, origin/HEAD) Swap SPI_1 and QSPI connections
* ddf1142 Change MSS configuration
* d9f830d (HEAD -> qspi, origin/trunk, origin/qspi) Add ADAPTER abstraction_
  ↳ layer
* b5bd77f Add DRAM subsys
* 521a2b6 Add base files
* 7152b35 Modularized MSS starting point
```

Notice how you're parked at commit `d9f830d`.

Now you pick the branch that has the best starting-point for you and

```
git checkout -b [my-new-variant] [start-branch]
```

where `start-branch` is either `qspi` or `default`, depending on your requirements.

Tip: From here you can switch your builder to use the cloned Repository directly:

```
MSS:
  type: git
  local: your-workdir-here
  branch: my-new-variant
```

Just remember to commit your changes **before** running the builder; otherwise it won't pick them up.

Looking at the graph above, commit `ddf1142` and `f3228ed` is provided as an example of what's possible, but obviously only the sky is the limit.

5.17.2 Backannotation

Once happy with your new variant, you can backannotate your Repository by:

```
git bundle create ~/builder/sources/FPGA-design/mss.bundle --all
```

and then undo the change you made to `local` in the build-options `.yaml` file.

Obviously, you do not want to change the `branch` value.

5.17.3 Version check

The running version can be seen at line **341** in the figure above.

5.18 Help! I broke my board. Now what?

Ok, so you managed to create, upload a bad Gateware build and now BeagleV-Fire won't boot anymore. Is it dead? Do you throw it out or RMA the thing?

Never fear, help is here, but first things first: The Big Picture!

5.18.1 The Big Picture

Just like you got taught in First Aid training, the steps are:

- Stop the Accident.
- Get the Big Picture.

So we turn off the power and try to get the Big Picture.

Like so many of it's cousins, BeagleV-Fire also comes with a debug serial port, and we need to connect to it for a level 1 triage:

If you already have, you're all set and can proceed to the next section. Otherwise, please take a look at [how to set it up](#) and then come back here.

Right, you're back. Lets get your favorite Serial Terminal program fired up and set the BAUDRATE to **115200**. From here on out, we're going to assume that your serial is working.

Tip: If in doubt, short RX and TX **on the probe side** and type something in the Terminal.

5.18.2 Level 1: Triage

Ok, now that we're no longer driving blind, lets turn the power back on and take a first quick look at the situation.

One of two things will happen on the Terminal at this point:

- Absolutely nothing.
- The hart-software-services, HSS for short, will greet you.

It's dead, Jim!

Ouch! That was the worst possible outcome. You've broken the processors ability to start properly. Did you mess with the MSS configuration?

In this case, you're going to need a `FlashPro 5` from Microchip for JTAG access. This procedure is outside the scope of this document, but luckily Microchip has good documentation on how to recover a stalled PolarFire SoC.

HSS started, phew!

Alright, it's not completely dead, so we continue down the path.

Next junction:

- It does not find and execute u-boot.
- It does!

In any case, onward to level 2...

5.18.3 Level 2: U-Boot

U-Boot was not found by HSS

Ok, uncommon problem when flashing Gateware, but nevertheless, lets address the problem:
Your eMMC image got damaged, so you'll *have to reprogram it*.

Once that's taken care of, we can continue to the next step.

U-Boot found and loaded

So far, so good. U-Boot got loaded.

Next junction:

- It is unable to locate a Linux kernel.
- It boots into Linux.

No Linux kernel

Remedy same as before; the eMMC image is damaged, so you'll *have to reprogram it*.

Note:

As reprogramming the eMMC fixes **both** U-Boot **and** Linux kernel at the same time, it is very unlikely that you'll hit this problem twice.

Linux boots, but Oops's before reaching Userland

This is **the most** common scenario, so lets examine how to deal with that:

Press the **Reset** button and then let BeagleV-Fire restart,
but interrupt U-Boot by pressing **<ESC>** to break into the command prompt.

Tip:

If by some hand of Evil, U-Boot does not react to you pressing **<ESC>**, it might be because HSS is still hugging the serial port.

Try restarting the process by pressing the **Reset** button, but this time you interrupt on the HSS step by hitting **<Enter>**.

(yes, the magic incantation is different).

Once on the HSS command prompt, give the command `boot` to proceed and return here.

5.18.4 Level 3: Linux boot

Now, what U-Boot normally does behind the scenes is to run the following:

```
# Try to boot a fitImage from eMMC/SD

setenv fdt_high 0xffffffffffffffff
setenv initrd_high 0xffffffffffffffff

load mmc 0:${distro_bootpart} ${scriptaddr} beaglev_fire.itb;
bootm start ${scriptaddr}#kernel_dtb;
bootm loados ${scriptaddr};
# Try to load a ramdisk if available inside fitImage
bootm ramdisk;
bootm prep;
fdt set /soc/ethernet@20112000 mac-address ${icicle_mac_addr0};
fdt set /soc/ethernet@20110000 mac-address ${icicle_mac_addr1};
run design_overlays;
bootm go;
```

Since most of the time the kernel will be Oopsing on some device or memory you're trying to bring to the attention of the kernel, the way to push through to Userland is to make the kernel forget about your device completely.

As this knowledge is stored in the device-tree, what we want is to run the above, **except** for the `run design_overlays` bit.

Unfortunately, since the command buffer of U-Boot isn't overwhelmingly big, we have to break up the commands into bite-size chunks, so that's what we'll do:

Copy each of the following lines, one by one, using the nifty `Copy` button and paste it into your Terminal. (Reveal the button by hovering your Mouse over the line)

```
setenv fdt_high 0xffffffffffffffff
```

```
setenv initrd_high 0xffffffffffffffff
```

```
load mmc 0:2 ${scriptaddr} beaglev_fire.itb;
```

```
bootm start ${scriptaddr}#kernel_dtb;
```

```
bootm loados ${scriptaddr};
```

```
bootm ramdisk;
```

```
bootm prep;
```

```
fdt set /soc/ethernet@20112000 mac-address ${icicle_mac_addr0};
```

```
fdt set /soc/ethernet@20110000 mac-address ${icicle_mac_addr1};
```

Note:

We'll be skipping the offending overlays; this is **very much on purpose**.

Finally, this next bit should get us to Userland and we're free to upload a hopefully fixed Gateway build:

```
bootm go;
```

Tip:

If you're thinking, "hang on" here, you'd be right: You can do all kinds of other nifty things with that `fdt` command before you tell the kernel to rip.

The Sky's the limit; just be careful of that Sun burning those wings of yours...

5.19 Libero Setup - Ubuntu 24.04

Note: This guide is for advanced users comfortable with Linux system administration. It assumes a fresh installation of Ubuntu 24.04.1 LTS Desktop edition. Ubuntu 24.04.1 was used for testing this guide but, it's not directly supported by microchip libero and it's not guaranteed that other point releases will work without modification.

This guide is intended to help users to setup the libero design tools on a fresh ubuntu 24.04.1 installation. Similar steps are followed to what is provided the the [Microchip 2024.2 and Older FPGA Tools Installation Guide](#), [Microchip 2025.1 FPGA Tools Installation Guide](#), and that provided by Penguin a member of the BeagleV-Fire Community.

At the time of writing this guide versions of Libero SOC web installer v2024.2 and SoftConsole v2022.2 were available. It is possible that other versions of the installation process will work by substituting the appropriate version numbers but this has not been tested.

5.19.1 Steps

Download Tools

- Download the Libero SOC web installer from [microchip website - libero-software-later-versions](#)
- Download SoftConsole from [microchip website - softconsole](#)

Prepare Install Files

Move and prepare the install tools in an isolated location

```
mkdir -p ~/Documents/MicrochipInstallers
cd ~/Documents/MicrochipInstallers
mv ~/Downloads/Libero_SoC_v2024.2_Web_lin.zip .
mv ~/Downloads/Microchip-SoftConsole-v2022.2-RISC-V-747-linux-x64-installer.
  ↪run .
unzip Libero_SoC_v2024.2_Web_lin.zip
chmod +x Microchip-SoftConsole-v2022.2-RISC-V-747-linux-x64-installer.run
```

The default installation of Libero tries to pop things in a root only location. We are going to move things to a unified location in the user home directory. For now just create the storage folder for the next step.

```
mkdir -p ~/Microchip
```

Install the Tools

Start by installing Libero SOC. Don't forget to update the paths for the install directory and common directory to be in ~/Microchip We will be installing all of the tools.

```
~/Documents/MicrochipInstallers/launch_installer.sh
```

hold off on the post install script for the moment. It will be handled in a later step.

After Libero is completed run the installer for SoftConsole.

```
~/Documents/MicrochipInstallers/Microchip-SoftConsole-v2022.2-RISC-V-747-  
→linux-x64-installer.run
```

Getting a Libero License

To use Libero you will need to get a license from Microchip. Follow the steps below to get a free 1 year floating license.

- Visit [Microchip's fpga license page](#).
- Choose Libero Silver 1Yr Floating License for Windows/Linux Server from the list.
- Enter your MAC address and click register.
- Download your new license from your email once it arrives.
- Extract the license to ~/Microchip/license.
- Update the hostname field on line 1 marked by <put .hostname.here> with localhost

Post Install

Updated Dependencies Install Script Libero comes packaged with a script to install a few tools after the initial installation. Some of these have been renamed for Ubuntu 24.04. Run The following commands instead.

```
sudo dpkg --add-architecture i386  
sudo apt update  
sudo apt install -y libc6:i386 \  
libdrm2:i386 \  
libexpat1:i386 \  
libfontconfig1:i386 \  
libfreetype6:i386 \  
libglapi-mesa:i386 \  
libglvnd0:i386 \  
libgl1:i386 \  
libice6:i386 \  
libsm6:i386 \  
libuuid1:i386 \  
libx11-6:i386 \  
libx11-xcb1:i386 \  
libxau6:i386 \  
libxcb-dri2-0:i386 \  
libxcb-glx0:i386 \  
libxext6:i386 \  
libxfixes3:i386 \  
libxi6:i386 \  
libxrandr2:i386 \  
libxrender1:i386 \  
libxt6:i386 \  
libxxf86vm1:i386
```

(continues on next page)

(continued from previous page)

```

libxcb1:i386 \
libxdamage1:i386 \
libxext6:i386 \
libxf86vm1:i386 \
libxxf86vm1:i386 \
zlib1g:i386 \
libflac12t64 \
libpcre3 \
libxcb-xinerama0 \
libxcb-xinput0 \
xfonts-intl-asian \
xfonts-intl-chinese \
xfonts-intl-chinese-big \
xfonts-intl-japanese \
xfonts-intl-japanese-big \
ksh \
libxft2:i386 \
libgtk2.0-0t64:i386 \
libcanberra-gtk-module:i386 \
libfreetype-dev \
libharfbuzz-dev

```

Remove libstdc++ References Trying to run `libero` directly at this point will yield an error similar to this:

```

/home/lucien/Microchip/Libero_SoC_v2024.2/Libero/bin64/libero_bin: /home/
→lucien/Microchip/Libero_SoC_v2024.2/Libero/lib64/rhel/libstdc++.so.6:
→version `GLIBCXX_3.4.30' not found (required by /usr/lib/x86_64-linux-gnu/
→libcuc.so.74)

```

This is expected because a packaged version of `stdlibc++` is being used that was shipped with the installation. We need to delete all of the local versions so the system libs can be used instead.

```
find ~/Microchip/Libero_SoC_v2024.2 -name "libstdc++.so.6" -type f -delete
```

Create tmp dir Running the license server will cause issues with a folder that cant be created because of permission issues.

```

12:46:59 (lmgrd) Can't make directory /usr/tmp/.flexlm, errno: 2(No such
→file or directory)

```

It is best to pre-create it with the appropriate permissions for use.

```

sudo mkdir /usr/tmp
sudo chown $USER:$USER /usr/tmp

```

Missing lib This step is only required if you are using older versions of the license tool i.e. the ones packaged with versions of `liber` older than 2024.2/ Licensing tools v11.19.6.0. This guide is using the bundled tools with a version new enough that this shouldnt be an issue.

Older versions of the license tools depend on the `lsb` and `lsb-core` packages. These have been phased out in Ubuntu 24.04. We now end up with a missing link which causes a strange *no such file or directory* error when launching the license daemon. We need to create an equivalent link for this. I suggest:

```
sudo ln -s /lib64/ld-linux-x86-64.so.2 /lib/ld-lsb-x86-64.so.3
```

Setup Script This setup script is based on the one provided by Beagle. It has been updated to use the versions of Libero suggested in this guide and the licensing tools that are provided with the installation.

```
#!/bin/bash

#-----
# Edit the following section with the location where the following tools are
# installed:
# - SoftConsole (SC_INSTALL_DIR)
# - Libero (LIBERO_IN(.venv) [lorforlinux@DRAGON-I167610 docs.beagleboard.
io]$ git push lucien HEAD:add_24_04_docs
```

To openbeagle.org:LucienMorey/docs.beagleboard.io.git

! [rejected] HEAD -> add_24_04_docs (non-fast-forward)

error: failed to push some refs to 'openbeagle.org:LucienMorey/docs.beagleboard.io.git' hint: Updates were rejected because a pushed branch tip is behind its remote hint: counterpart. If you want to integrate the remote changes, use 'git pull' hint: before pushing again. hint: See the 'Note about fast-forwards' in 'git push -help' for details.STALL_DIR)

```
#-----
export SC_INSTALL_DIR=/home/$USER/Microchip/SoftConsole-v2022.2-RISC-V-747 export
port LIBERO_INSTALL_DIR=/home/$USER/Microchip/Libero_SoC_v2024.2 export LI-
CENSE_FILE_DIR=/home/$USER/Microchip/license
#-----
# The following was tested on Ubuntu 24.04.1 with: # - Libero 2024.2 # - SoftConsole 2022.2
#-----

# # SoftConsole # # # Libero # export PATH=$PATH:$LIBERO_INSTALL_DIR/Libero/bin:$LIBERO_INSTALL_DIR/Libero/bin64
export PATH=$PATH:$LIBERO_INSTALL_DIR/Synplify/bin export PATH=$PATH:$LIBERO_INSTALL_DIR/Model/modeltech/linux
export LOCALE=C export LD_LIBRARY_PATH=/usr/lib/i386-linux-gnu:$LD_LIBRARY_PATH

# # Libero License daemon # export LM_LICENSE_FILE=1702@localhost export
SNPSLMD_LICENSE_FILE=1702@localhost

$LIBERO export PATH=$PATH:$SC_INSTALL_DIR/riscv-unknown-elf-gcc/bin export FPGEN-
PROG=$LIBERO_INSTALL_DIR/Libero/bin64/fpgenprog

# # Libero # export PATH=$PATH:$LIBERO_INSTALL_DIR/Libero/bin:$LIBERO_INSTALL_DIR/Libero/bin64
export PATH=$PATH:$LIBERO_INSTALL_DIR/Synplify/bin export PATH=$PATH:$LIBERO_INSTALL_DIR/Model/modeltech/linux
export LOCALE=C export LD_LIBRARY_PATH=/usr/lib/i386-linux-gnu:$LD_LIBRARY_PATH

# # Libero License daemon # export LM_LICENSE_FILE=1702@localhost export
SNPSLMD_LICENSE_FILE=1702@localhost

$LIBERO_INSTALL_DIR/Libero/bin64/lmgrd -c $LICENSE_FILE_DIR/License.dat -l $LI-
CENSE_FILE_DIR/license.log
```

I suggest popping the code snippet above in a file called `setup-microchip-tools.sh`, similar to the instructions from Beagle.

before executing Libero you will need to source this script from whatever location you decide to leave it.

```
source setup-microchip-tools.sh
```

after this you will be able to run `libero` from the terminal you sourced the script in. Don't forget to re-source the script when starting a new terminal environment if you want access to the tools.

5.19.2 Support and Acknowledgements

Getting this all to work was a bit of a journey. I received support from the friendly folk over in the [BeagleBone Discord](#). If you come across anything maybe head over there and try your luck!

Thanks to everyone who gave me a hand working through some of the quirks in getting this setup, Leoh in particular!

Written by Lucien Morey - 16/10/24

Chapter 6

Support

All support for BeagleV-Fire design is through BeagleBoard.org community at [BeagleBoard.org forum](https://beagleboard.org/forum).

6.1 Production board boot media

6.2 Certifications and export control

6.2.1 Export designations

- HS: 8471504090
- US HS: 8543708800
- EU HS: 8471707000

6.2.2 Size and weight

- Bare board dimensions: 86.38*54.61*18.8mm
- Bare board weight: 45.8g
- Full package dimensions: 140 x 100 x 40 mm
- Full package weight: 106g

6.3 Additional documentation

6.3.1 Hardware docs

For any hardware document like schematic diagram PDF, EDA files, issue tracker, and more you can checkout the [BeagleV-Fire design repository](#).

6.3.2 Software docs

For BeagleV-Fire specific software projects you can checkout all the [BeagleV-Fire project repositories group](#).

6.3.3 Support forum

For any additional support you can submit your queries on our forum, <https://forum.beagleboard.org/tags/c/beaglev/15/fire>

6.3.4 Pictures

6.4 Change History

Note: This section describes the change history of this document and board. Document changes are not always a result of a board change. A board change will always result in a document change.

6.4.1 Board Changes

For all changes, see <https://git.beagleboard.org/beaglev-fire/beaglev-fire/>. Versions released into production are noted below.

Table 6.1: BeagleV-Fire board change history

Rev	Changes	Date	By
A	Initial production version	2023-11-02	JK